

Exam Summary

Module Name

Student

Last updated: 09. May 2026

Table of Contents

1.	Week 1: [Topic Title]	5
1.1.	Woche 1: Business Intelligence & Data Visualization	5
2.	Week 2: [Topic Title]	12
2.1.	Woche 2: OLAP, Data Warehouses & SQL-Aggregationen	12
3.	Week 3: [Topic Title]	22
3.1.	Woche 3: Spatial Data Analytics — Einführung	22
4.	Week 4: [Topic Title]	33
4.1.	Woche 4: Spatial Data Analytics — PostGIS, Funktionen & Geodatenbanken ...	33
5.	Überblick	34
5.1.	1. Das EVAP-Prinzip bei Geodaten	34
5.2.	2. PostGIS einrichten	34
5.3.	3. Geodatentypen (Vektoren)	36
5.4.	4. Geodaten-Konstrukturen	37
5.5.	5. Koordinatenreferenzsysteme (EPSG-Codes)	38
5.6.	6. Metrische Funktionen (Zahl zurück)	39
5.7.	7. Räumliche Filter-Funktionen (Boolean zurück)	40
5.8.	8. Räumliche Abfrage-Muster (Rezepte für Übungsaufgaben)	41
5.9.	9. Geometrische Verarbeitungs-Funktionen (Geometrie zurück)	44
5.10.	10. Daten laden: Werkzeuge	45
5.11.	11. OpenStreetMap und PostPASS	46
5.12.	12. Spatial Relationships — Konzept	47
5.13.	13. Räumliche Indexe	48
5.14.	14. Geo-Normen und Datenformate	49
5.15.	15. Offene Geodatenquellen	49
5.16.	16. Vollständige Beispiel-Queries (Übungsreferenz)	50
5.17.	17. Zusammenfassung: Entscheidungsbaum für räumliche Abfragen	51
6.	Week 5: [Topic Title]	53
6.1.	Woche 5: Big Data Analytics und Cloud Platforms	53
6.2.	1. Big Data: Grundbegriffe und Definitionen	53
6.3.	2. Analytical Processing-Technologien	54
6.4.	3. Überblick: Data Processing Tools	55
6.5.	4. MapReduce	56
6.6.	5. Apache Spark	57
6.7.	6. Cloud Computing und Cloud Platforms	60
6.8.	7. PySpark Praxis: Vollständiges Lösungsrezept für Übungsaufgaben	62
6.9.	8. Zusammenfassung: Konzept-Landkarte	63
7.	Week 6: [Topic Title]	65
7.1.	Woche 6: Big Data Analytics, Cloud Platforms & Google Earth Engine	65
7.2.	Teil 1: Cloud vs. On-Premise	65
7.3.	Teil 2: Vergleich Cloud Analytics Services	67
7.4.	Teil 3: Spatial Data Analytics & Rasterdaten	68
7.5.	Teil 4: Google Earth Engine (GEE)	69
7.6.	Zusammenfassung: Entscheidungsbaum	74

8.	Week 7: [Topic Title]	75
8.1.	Woche 7: Data Integration — ETL/ELT	75
8.2.	1. Informations- und Datenintegration: Grundbegriffe	75
8.3.	2. Vorgehensweisen und Konflikttypen bei der Datenintegration	76
8.4.	3. Data Warehouse: Architektur und Wartung	77
8.5.	4. ETL — Extract, Transform, Load	78
8.6.	5. ELT — Extract, Load, Transform	80
8.7.	6. Tools für die Datenintegration	81
8.8.	7. Klassifikation von Datenfehlern	82
8.9.	8. Datenfluss: Schematische Übersicht	83
8.10.	9. Zusammenfassung: Entscheidungsmatrix	83
9.	Week 8: [Topic Title]	84
9.1.	Woche 8: Data Integration — Datenaufbereitung, Duplikaterkennung & Apache Hop	84
9.2.	Teil 1: Tools für die Datenintegration	84
9.3.	Teil 2: Datenaufbereitung und Datenreinigung	85
9.4.	Teil 3: Duplikaterkennung und Zeichenkettenvergleich	86
9.5.	Teil 4: Datenabgleich (Data Matching)	88
9.6.	Teil 5: Datenanreicherung (Data Enrichment)	88
9.7.	Teil 6: Synthetische Datengenerierung	89
9.8.	Teil 7: Apache Hop — Praktische ETL-Pipelines	90
9.9.	Zusammenfassung: Konzepte auf einen Blick	96
10.	Week 9: [Topic Title]	98
10.1.	Woche 9: Introduction and Preprocessing (1)	98
11.	Data Science & Data Mining — Grundlagen	99
11.1.	Was ist Data Science und Data Mining?	99
11.2.	CRISP-DM — Der Standard-Prozess	99
11.3.	Die vier Data-Mining-Aufgabentypen	100
12.	Was sind Daten? — Datentypen und Attribute	101
12.1.	Datenstruktur: Objekte und Attribute	101
12.2.	Attributtypen — Das wichtigste Klassifikationssystem	101
13.	Datenqualität und Preprocessing	103
13.1.	Datenqualität — Dimensionen	103
13.2.	Preprocessing-Techniken	103
14.	Statistische Grundlagen — Lagemasse	104
14.1.	Arithmetisches Mittel (Mean)	104
14.2.	Median	104
14.3.	Mode (Modalwert)	105
15.	Statistische Grundlagen — Streumasse	106
15.1.	Range (Spannweite)	106
15.2.	Quantile und Quartile	106
15.3.	Fünf-Zahlen-Zusammenfassung (Five-Number Summary)	107
15.4.	Boxplot	107
15.5.	Varianz und Standardabweichung	107
16.	Datenvisualisierung — Überblick	109
16.1.	Histogramm	109
16.2.	Scatter Plot (Streudiagramm)	109
16.3.	Weitere Visualisierungstypen	109

17. Week 10: [Topic Title]	110
17.1. Week 10: Preprocessing (2) & (3) — Noisy/Missing Data, Correlation, Normalization, Discretization, Data Reduction	110
18. 1. Fehlende Daten (Missing Data)	111
18.1. 1.1 Typen von fehlenden Daten	111
18.2. 1.2 Umgang mit fehlenden Werten	111
19. 2. Noisy Data & Binning zur Glättung	112
19.1. 2.1 Binning-Methoden zur Glättung	112
19.2. 2.2 Ausreisser erkennen und behandeln	113
20. 3. Korrelationsanalyse	114
20.1. 3.1 Pearson Korrelationskoeffizient (für numerische Daten)	114
20.2. 3.2 Chi-Quadrat-Test (für nominale Daten)	115
21. 4. Normalisierung	117
21.1. 4.1 Min-Max-Normalisierung	117
21.2. 4.2 Decimal Scaling	117
21.3. 4.3 Z-Score Normalisierung (Standardisierung)	118
22. 5. Diskretisierung (Discretization)	120
22.1. 5.1 Equidepth Binning (gleich viele Elemente pro Bin)	120
22.2. 5.2 Equiwidth Binning (gleich breite Intervalle)	120
22.3. 5.3 Bin-Grenzen und Zuweisung unbekannter Instanzen	121
22.4. 5.4 Weitere Diskretisierungsmethoden	121
23. 6. Data Reduction	122
23.1. 6.1 Curse of Dimensionality	122
23.2. 6.2 Sampling (Numerosity Reduction)	122
23.3. 6.3 Feature Selection	122
24. Week 11: [Topic Title]	124
24.1. Chapter 1: [Chapter Title]	124
25. Week 12: [Topic Title]	125
25.1. Chapter 1: [Chapter Title]	125
26. Week 13: [Topic Title]	126
26.1. Chapter 1: [Chapter Title]	126
27. Week 14: [Topic Title]	127
27.1. Chapter 1: [Chapter Title]	127

Week 1: [Topic Title]

Woche 1: Business Intelligence & Data Visualization

1. Datenkompetenz (*Data Literacy*)

Kernidee: Datenkompetenz ist die Fähigkeit, Daten kritisch zu sammeln, zu managen, zu bewerten und anzuwenden — nicht als Selbstzweck, sondern für konkrete Entscheidungen.

Die vier Leitfragen der Datenkompetenz:

Tip: Woran erkenne ich Datenkompetenz-Fragen in der Prüfung? Aufgaben die fragen “Darf man diesen Datensatz verwenden?” oder “Was will man mit diesen Daten erreichen?” testen Datenkompetenz.

- **Was will ich?** → Ziel definieren (konkrete Anwendung, nicht Daten um ihrer selbst willen)
- **Was kann ich?** → Datenquellen, Qualität, technische Möglichkeiten und Grenzen prüfen
- **Was darf ich?** → Datenschutz, Urheberrecht, Lizenzfragen klären (z.B. OPEN-BY-ASK beim BFS: freie Nutzung, Quellenangabe Pflicht, kommerzielle Nutzung nur mit Bewilligung)
- **Was soll ich?** → Ethischer Anspruch: Daten zum Wohl von Individuen und Gesellschaft nutzen

2. Datentypen und Skalenniveaus

Kernidee: Jedes Datenmerkmal hat ein Skalenniveau — dieses bestimmt, welche Analysen und Visualisierungen zulässig und sinnvoll sind.

1.1.2.1. Oberste Ebene: Qualitativ vs. Quantitativ

Qualitativ (kategorisch): Beschreibt Eigenschaften, nicht messbare Mengen. Meist Text, kann aber numerisch codiert sein.

- Beispiele: Augenfarbe, Autotyp, Ja/Nein-Antworten, Farbe eines Regals

Quantitativ: Beschreibt zählbare/messbare Mengen. Immer numerisch.

- Beispiele: Entfernung, Geschwindigkeit, Höhe (100 cm), Anzahl Objekte (10 Stück)

Merkhilfe Quiz: “Ein Regal hat 10 Objekte (quantitativ) und ist rot (qualitativ).”

1.1.2.2. Die vier Skalenniveaus — Merkkürzel “N-O-I-R”

N-O-I-R: Die vier Skalenniveaus

1. Nominal — “ein Name, etwas Benanntes”

- Nur Kategorien, keine Reihenfolge, kein Abstand, kein Nullpunkt
- Erlaubte Operationen: Gleich / Ungleich ($=$, $\neq$)

- Datentyp: meist Text
 - Beispiele: Augenfarbe (blau, grün, braun), Geschlecht, Postleitzahl als Kategorie, Automarke
- 2. Ordinal** — “eine Ordnung, ein Rang”
- Kategorien mit sinnvoller Reihenfolge, aber Abstände nicht bedeutsam
 - Erlaubte Operationen: Gleich / Ungleich + Größer/Kleiner ($<$, $>$)
 - Datentyp: meist Ganzzahl oder Text mit Rang
 - Beispiele: Schulnoten (1–6), Zufriedenheitsskala (sehr gut / gut / mittel / schlecht), Militärränge, Platzierungen
- 3. Intervall** — “ein Maßstab, etwas Relatives”
- Geordnete Werte mit gleichmäßigen, bedeutsamen Abständen, aber *kein* absoluter Nullpunkt
 - Erlaubte Operationen: $+$, $-$ (Differenzen sinnvoll), aber *keine* Verhältnisse
 - Datentyp: meist Fließ- oder Festkommazahl
 - Beispiele: Temperatur in $^{\circ}\text{C}$ oder $^{\circ}\text{F}$ (0°C ist nicht “keine Temperatur”), Jahreszahlen, IQ-Werte
- 4. Ratio (Verhältnis)** — “ein absoluter Nullpunkt”
- Wie Intervall, aber mit absolutem Nullpunkt $\rightarrow$ Verhältnisse sinnvoll (“doppelt so schwer”)
 - Erlaubte Operationen: $+$, $-$, $\times$, $\div$ (alle arithmetischen Operationen)
 - Datentyp: meist Fließ- oder Festkommazahl
 - Beispiele: Gewicht, Länge, Entfernung, Temperatur in Kelvin, Bevölkerungszahl, Einkommen
- Hierarchie:** Ratio $\supset$ Intervall $\supset$ Ordinal $\supset$ Nominal (jede höhere Skala enthält alle Eigenschaften der niedrigeren)

Exam Trap: Häufige Verwechslungen:

- Postleitzahlen sind **Nominal**, nicht Ratio — obwohl sie Zahlen sind, hat “8000 + 1 = 8001” keine sinnvolle Bedeutung
- Jahreszahlen sind **Intervall**, nicht Ratio — das Jahr 0 ist kein absoluter Nullpunkt; “2024 ist doppelt so groß wie 1012” ist sinnlos
- Schulnoten sind **Ordinal**, nicht Intervall — der Abstand zwischen Note 1 und 2 ist nicht zwingend gleich dem Abstand zwischen 5 und 6
- Temperaturen in $^{\circ}\text{C}$ sind **Intervall** (0°C = kein Nullpunkt), in Kelvin sind sie **Ratio** (0 K = absoluter Nullpunkt)

Tip: Schnell-Test: Welches Skalenniveau?

Frage 1: Kann man sinnvoll sortieren/vergleichen? $\rightarrow$ Nein $\rightarrow$ **Nominal**

Frage 2 (wenn Ja): Sind die Abstände zwischen Werten bedeutsam und gleichmäßig? $\rightarrow$ Nein $\rightarrow$ **Ordinal**

Frage 3 (wenn Ja): Gibt es einen absoluten Nullpunkt (= “nichts davon”)? $\rightarrow$ Nein $\rightarrow$ **Intervall**; Ja $\rightarrow$ **Ratio**

3. Business Intelligence (BI) und Decision Support Systems (DSS)

Kernidee: BI-Tools helfen Unternehmen, Rohdaten in handlungsleitende Erkenntnisse umzuwandeln; DSS stellen die Methoden bereit, um auf Basis dieser Erkenntnisse Entscheidungen zu treffen.

1.1.3.1. BI-Tools vs. DSS

Business Intelligence (BI) Tools:

- Sammeln, analysieren und visualisieren Daten
- Umfassen: Data Warehousing, Data Mining, Reporting, Dashboards
- Liefern die *Grundlage* für Entscheidungen (Was ist passiert? Was ist der Ist-Zustand?)
- Bekannte Tools: Tableau, Microsoft PowerBI, Looker (Google Data Studio), Apache Superset (Open Source), Oracle Analytics, Datawrapper

Decision Support Systems (DSS):

- Computergestützte Systeme mit Methoden zur Entscheidungsunterstützung
- Helfen Entscheidungsträgern, bessere und fundiertere Entscheidungen zu treffen
- Fokus auf Analyse des Ist-Zustands und zurückblickende Analyse

Unterschied: BI liefert *Daten und Erkenntnisse*; DSS liefert *Werkzeuge und Methoden* für die Entscheidung selbst.

1.1.3.2. Methoden von DSS

Drei Hauptmethoden:

1. Datenbankansatz (inkl. Visualisierung):

- Benutzer konsultieren → konzeptionelles Modell entwickeln → logisches Modell implementieren → erneut konsultieren (iterativ)

2. Data-Mining-Ansatz:

- Lerndatensatz wählen → Modell erstellen → Testdatensatz wählen → Modellgüte berechnen

3. Informationsbeschaffungsansatz:

- Benutzer nach Relevanz befragen → Relevanz durch Retrieval-Algorithmus modellieren → Ergebnisse vergleichen (Recall, Präzision)

1.1.3.3. Gemeinsame Funktionen von BI-Tools

Alle BI-Tools bieten typischerweise:

- **Analysieren:** Daten selektieren, filtern, aufbereiten
- **Visualisieren:** Listen, interaktive Diagramme, Karten
- **Präsentieren:** Cockpits / Dashboards zusammenstellen
- **Publizieren:** Dashboards über das Web teilen
- Dazu: Benutzerverwaltung

1.1.3.4. Evolution von BI

Die Entwicklung verlief grob: 1990er → BI Reporting / OLAP / Data Warehouses → 2000er → Interactive BI / In-Memory → 2010er → Big Data (Batch & Real-Time), Graph Databases

4. Data Visualization (DataViz)

1.1.4.1. DataViz vs. InfoViz

DataViz = Daten grafisch darstellen, damit Menschen Muster schneller erkennen — gute grafische Kodierung (Diagramme, Karten für Analyse oder Kommunikation)

InfoViz (InfoVis) = DataViz + interaktive Systeme: menschliche Wahrnehmung/Kognition, UI/UX-Design, Interaktionstechniken, Evaluation, Datenperspektiven

Merksatz: DataViz = gute grafische Kodierung; InfoVis = gute grafische Kodierung *plus* Interaktion, UI/UX, Kognition als System

1.1.4.2. Datenperspektiven: What / When / Where / With whom

Tip: Woran erkenne ich, welche Perspektive gefragt ist?

- **What? (thematisch):** Vergleiche zwischen Kategorien, Gruppen, Werten → Welche Kategorie ist größer? Wie verteilen sich Werte?
- **When? (temporal):** Zeitliche Entwicklung, Trends, Zeitreihen → Wie verändert sich etwas über die Zeit?
- **Where? (räumlich):** Werte auf Karten, geografische Muster, Hotspots → Wo tritt etwas auf?
- **With whom? (Netzwerk):** Beziehungen und Flüsse zwischen Akteuren/Objekten → Wer hängt mit wem zusammen?

1.1.4.3. Warum Visualisierung wichtig ist

- Menschen sind **schlecht** im Vergleich von Zahlen, aber **gut** im Erkennen grafischer Muster
- Visualisierung ist nicht nur Veranschaulichung, sondern ein eigenständiges **Analysewerkzeug**
- Häufig erkennt man Zusammenhänge in Daten erst durch geschickte Darstellung

1.1.4.4. Die Klassiker: Tufte und Bertin

Edward Tufte (1942–heute): Bekanntester Informationswissenschaftler/Grafikdesigner; Kernprinzip: maximale Informationsdichte bei minimaler grafischer “Tinte” (Data-Ink-Ratio).

Jacques Bertin (1918–2010): Französischer Kartograf; entwickelte die Theorie der **6 grafischen Variablen** (visuelle Variablen):

Bertins 6 grafische Variablen — ein Zeichen kann kodiert werden durch:

1. **Farbe** (Farbton / Hue)
2. **Form** (Shape)
3. **Muster** (Texture/Pattern)
4. **Helligkeit** (Value/Brightness)
5. **Richtung** (Orientation)
6. **Größe** (Size)

Je nach Skalenniveau empfiehlt Bertin unterschiedliche visuelle Variablen:

- Nominal → Farbe, Form, Muster (unterschiedlich, aber nicht geordnet)
- Ordinal → Helligkeit, Größe (hat eine natürliche Ordnung)

- Intervall/Ratio → Größe, Helligkeit (proportionale Abstände darstellbar)

5. Die richtige Grafik wählen

1.1.5.1. Skalenniveau → empfohlene Diagrammtypen

Rezept: Diagrammtyp aus Skalenniveau ableiten

Schritt 1: Bestimme das Skalenniveau der relevanten Variablen (N-O-I-R)

Schritt 2: Bestimme die Botschaft / den Nachrichtentyp (siehe Stephen Few)

Schritt 3: Wähle den passenden Diagrammtyp:

| Skalenniveau | Business Charts | Karten | |—|—|—| | **Nominal** | Torten-, Balkendiagramme | Dot map, Network map, Colored area map | | **Ordinal** | Gestapelte Balken, Punktdiagramme | Symbol map, Ordered network map, Ordered colored map | | **Intervall** | Liniendiagramme, Streudiagramme | Graduated symbol map, Flow map, Choropleth map | | **Ratio** | Histogramme, Boxplots | (wie Intervall) |

1.1.5.2. Die 7 quantitativen Nachrichtentypen (Stephen Few, 2004)

Wenn die Botschaft ist... → dann nimm...

1. **Nominaler Vergleich** (Kategorien vergleichen ohne Reihenfolge) → **Bar Chart** (horizontal oder vertikal)
2. **Zeitserie** (Entwicklung über Zeit) → **Liniendiagramm** (horizontal)
3. **Rangfolge** (geordnete Liste) → **Balkendiagramm** (horizontal)
4. **Teil-zu-Ganzes** (Anteile am Ganzen) → **Bar Chart** (horizontal oder vertikal)
5. **Abweichung** (Differenz von einem Referenzwert) → **Bar Chart** (horizontal oder vertikal)
6. **Häufigkeitsverteilung** (wie oft tritt ein Wert auf) → **Bar Chart** (horizontal)
7. **Korrelation** (Zusammenhang zwischen zwei Variablen) → **Scatter Plot**

Exam Trap: Häufige Fehler bei der Diagrammwahl:

- **Liniendiagramm statt XY-Streudiagramm für Zeitreihen mit unregelmäßigen Abständen:** Ein normales Liniendiagramm verteilt alle Datenpunkte gleichmäßig auf der X-Achse — auch wenn die Zeitabstände ungleich sind! Das führt zu irreführenden Steigungen. Lösung: XY-Streudiagramm (Scatter Plot) mit Linien verwenden, dann entsprechen die horizontalen Abstände den echten Zeitabständen.
- **Tortendiagramm für zu viele Kategorien:** Mehr als 5–6 Segmente sind schwer lesbar. Besser: Balkendiagramm.
- **Defaults der Office-Programme blind übernehmen:** Excel/LibreOffice schlagen oft nicht den optimalen Diagrammtyp vor. Immer selbst nachdenken!
- **3D-Diagramme:** Fast immer irreführend, da perspektivische Verzerrung die Werte verändert.

Tip: Entscheidungsbaum: Scatter Plot vs. Liniendiagramm bei Zeitreihen

Frage: Sind die Zeitabstände zwischen den Messpunkten **gleichmäßig**?

- Ja → Liniendiagramm akzeptabel
- Nein (z.B. Daten von 1950, 1960, 1970... aber auch 1975) → **XY-Streudiagramm mit Linien** verwenden!

Begründung: Nur der Scatter Plot mappt die X-Position proportional zum tatsächlichen Zeitwert.

6. Praktisches Rezept: Business Chart erstellen (Übung 1)**Allgemeines Rezept: Zeitreihen-Diagramm in LibreOffice / Excel erstellen****Schritt 1: Datei öffnen und relevante Daten identifizieren**

- Welche Zeile/Spalte enthält die X-Achsen-Variable (z.B. Jahreszahlen)?
- Welche Zeile(n)/Spalte(n) enthalten die Y-Werte (z.B. Bevölkerungszahlen)?
- Beschriftungen (Labels) notieren

Schritt 2: Daten auswählen

- In LibreOffice: Ctrl+Klick auf Zeilennummern für mehrere nicht-benachbarte Zeilen
- Beschriftungsspalte/-zeile mit einschließen

Schritt 3: Diagramm einfügen

- Menü: Einfügen → Diagramm...

Schritt 4: Richtigen Diagrammtyp wählen

- Für Zeitreihen mit ungleichen Abständen: **XY (Streudiagramm)** → Variante “Punkte und Linien” oder “Nur Linien”
- Für Zeitreihen mit gleichen Abständen: Liniendiagramm möglich

Schritt 5: Datenbereich konfigurieren

- “Datenreihen in Zeilen” anwählen (wenn Daten zeilenweise angeordnet)
- “Erste Zeile als Beschriftung” anwählen
- “Erste Spalte als Beschriftung” anwählen

Schritt 6: Fertigstellen und optimieren

- Achsenbeschriftungen prüfen (Einheiten angeben!)
- Diagramm strecken wenn nötig (Datenpunkte besser sichtbar)
- Legende prüfen (sind alle Datenreihen klar beschriftet?)

Schritt 7: Diagrammwahl begründen

- Warum dieser Typ? (z.B. “XY-Diagramm, weil die Zeitabstände ungleichmäßig sind und nur so die Steigungen korrekt dargestellt werden”)

Beispiel (Bevölkerungsdaten CH 1950–2024):

- X-Achse: Jahreszahlen (Zeile 2) → ungleichmäßige Abstände → XY-Streudiagramm!
- Y-Achse 1: Ständige Wohnbevölkerung in 1'000 (Zeile 4)
- Y-Achse 2: Davon Ausländer in 1'000 (Zeile 5)
- Botschaft: Zeitserie → Liniendiagramm-Variante des XY-Diagramms

Tip: Woran erkenne ich, welcher Diagrammtyp in der Aufgabe gefragt ist?

- Aufgabe nennt “zeitliche Entwicklung” / “Trend” / “über die Jahre” → **Zeitreihe** → Liniendiagramm oder XY-Scatter mit Linien
- Aufgabe nennt “Vergleich zwischen Kategorien” → **Nominaler Vergleich** → Bar Chart
- Aufgabe nennt “Anteile” / “Prozent von Gesamt” → **Teil-zu-Ganzes** → Bar Chart (oder Pie Chart bei wenigen Kategorien)
- Aufgabe nennt “Zusammenhang zwischen zwei numerischen Variablen” → **Korrelation** → Scatter Plot
- Aufgabe nennt “Häufigkeit” / “wie oft” → **Verteilung** → Histogramm / Bar Chart

7. Apache Superset — BI-Tool Grundlagen (Übung 2 & 3)

Kernidee: Apache Superset ist ein Open-Source-BI-Tool, das den kompletten Workflow von Datenquelle über Chart bis zum veröffentlichten Dashboard abdeckt.

Workflow in Apache Superset:

1. **Datenbankverbindung herstellen** (oder CSV hochladen)
2. **Dataset definieren** (Tabelle/View aus der Datenbank auswählen)
3. **Chart erstellen:** Charttyp wählen → X-Achse / Y-Achse / Dimensionen konfigurieren → speichern
4. **Dashboard erstellen:** Charts zum Dashboard hinzufügen
5. **Dashboard publizieren:** Link teilen (Dashboard → Publish → Link kopieren)

CSV-Upload: Für eigene Datensätze (Aufgabe 3) muss die CSV-Datei zuerst in Superset hochgeladen werden (Datenbank-Upload-Funktion), bevor sie als Dataset verwendet werden kann.

Wichtig beim CSV: Trennzeichen (Komma / Semikolon), Zeichenkodierung (UTF-8), erste Zeile als Header sicherstellen.

8. Zusammenfassung: Entscheidungsmatrix

Wenn ich eine Aufgabe sehe die fragt...

“Welches Skalenniveau?” → N-O-I-R-Test (sortierbar? gleichmäßige Abstände? absoluter Nullpunkt?)

“Welches Diagramm?” → Erst Botschaft bestimmen (Few’s 7 Typen), dann Skalenniveau prüfen

“Warum XY-Streudiagramm statt Liniendiagramm?” → Ungleichmäßige Zeitabstände: XY mappt X-Position proportional zum echten Wert

“Darf ich diese Daten nutzen?” → Datenkompetenz-Check: Was will/kann/darf/soll ich?

“Was ist BI? Was ist DSS?” → BI = Daten & Erkenntnisse; DSS = Methoden & Werkzeuge für Entscheidungen; BI liefert Input für DSS

Week 2: [Topic Title]

Woche 2: OLAP, Data Warehouses & SQL-Aggregationen

1. OLTP vs. OLAP — Grundunterschied

Kernidee: Zwei fundamental verschiedene Datenbanknutzungsarten — transaktional (viele kurze Schreiboperationen) vs. analytisch (wenige, komplexe Leseoperationen auf historischen Daten).

Wenn Aufgabe fragt: “OLTP oder OLAP?”

- Viele kurze Transaktionen, mission-critical, normalisiert, aktuell → **OLTP**
- Komplexe Analysen, historische Daten, Redundanz erlaubt, Ad-hoc-Queries → **OLAP**

Merkmal	OLTP	OLAP		Anfragen	häufig, einfach	selten, komplex
Datenvolumen pro Anfrage	klein	gross		Daten	aktuell	historisch + aktuell
Optimierung	schneller Update	schnelle Kalkulation		Normalisierung	hoch (3NF)	niedrig erlaubt

Warum getrennte Systeme? Paralleles Ausführen von OLAP-Anfragen auf OLTP-Datenbeständen beeinträchtigt die Performance der OLTP-Anwendungen. Ein DB-System kann nicht gleichzeitig für beide optimiert werden.

2. Data Warehouse — Architektur und Begriffe

Kernidee: Ein Data Warehouse (DW) ist ein separates, für Analysen optimiertes Datenbanksystem, das Daten aus verschiedenen operativen Quellen konsolidiert, bereinigt und historisiert speichert.

2.1.2.1. 2.1 Gesamtarchitektur (OLAP-Systemüberblick)

OLTP-Systeme (operative SQL, Quellen) BI-Tools)	→ ETL (Extract, Transform, Load)	→ Data Warehouse (relational, Star/Snowflake)	→ Data Marts / OLAP Cubes (vorberechnete Aggregate, materialisierte Views)	→ Reporting (Ad-hoc MDX,
--	--	--	--	-----------------------------

ETL (Extract-Transform-Load):

- **Extract:** Daten aus heterogenen Quellen extrahieren
- **Transform:** Bereinigen, vereinheitlichen, anreichern
- **Load:** Ins DW laden und aktualisieren

Business Intelligence (BI): Übergeordneter Begriff — Skills, Tools und Anwendungen für besseres Geschäftsverständnis (z.B. Apache Superset, Tableau).

Woran erkenne ich eine DW-Entwurfsaufgabe?

- Aufgabe nennt mehrere operative Quellsysteme
- Gefordert sind Auswertungen nach verschiedenen Dimensionen (Zeit, Produkt, Region...)
- Vergleiche mit Vorperioden, Budgetvergleiche
- Grosse historische Datenmengen

3. Star-Schema und Snowflake-Schema

Kernidee: Das Star-Schema ist das zentrale Datenmodell im Data Warehouse — eine zentrale Faktentabelle mit numerischen Messwerten, umgeben von denormalisierten Dimensionstabellen.

2.1.3.1. 3.1 Faktentabelle

- Enthält **numerische, aggregierbare Messwerte** (Measures/Fakten): z.B. Umsatz, Anzahl, Preis
- Enthält **Fremdschlüssel** auf alle Dimensionstabellen
- Typischerweise sehr gross (Millionen von Zeilen)
- So granular wie möglich aufbauen!

2.1.3.2. 3.2 Dimensionstabellen

- Beschreiben die Fakten (Kontext): Zeit, Produkt, Kunde, Region, Kanal
- Oft **denormalisiert** (Star) oder **normalisiert** (Snowflake)
- Haben in der Regel einen **Surrogatschlüssel** (künstlicher PK, unabhängig vom Businesskey)
- Klein bis mittelgross (1000 Zeilen)

Rezept: Star-Schema entwerfen

1. **Analysiere geforderte Auswertungen:** Welche Kennzahlen (= Fakten) sollen berechnet werden? Welche Gruppierungskriterien (= Dimensionen) kommen vor?
2. **Bestimme Faktentabelle(n):** Eine pro Geschäftsprozess (z.B. Verkauf, Lagerbestand, Budget)
3. **Bestimme Granularität:** Was entspricht genau einer Zeile? (z.B. ein Verkaufsposten = 1 Zeile)
4. **Bestimme Dimensionen:** Für jede Gruppierungsdimension eine Tabelle (Zeit, Produkt, Kunde, ...)
5. **Bestimme Fakten-Attribute:** Numerische Werte in der Faktentabelle (Anzahl, Preis, Betrag)
6. **Füge Dimensionsattribute ein:** Hierarchien flach in Dimensionstabelle (Star) oder normalisiert (Snowflake)
7. **Zeichne das Schema:** Faktentabelle zentral, Dimensionen als Satelliten

Beispiel (Video-Shop):

- Auswertung: Verkaufszahlen nach Film, Genre, Kunde, Datum, Monat, Jahr
- Faktentabelle `fact_sales`: `sale_id`, `sale_detail_id`, `movie_id` (FK), `cust_id` (FK), `time_id` (FK), `quantity`, `sales_amount`
- Dimensionen: `dim_movie` (`movie_id`, `name`, `price`, `subgenre`, `genre`), `dim_customer` (`cust_id`, `name`, `city`, `gender`, `decade`, ...), `dim_time` (`time_id`, `date`, `month`, `season`, `year`)

Erkennungsmerkmale für Star vs. Snowflake:

- Dimensionsattribute mit Hierarchien **flach in einer Tabelle** (z.B. Ort, Region, Land alle in `dim_customer`) → **Star-Schema**
- Hierarchieebenen in **separaten Tabellen** mit FK-Verknüpfungen (z.B. `dim_ort` → `dim_region` → `dim_land`) → **Snowflake-Schema**
- Star: einfachere Abfragen (1-stufige Joins), mehr Redundanz
- Snowflake: weniger Redundanz, komplexere Joins

2.1.3.3. 3.3 Klassifizierung von Faktentabellen**Wenn Aufgabe fragt: “Welcher Faktentabellentyp?”**

- Jede Zeile = ein Ereignis zu einem Zeitpunkt (z.B. Verkauf) → **Transaction Fact Table**
- Jede Zeile = kumulativer Stand zu regelmässigen Intervallen (z.B. Lagerbestand jeden Abend) → **Periodic Snapshot Fact Table**
- Jede Zeile = Geschäftstransaktion mit mehreren Zeitpunkten (z.B. Bestellung: Eingang → Lieferung → Rechnung) → **Accumulating Snapshot Fact Table**

Beispiele:

- `fact_sales` (Videoverkäufe): Transaction Fact Table — jede Zeile ist ein Verkaufsposten
- `fact_stock` (Lagerbestand täglich): Periodic Snapshot — Bestand wird täglich um 23h erfasst
- Bestellungsabwicklung mit mehreren Schritten: Accumulating Snapshot

2.1.3.4. 3.4 Spezielle Dimensionen

Degenerate Dimension: Kein eigene Tabelle — die Dimension lebt als Attribut in der Faktentabelle.

- Beispiel: `BestellItemNr` in `FaktBestellung` (identifiziert den Order-Header, braucht keine eigene Tabelle)

Role-Playing Dimension: Eine Dimension wird von der Faktentabelle **mehrfach in verschiedenen Rollen** referenziert.

- Beispiel: `DimZeit` wird zweimal referenziert — einmal als `BestellZeit_ID`, einmal als `LieferZeit_ID`

Junk Dimension: Sammelt korrelierte Flags und Indikatoren, die sonst die Faktentabelle aufblähen würden.

Conformed Dimension: Eine Dimension (z.B. Zeit, Geographie) die von **mehreren Faktentabellen** gemeinsam genutzt wird und überall dieselbe Bedeutung hat. Ermöglicht Abfragen über mehrere Faktentabellen.

- Beispiel: `DimZeit` und `DimWeinart` sind in beiden Sternen (Bestellstern und Budgetstern) vorhanden.

Häufiger Fehler: Surrogatschlüssel vergessen

Dimensionstabellen brauchen immer einen künstlichen Surrogatschlüssel (z.B. `Zeit_ID`, `Produkt_ID`) als PK, NICHT den natürlichen Businesskey. Warum? Bei SCD Typ 2 können mehrere Zeilen für dieselbe Entität existieren (verschiedene Historienversionen) — der Businesskey wäre dann nicht mehr eindeutig.

4. *Slowly Changing Dimensions (SCD)*

Kernidee: Dimensionsattribute können sich im Laufe der Zeit ändern (z.B. Kunde zieht um, Preis ändert sich) — SCD definiert wie mit dieser Änderung umgegangen wird.

2.1.4.1. SCD-Typen im Überblick

Wenn Aufgabe fragt: “Wie Änderungen in Dimension behandeln?”

- Alte Werte nicht relevant, einfach überschreiben → **SCD Typ 1** (Overwrite)
- Historienerhaltung wichtig, mehrere Versionen einer Entität → **SCD Typ 2** (neue Zeile)
- Nur aktueller + ein vorheriger Wert nötig → **SCD Typ 3** (zusätzliche Spalte)

2.1.4.2. SCD Typ 2 (der wichtigste)

Vorgehen:

1. Neue Zeile für den Datensatz einfügen mit neuen Attributwerten
2. Alte Zeile bleibt erhalten (historische Version)
3. Gültigkeitsbereich speichern: Attribute `valid_from`, `valid_to` (oder `start_date`, `end_date`)
4. Surrogat-PK wird erweitert (Business-Key allein nicht mehr eindeutig)
5. Optional: Status-Attribut 'Active' / 'Inactive'

```
-- Beispiel SCD Typ 2: Kunde wechselt Stadt per 1.2.2013
-- Schritt 1: Alte Zeile deaktivieren
UPDATE dim_customer
SET valid_to = '2013-01-31', status = 'Inactive'
WHERE cust_id_bk = 42 AND status = 'Active';

-- Schritt 2: Neue Zeile einfügen
INSERT INTO dim_customer (cust_id_bk, name, city, valid_from, valid_to, status)
VALUES (42, 'Marxer, Markus', 'Zuerich', '2013-02-01', '9999-12-31', 'Active');
```

Auswertung mit SCD Typ 2:

- **Über Business-Key** (ignoriert Historisierung): `GROUP BY cust_id_bk` — gibt Gesamtumsatz aller Perioden
- **Über Surrogatschlüssel** (historisierend): `JOIN` mit dem historisch korrekten Dim-Eintrag via Business-Key UND `valid_from/valid_to` — gibt Umsatz nur für bestimmte Wohnadresse

Häufiger Fehler: Beim ETL-Prozess falschen Dim-Eintrag nehmen

Beim Laden der Faktentabelle muss das **zeitlich korrekte Tupel** der Dimensionstabelle gefunden werden: `WHERE dim.cust_id_bk = oltp.cust_id AND oltp.sale_date BETWEEN dim.valid_from AND dim.valid_to`

Wenn man nur nach Business-Key joined, nimmt man eventuell den falschen (neueren) Eintrag.

2.1.4.3. Zeitdimension

Die Zeitdimension wird typischerweise **vorab vollständig befüllt** (alle Tage eines Betrachtungszeitraums), nicht nur aus Transaktionen abgeleitet. Sie enthält alle relevanten Zeitattribute als eigene Spalten:

ZeitId	Datum	Jahr	Monat	Tag	MonatsName	Quartal	Halbjahr
1	2004-05-01	2004	5	1	May	Q 2 2004	H 1 2004

Warum? Damit Abfragen wie “alle Verkäufe im Q2” einfach mit `WHERE quartal = 'Q 2 2004'` formuliert werden können, ohne Datumsfunktionen.

Wichtig: KW-Hierarchie und Monats-Hierarchie können sich widersprechen (eine KW kann in zwei Monate fallen) → mehrere Hierarchien in der Zeitdimension nötig.

—

5. SQL-Aggregationen für OLAP

2.1.5.1. 5.1 GROUP BY (Basis)

Kernidee: Teilt die Ergebnismenge in Gruppen auf und berechnet pro Gruppe einen aggregierten Wert.

```
SELECT col1, col2, AGG_FUNC(measure)
FROM tabelle
GROUP BY col1, col2
ORDER BY col1, col2;
```

Verfügbare Aggregatfunktionen: `SUM()`, `COUNT()`, `MIN()`, `MAX()`, `AVG()`

2.1.5.2. 5.2 GROUP BY ROLLUP

Kernidee: Berechnet hierarchische Zwischensummen von rechts nach links plus eine Gesamtsumme — ersetzt mehrere UNION ALL-Queries.

Formel: Bei `ROLLUP(A, B, C)` entstehen Gruppierungen: (A, B, C) , (A, B) , (A) , $()$

Rezept: ROLLUP anwenden

- Bestimme die Hierarchie der Attribute (von spezifisch zu allgemein)
- Reihenfolge im ROLLUP: spezifischstes Attribut zuerst, dann aufsteigend genereller
- `GROUP BY ROLLUP(A, B)` erzeugt:
 - Zeilen mit (A, B) = normale Gruppierung
 - Zeilen mit (A, NULL) = Subtotal pro A (über alle B)
 - Zeile mit $(\text{NULL}, \text{NULL})$ = Gesamttotal
- `NULL` in einer Spalte bedeutet: “diese Dimension wurde aggregiert”

Beispiel: Verkäufe nach Jahr und Monat mit Jahrestotals:

```
SELECT year, month, COUNT(*) AS verkauefe
FROM fact_sales
JOIN dim_time ON fact_sales.time_id = dim_time.time_id
GROUP BY ROLLUP(year, month)
ORDER BY year, month;
```

Ergebnis: Zeilen (2015, Jan, 50), (2015, Feb, 30), ..., (2015, NULL, 480), (NULL, NULL, 950)

Äquivalenz: `GROUP BY ROLLUP(A, B)` entspricht:

GROUP BY GROUPING SETS ((A, B), (A), ())

2.1.5.3. 5.3 GROUP BY CUBE

Kernidee: Berechnet Subtotals für **alle möglichen Kombinationen** der angegebenen Attribute — wie ROLLUP aber symmetrisch.

Formel: Bei n Attributen gibt es 2^n Kombinationen (inklusive Gesamttotal):
 $\text{CUBE}(A, B, C) \rightarrow 2^3 = 8$ Kombinationen

Die Kombinationen für $\text{CUBE}(A, B, C)$ sind: $(A, B, C), (A, B), (A, C), (B, C), (A), (B), (C), ()$

Rezept: CUBE anwenden

1. Bestimme welche Attribute **alle möglichen Kombinationen** von Subtotals brauchen
2. **GROUP BY CUBE**(A, B, C) — alle 2^n Kombinationen werden berechnet
3. Unterschied zu ROLLUP: CUBE erzeugt AUCH (B, NULL, NULL) und (NULL, C, NULL) — also Totals für B allein und C allein
4. Wann CUBE statt ROLLUP? Wenn keine natürliche Hierarchie zwischen den Dimensionen besteht

Beispiel: Verkäufe nach Subgenre und Geschlecht — mit Totals pro Subgenre, pro Geschlecht und Gesamttotal:

```
SELECT m.subgenre, c.gender, COUNT(*) AS anzahl
FROM fact_sales fs
JOIN dim_movie m ON fs.movie_id = m.movie_id
JOIN dim_customer c ON fs.cust_id = c.cust_id
GROUP BY CUBE(m.subgenre, c.gender)
ORDER BY 1, 2;
```

ROLLUP vs. CUBE vs. GROUPING SETS — wann was?

- **ROLLUP(A, B, C):** Hierarchische Aggregation ($A > B > C$), z.B. Jahr > Quartal > Monat. Erzeugt $n + 1$ Gruppierungen.
- **CUBE(A, B, C):** Alle möglichen Kombinationen, keine Hierarchie. Erzeugt 2^n Gruppierungen.
- **GROUPING SETS(...):** Explizit aufgezählt welche Kombinationen gebraucht werden. Maximale Kontrolle.

Prüfungsfalle: Bei ROLLUP kommt z.B. (NULL, Jahr) NICHT vor — NULL steht immer an der Stelle des aggregierten "allgemeineren" Attributs.

2.1.5.4. 5.4 GROUP BY GROUPING SETS

Kernidee: Explizit angeben, welche Zwischensummen benötigt werden — maximale Flexibilität.

```
-- Nur Totals pro Modell und pro Jahr (kein Gesamttotal, keine Kombination):
GROUP BY GROUPING SETS (modell, jahr)
```

```
-- Entspricht ROLLUP(modell, jahr):
GROUP BY GROUPING SETS ((modell, jahr), modell, ())
```

2.1.5.5. 5.5 GROUPING()-Funktion

Gibt eine Bitmaske zurück, die anzeigt welche Attribute im aktuellen Gruppierungssatz **nicht** enthalten sind (= aggregiert wurden).

```
SELECT modell, jahr, SUM(anzahl), grouping(modell, jahr) AS g
FROM autoverkaeufe
GROUP BY ROLLUP(modell, jahr);
-- g=0: beide Attribute in Gruppe (normale Zeile)
-- g=1: nur modell in Gruppe (Jahr wurde aggregiert)
-- g=3: keine Attribute in Gruppe (Gesamttotal), 3 = binär 11
```

Häufiger Fehler: NULL-Bedeutung verwechseln

NULL in einer Aggregationsspalte bedeutet NICHT “kein Wert” — es bedeutet “diese Dimension wurde zu einer Gesamtsumme aggregiert” (ALL). Um echte NULLs von Aggregations-NULLs zu unterscheiden: GROUPING(col) verwenden.

6. Pivot-Tabellen

Kernidee: Zeilen in Spalten umwandeln — Werte einer Spalte werden zu eigenen Spalten mit aggregierten Werten.

2.1.6.1. 6.1 Mit FILTER-Klausel (SQL:2003)

```
SELECT modell,
    SUM(anzahl) FILTER (WHERE jahr = 1990) AS "1990",
    SUM(anzahl) FILTER (WHERE jahr = 1991) AS "1991",
    SUM(anzahl) FILTER (WHERE jahr = 1992) AS "1992"
FROM autoverkaeufe
GROUP BY modell;
```

Problem: Statisches Pivot — alle Spaltenwerte müssen manuell aufgezählt werden.

2.1.6.2. 6.2 Mit PostgreSQL crosstab

```
-- Extension einmalig installieren:
CREATE EXTENSION tablefunc;

-- Pivot-Abfrage:
SELECT *
FROM crosstab(
    'SELECT modell, jahr, SUM(anzahl)
    FROM autoverkaeufe
    GROUP BY modell, jahr ORDER BY 1,2',
    'SELECT DISTINCT jahr FROM autoverkaeufe ORDER BY 1'
) AS tmp(modell text, "1990" int, "1991" int, "1992" int);
```

Achtung bei PostgreSQL crosstab: Die Spaltennamen im AS tmp(...) müssen manuell angegeben werden (statisches Pivot). Für dynamisches Pivot braucht man andere Techniken (z.B. JSON, dynamisches SQL).

2.1.6.3. 6.3 Mit CASE (universell, alle DBMS)

```
SELECT modell,
       SUM(CASE WHEN jahr = 1990 THEN anzahl ELSE 0 END) AS "1990",
       SUM(CASE WHEN jahr = 1991 THEN anzahl ELSE 0 END) AS "1991"
FROM   autoverkaeufe
GROUP BY modell;
```

7. Data Marts und Materialisierungs-Hierarchie

Kernidee: Ein Data Mart ist eine spezialisierte, vorberechnete Teilmenge des DW für eine bestimmte Abteilung oder Analyse — kleiner, schneller, optimierter.

2.1.7.1. Materialisierungs-Hierarchie (Lattice/Verband)

Zeigt welche vorberechneten Aggregate wiederverwendet werden können:

```
{ Modell, Jahr, Farbe }
  /      |      \
{Modell,Jahr} {Modell,Farbe} {Jahr,Farbe}
  |      \ /      |      \ /      |
{Modell}  {Jahr}      {Farbe}
      \      |      /
        { }
```

Regel: Teilaggregat T ist wiederverwendbar für Aggregation A , wenn ein gerichteter Pfad von T nach A existiert (d.h. A ist allgemeiner als T).

Beispiel: {Modell, Jahr} kann zur Berechnung von {Modell} und {} wiederverwendet werden — aber NICHT für {Jahr, Farbe}.

8. Data Cube (OLAP Cube)

Kernidee: Mehrdimensionale Datenstruktur mit vorberechneten Aggregaten auf allen Hierarchieebenen aller Dimensionen — optimal für interaktive OLAP-Analysen.

2.1.8.1. Bestandteile eines Cubes

- **Measures (Fakten):** Numerische Kennzahlen (Umsatz, Kosten, Gewinn). Oft eigene Dimension (“Kennzahlen-Dimension”)
- **Dimensionen:** Beschreibende Achsen (Zeit, Produkt, Region). Jede Dimension hat Hierarchien.
- **Hierarchien:** z.B. Tag → Woche → Monat → Quartal → Jahr
- **Members (Ausprägungen):** Konkrete Werte einer Dimension (z.B. “Januar”, “Merlot”)
- **Zellen:** Schnittpunkt aller Dimensionen = ein Messwert

2.1.8.2. OLAP-Operationen auf dem Cube

OLAP-Operationen — wann welche?

- **Drill-Down:** Detaillierungsgrad erhöhen, z.B. von Quartalen zu Monaten. “Zoom in.”
- **Roll-Up:** Detaillierungsgrad verringern, z.B. von Monaten zu Quartalen. “Zoom out.”
- **Slice:** Eine Dimension auf einen Wert fixieren, z.B. nur Region = “Schweiz”. Gibt 2D-Querschnitt.
- **Dice:** Mehrere Dimensionen einschränken, z.B. Region IN (‘CH’, ‘D’) AND Zeit = Q1. Gibt Sub-Cube.
- **Pivot (Rotate):** Achsen austauschen, Zeilen und Spalten vertauschen.

2.1.8.3. Cube-Dimensionalität

- **0D Cube:** Ein Punkt (Gesamtsumme)
- **1D Cube:** Eine Linie (GROUP BY eine Dimension)
- **2D Cube:** Kreuztabelle (GROUP BY zwei Dimensionen)
- **3D Cube:** Würfel (GROUP BY drei Dimensionen)

Abfragesprache: MDX (Multidimensional Expressions, proprietär von Microsoft) — Alternative zu SQL für Cube-Abfragen.

9. Analytische Funktionen (Window Functions)

Kernidee: Window-Funktionen berechnen einen Wert pro Zeile basierend auf einer “Fenstermenge” anderer Zeilen — ohne die Ergebnismenge zu reduzieren (im Gegensatz zu GROUP BY).

Syntax-Grundstruktur:

```
FUNKTION() OVER (
  [PARTITION BY col1, col2] -- Gruppen definieren
  [ORDER BY col3]           -- Reihenfolge innerhalb Partition
  [ROWS/RANGE BETWEEN ...] -- Fenstergrösse einschränken
)
```

2.1.9.1. 9.1 Ranking-Funktionen

```
SELECT name, salaer,
  ROW_NUMBER() OVER (ORDER BY salaer DESC) AS nr,    -- immer eindeutig
  RANK()        OVER (ORDER BY salaer DESC) AS rank,  -- Lücken bei Gleich
  DENSE_RANK() OVER (ORDER BY salaer DESC) AS drank, -- keine Lücken
  NTILE(4)      OVER (ORDER BY salaer DESC) AS quartil -- in N Gruppen
FROM angestellter;
```

RANK vs. DENSE_RANK vs. ROW_NUMBER:

Bei Gehältern 5000, 4000, 4000, 3000:

- **RANK:** 1, 2, 2, 4 (Lücke nach Gleichstand)
- **DENSE_RANK:** 1, 2, 2, 3 (keine Lücke)
- **ROW_NUMBER:** 1, 2, 3, 4 (immer eindeutig)

Rezept: Top-N-Abfrage mit RANK()

1. CTE erstellen der die Ränge berechnet
2. Äussere Abfrage filtert auf rang <= N

```
WITH ranking AS (  
    SELECT name, SUM(sales_amount) AS umsatz,  
           RANK() OVER (ORDER BY SUM(sales_amount) DESC) AS rang  
    FROM fact_sales  
    JOIN dim_customer ON fact_sales.cust_id = dim_customer.cust_id  
    GROUP BY name  
)  
SELECT * FROM ranking WHERE rang <= 3;
```

2.1.9.2. 9.2 PARTITION BY — Rang innerhalb Gruppen

```
SELECT abtnr, name, salaer,  
       SUM(salaer) OVER (PARTITION BY abtnr) AS abt_total,  
       RANK() OVER (PARTITION BY abtnr ORDER BY salaer DESC) AS rang_in_abt  
FROM angestellter;
```

Unterschied GROUP BY vs. PARTITION BY:

- GROUP BY: gibt ein Ergebnistupel pro Gruppe (reduziert Zeilen)
- PARTITION BY: gibt ein Ergebnistupel pro Originalzeile (Zeilen bleiben erhalten, Wert aus Fenster wird angehängt)

2.1.9.3. 9.3 Year-to-Date (YTD)

```
SELECT year, month, SUM(sales_amount) AS betrag,  
       SUM(SUM(sales_amount)) OVER (  
           PARTITION BY year  
           ORDER BY month  
       ) AS ytd  
FROM fact_sales  
JOIN dim_time ON fact_sales.time_id = dim_time.time_id  
GROUP BY year, month;
```

Beachte: SUM(SUM(...)) — die innere SUM ist die Aggregation des GROUP BY, die äussere SUM ist die Window-Funktion die kumuliert.

2.1.9.4. 9.4 Moving Average (gleitender Durchschnitt)

```
SELECT monat, SUM(sales_amount) AS betrag,  
       ROUND(AVG(SUM(sales_amount)) OVER (  
           ORDER BY monat  
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
       ), 0) AS moving_avg_3  
FROM fact_sales  
JOIN dim_time ON ...  
GROUP BY monat;
```

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW = die aktuelle Zeile plus die 2 vorherigen

Week 3: [Topic Title]

Looking at the errors, I can see two issues:

1. Lines 61-64: The `*E*`, `*V*`, `*A*`, `*P*` pattern with bold single letters is causing “unclosed delimiter” errors because Typst interprets `*E*` as bold markup, and when combined with the list items, something is going wrong. Actually, looking more carefully, the issue might be that `*E*rfassung` etc. are fine in normal text but the error points to position 2 (the `*`). Let me re-examine - the error says “unclosed delimiter” at the `*` character position. This suggests the `*` bold markers are somehow unclosed. But they look balanced... The real issue might be something earlier on line 60 or before that causes a parsing problem cascading.
2. Line 548: `#tip-box[` has an unclosed bracket - the content at the very end is cut off: `2. Vektor oder Raster? (best - the tip-box is never closed!`

Let me fix both issues - the tip-box needs to be closed, and I'll check if the bold markers on lines 61-64 need escaping. Since the errors show unclosed delimiter at position 2 (the `*`), and looking at the list items - `*E*rfassung`, these should be fine in Typst. The cascade might be from the unclosed tip-box at the end. Let me just close the tip-box and see.

Woche 3: Spatial Data Analytics — Einführung

Überblick und Lernziele

Nach dieser Woche kannst du:

- Erklären, was Geodaten sind und warum sie besonders sind
- Koordinatenreferenzsysteme unterscheiden und korrekt einsetzen
- Vektor- vs. Rasterdaten unterscheiden und ihre Vor-/Nachteile nennen
- GIS-Architekturen und Produkte einordnen
- GIS-Dateiformate erkennen und das richtige wählen
- Einfache räumliche Analysen mit Vektordaten durchführen (Übungsaufgabe: Autobahn)

1. Was ist Spatial Data Analytics?

Kernidee: Daten mit geografischem Bezug werden mit speziellen Methoden analysiert, um räumliche Muster, Verteilungen und Beziehungen zu erkennen — also die Frage “Wo?” mit “Was?” und “Wann?” zu verbinden.

Geodatenanalyse untersucht Daten mit einer geografischen/räumlichen Komponente mittels:

- visueller Methoden (Karten, Dashboards)
- analytischer Methoden (räumliche Abfragen, Joins)
- statistischer Methoden (Clusterung, Hotspot-Analysen, Zeitreihenmodelle)

Tip: Woran erkenne ich eine Spatial-Analytics-Aufgabe?

- Die Aufgabe enthält Koordinaten, Adressen, oder Begriffe wie “in der Nähe von”, “innerhalb von”, “überschneidet”
- Es geht um Karten, Layer, Polygone, Linien oder Punkte

- Es wird nach räumlichen Beziehungen gefragt (z.B. Puffer, Verschneidung, Distanz)

Typische Anwendungen:

- Klimaforschung: Hitzeinsel-Analyse
- Verkehr: Routing, Flottenmanagement, selbstfahrende Autos
- eGovernment: Stadtplanung, Ressourcenverwaltung
- Gesundheit: Krankheitslokalisation
- Logistik: “Geo-Targeted Marketing”

Spatial Analytics Entscheidungsbaum:

- Frage “Wo ist X?” → Geocoding / Punktdaten
- Frage “Was ist in der Nähe von Y?” → Pufferanalyse
- Frage “Welche Objekte überschneiden sich?” → Verschneidung (Intersection)
- Frage “Wie komme ich von A nach B?” → Routing / Netzwerkanalyse
- Frage “Wo gibt es Häufungen?” → Cluster-/Hotspot-Analyse

2. Geoinformationssysteme (GIS)

Kernidee: Ein GIS ist ein spezialisiertes Informationssystem für Geodaten — es deckt den gesamten Lebenszyklus von Geodaten ab: Erfassen, Verwalten, Analysieren und Präsentieren.

3.1.3.1. Definition und EVAP-Prinzip

Ein GIS ist ein Informationssystem zur:

- **Erfassung** (Eingabe)
- **Verwaltung** (Speicherung)
- **Analyse**
- **Präsentation** (Darstellung)

...aller Daten, die einen Teil der Erdoberfläche und die darauf befindlichen Einrichtungen beschreiben.

EVAP-Prinzip anwenden:

1. **Erfassung:** GPS/GNSS (outdoor), WLAN/Bluetooth/SLAM (indoor), Digitalisierung, Fernerkundung
2. **Verwaltung:** Geodatenbank (PostgreSQL+PostGIS, Oracle, MySQL, DuckDB) mit Geodatentypen, Funktionen, 2D-Indexen
3. **Analyse:** Räumliche Abfragen, Verschneidungen, Puffer, Routing
4. **Präsentation:** Karte, Businessgrafik, Tabelle, Webkarte

Beispiel: Kebab-Restaurants in Rapperswil finden:

- E: OSM-Daten (Koordinaten + Tags)
- V: PostGIS-Datenbank mit Geometriespalte
- A: SQL-Abfrage mit ST_Distance / Bounding Box Filter
- P: Webkarte mit Leaflet oder GeoJSON auf geojson.io

3.1.3.2. Datenschutz-Hinweis (Indoor-Lokalisierung)

Exam Trap: Datenschutz bei Geodaten: Studie von Montjoye et al. zeigt: nur **4 räumlich-zeitliche Datenpunkte** (Zeit + Ort) reichen, um 95% aller Menschen in einem “anonymen” Datensatz eindeutig zu identifizieren. Indoor-Lokalisierung (ARKit/ARCore) liefert tausende solcher Punkte pro Minute — scheinbar anonyme Bewegungsdaten sind faktisch personenbezogen.

3. Koordinatenreferenzsysteme (KRS/CRS)

Kernidee: Die Erde ist eine Kugel (genauer: ein Geoid/Ellipsoid), Bildschirme und Karten sind flach — eine Projektion bildet die gekrümmte Oberfläche auf eine 2D-Ebene ab. Jedes Koordinatensystem hat einen standardisierten EPSG-Code.

3.1.4.1. Abstraktionshierarchie

Geoid → Erdellipsoid → Lokales Ellipsoid → Projektion → 2D-Koordinaten

Synonyme: KRS, KBS, CRS (coordinate reference system), SRS (spatial reference system), SRID (spatial reference system ID)

3.1.4.2. Die drei wichtigsten Koordinatensysteme

Die drei Standard-CRS im Überblick:

1. WGS84 — EPSG:4326 (“GPS-Koordinaten”)

- Art: geozentrisch, geografisch, in Grad
- Format: Breitengrad/Längengrad (lat/lon) — Reihenfolge: erst N, dann E
- Beispiel: 47.223, 8.817 (Rapperswil)
- Verwendung: GPS, GeoJSON-Standard, Datenaustausch

2. Web Mercator — EPSG:3857 (“Online-Karten”)

- Art: kartesisch (eben), metrisch
- Beispiel: 981523, 5978698
- Verwendung: Google Maps, OpenStreetMap, alle Online-Karten
- Problem: Flächenverzerrung in hohen Breitengraden → nicht für militärische/präzise Anwendungen

3. CHLV95 — EPSG:2056 (“Schweizer Koordinaten”)

- Art: kartesisch-metrisch, Landesvermessung CH
- Format: 7-stellige Koordinaten, E/N
- Beispiel: 2'704'404, 1'231'247
- Verwendung: Alle offiziellen Schweizer Geodaten (swisstopo etc.)

Tip: CRS erkennen und unterscheiden:

- Koordinaten mit Dezimalgrad (z.B. 47.2, 8.8) → WGS84 (EPSG:4326)
- 6-7-stellige Zahlen in Millionen (z.B. 981523) → Web Mercator (EPSG:3857)
- 7-stellige Koordinaten mit “2'7xx'xxx” / “1'2xx'xxx” → CHLV95 (EPSG:2056)
- Faustregel: lat/lon-Koordinaten immer auf 5 Stellen runden (nicht 16-stellig speichern!)
- Tool: <https://map.geo.admin.ch> zeigt Koordinaten in allen Systemen

Exam Trap: Häufige Fehler bei Koordinatensystemen:

- **lon/lat vs. lat/lon verwechseln:** GeoJSON verwendet [lon, lat] (X, Y), GPS-Geräte oft lat, lon — immer prüfen!
- **Falsches CRS in der DB:** ST_Transform() vergessen → räumliche Abfragen liefern falsche Ergebnisse
- **Web Mercator für Flächenberechnungen:** Mercator verzerrt Flächen stark → für genaue Berechnungen WGS84 oder CHLV95 verwenden
- **SRID nicht gesetzt:** PostGIS-Geometrie ohne SRID ist unbrauchbar für räumliche Joins

3.1.4.3. CRS-Transformation in PostGIS

```
-- Transformation von beliebigem CRS nach WGS84 (für Visualisierung)
SELECT ST_Transform(geom, 4326) AS geom_wgs84
FROM meine_tabelle;
```

4. Geodaten — Typen und Besonderheiten

Kernidee: Geodaten sind Daten mit Raumbezug. Sie sind komplexer als normale Daten, weil sie spezielle Datenstrukturen, Formate und Konsistenzbedingungen erfordern.

3.1.5.1. Formen von Geodaten

Geodaten können vorliegen als:

- Geografische **Namen** (z.B. “Garnhänke Rapperswil”)
- **Adressen** (Post-/Gebäudeadressen)
- **Codes** (z.B. Pluscode “6RF8+94”)
- **Geometrien:** Punkte (Koordinaten), Linien (LineString), Flächen (Polygon)
- **Pixel/Voxel** (Rasterdaten)

3.1.5.2. Warum sind Geodaten besonders?

- **Komplexe Datenstruktur** → viele Formate
- **Komplexe Konsistenzbedingungen** (reale Welt ist kompliziert)
- **Grosser Erfassungsaufwand** (terrestrisch, Luftbild, LiDAR)
- **Grosse Datenmengen** (Big Data: Google Earth Engine = Petabytes)
- **Metadaten** notwendig: CRS, Auflösung, Qualität, Semantik

3.1.5.3. Geo-Metadaten

Zu Geodaten gehören immer Metadaten:

- **Koordinatensystem** (SRID/EPSG-Code)
- **Metrik/Masseinheiten** (Grad oder Meter?)
- **Granularität/Auflösung** (Auflösung am Boden in cm/px)
- **Semantik** (“Was ist ein Gebäude?” — administrative Definition!)

5. Vektor- vs. Rasterdaten

Kernidee: Es gibt zwei grundlegende Arten, geografische Phänomene digital zu repräsentieren — als diskrete Vektorobjekte oder als kontinuierliches Rasterfeld.

3.1.6.1. Zwei Modellsichten

Diskrete Entitäten-/Objekte-Sicht:

- Räumlich begrenzte Objekte (z.B. ein Gebäude, eine Strasse)
- Natürlich als Vektordaten modelliert

Kontinuierliche Felder-Sicht:

- Überall im Raum gibt es einen Wert (z.B. Luftdruck, Temperatur)
- Natürlich als Rasterdaten modelliert

3.1.6.2. Vektordaten

Vektordaten bestehen aus:

- **Geometrie:** Punkt (Point), Linie (LineString), Fläche (Polygon) — mit Koordinaten
- **Topologie:** Nachbarschaftsbeziehungen (wichtig für Navigation, Routing)
- **Sachdaten** (Attribute/Thematische Daten): z.B. Name, Kategorie, Länge

Typische Struktur (Feature Class):

- Eine Tabelle mit **genau einem Geometrie-Attribut** + beliebig viele Sachdaten-Attribute

3.1.6.3. Rasterdaten

- Zusammengehörige Menge von (Farb-)Pixeln mit Raumbezug (georeferenziert)
- Erfasst durch CCD/CMOS-Sensoren (Scanner, Kameras, Satelliten)
- Format: GeoTIFF (Standard)
- Jeder Pixel hat einen (Farb-)Wert

Vektor vs. Raster — wann was?

Kriterium	Vektordaten	Rasterdaten	— — —	Objekttyp	diskrete Objekte	
kontinuierliche Felder		Einzelns	ansprechbar?	Ja	Nein (nur Pixel)	
Skalierbar?		Ja (verlustfrei)		Nein (verpixelt)		
Darstellung	langsamer (berechnet)	schneller				
Herstellung	teurer	billiger		Typisches Format	GeoJSON, GeoPackage	GeoTIFF
Beispiel	Strasse, Gebäude, POI	Luftbild, Temperaturkarte		Übung in diesem Kurs		
Hauptthema	✓	kurz (Google Earth Engine)				

6. GIS-Prinzipien der Geoinformationsverarbeitung

3.1.7.1. Ebenenprinzip / Layerprinzip

Kernidee: Geodaten werden in thematischen Schichten (Layers) organisiert. Das gemeinsame Koordinatensystem erlaubt den Lagevergleich zwischen unabhängigen Datenquellen — das ist die eigentliche Stärke von GIS.

GIS

- └ Layer: BEVÖLKERUNG
- └ Layer: KLIMADATEN
- └ Layer: LANDNUTZUNG
- └ Layer: VERMESSUNG
- └ Layer: VERSORGUNGSNETZE

Koordinaten ermöglichen vertikalen Lagevergleich — man kann Objekte aus verschiedenen Layern räumlich verknüpfen ohne explizite Fremdschlüssel.

3.1.7.2. Raumbezug als universeller Fremdschlüssel

Kernidee: Koordinaten ersetzen in GIS den klassischen Datenbank-Fremdschlüssel. Jedes Objekt mit denselben Koordinaten (im selben CRS) kann mit jedem anderen Objekt verknüpft werden — auch wenn die Daten aus völlig unterschiedlichen Quellen stammen.

Tip: Räumlicher Join (Spatial Join): Statt `WHERE a.id = b.fk_id` wird verknüpft mit räumlichen Prädikaten:

- `ST_Intersects(a.geom, b.geom)` — überschneiden sich?
- `ST_Within(a.geom, b.geom)` — a liegt innerhalb von b?
- `ST_DWithin(a.geom, b.geom, distanz)` — a liegt maximal X Meter von b entfernt?

Data Enrichment: Adressen + Koordinaten = Geocoding; Punkte + Bezirksgrenzen = sozioökonomische Daten zuweisen.

3.1.7.3. Grafik-View-Prinzip

- Geodaten sind in der Datenbank **grafiklos** gespeichert (nur Koordinaten + Attribute)
- Die Grafik (Styling: Farbe, Breite, Beschriftung) wird **real-time** aus der DB extrahiert und gerendert
- Trennung: Daten $\neq$ Darstellung (wie bei Web: HTML $\neq$ CSS)

Beispiel Autobahn A1:

- Daten: `{type: LineString, coordinates: [...], name: "A1"}`
- Grafik: `{Farbe: Grün, Breite: 3mm, Beschriftung: "A1", Schrift: Helvetica}`

3.1.7.4. Massstabsabhängige Darstellung (GIS-Zoom)

- Bei kleinem Massstab (weit weg gezoomt): wenige, vereinfachte Objekte
- Bei grossem Massstab (nah herangezoomt): mehr Details, mehr Objekte
- Automatische Generalisierung: Gebäude werden zu Punkten, Linien vereinfacht

3.1.7.5. Mengenabhängige Darstellung

- Viele Punkte auf einer Karte → **Point Clustering** (Punkte werden zu Cluster-Symbolen gruppiert)
- Verhindert “überladene” Karten bei grossen Datensätzen

—

7. GIS-Dateiformate

Kernidee: Geodaten existieren in vielen Formaten — die Wahl des Formats hängt von Verwendungszweck, Komplexität der Geometrie und Interoperabilität ab.

3.1.8.1. Vektorformate

Welches Format wählen?

GeoJSON — Standard für Web und Datenaustausch

- Textbasiert (JSON), menschenlesbar
- Struktur: `FeatureCollection` → `features[]` → `{geometry, properties}`
- Koordinaten immer in WGS84 (EPSG:4326), Reihenfolge: [lon, lat]
- Validierung: geojson.io oder <http://geojsonlint.com>
- ⚠ Koordinaten auf 5 Stellen runden (nicht 16-stellig!)

GeoPackage (.gpkg) — Standard für Desktop-GIS

- SQLite-Datenbank als Datei
- Binärformat; unterstützt Vektor- UND Rasterdaten
- Nachfolger des Shapefile (empfohlen!)

Shapefile (.shp + .dbf + ...) — Legacy-Format

- Mind. 2 Dateien: Geometrie (.shp) + Sachdaten (.dbf)
- Weit verbreitet aber veraltet → vermeiden! (<http://switchfromshapefile.org>)

CSV — nur für Punkte

- Geeignet wenn Attribute `lat` und `lon` vorhanden
- Für Linien/Polygone: WKT-String-Spalte verwenden (z.B. "LINESTRING(10 10, 20 20)")

DXF — für reine Geometrie/CAD

- Kein Sachdaten-Support; von AutoCAD
- Nur wenn CAD-Austausch nötig

GeoParquet — für Big Data

- Erweiterung von Apache Parquet (spaltenorientiert)
- Effiziente Komprimierung, schnelle Abfragen auf grossen Datensätzen

3.1.8.2. Rasterformate

- **GeoTIFF** → Standard für Raster-Geodaten-Austausch (georeferenziert)
- **PNG/JPG** → nur mit Zusatzdatei "Worldfile" georeferenziert
- **PDF** → für Drucker, nicht für GIS-Verarbeitung

3.1.8.3. WKT (Well-Known Text) — Geometrie als Text

```
Punkt:    POINT(8.817 47.223)
Linie:    LINESTRING(8.81 47.22, 8.82 47.23, 8.83 47.24)
Fläche:   POLYGON((8.81 47.22, 8.82 47.22, 8.82 47.23, 8.81 47.22))
```

8. GIS-Architekturen und Produkte

Kernidee: GIS-Systeme sind aufgeteilt in ein **Basissystem** (Werkzeugkasten mit GIS-Funktionen) und **Fachschalen** (domänenspezifische Applikationen darüber).

3.1.9.1. Desktop-GIS**QGIS** (Open Source, GPL)

- Windows, macOS, Linux
- Empfohlen für diese Vorlesung
- Mobile Variante: QField

ArcGIS (kommerziell, ESRI)

- ArcGIS Pro (Desktop) + ArcGIS Online (Cloud)
- Marktführer im professionellen Bereich

3.1.9.2. Geodatenbanken

Alle bekannten DBMS unterstützen Geodaten:

- **PostgreSQL + PostGIS** (Open Source, empfohlen)
- Oracle Spatial, MS SQL Server (kommerziell)
- MySQL/MariaDB, DuckDB, SQLite/GeoPackage

Geodatenbanken bieten:

- Geodatentypen (Geometry, Geography)
- Räumliche Funktionen (ST_*)
- Spezielle 2D-Indexe (R-Tree / GiST-Index)

3.1.9.3. Web Mapping (No-Code bis Low-Code)

Für Datenanalysten/Journalisten:

- **Datawrapper** — beliebt bei Journalisten, einfach
- **uMap** — Open Source, basiert auf OSM
- **Felt** — “Google Docs für Karten”
- **map.geo.admin.ch** — Schweizer Behördendaten

Für Software Engineers (JS/Python):

- JavaScript: Leaflet, MapLibre (beide empfohlen), OpenLayers
- Python Backend: GeoPandas, Shapely, OSMnx
- Jupyter: matplotlib + Overpass-API

3.1.9.4. Geovisualisierung für SQL-Entwickler

```
-- In pgAdmin 4: Geometry Viewer Button
-- Voraussetzung: Geometrie muss in EPSG:4326 sein!
SELECT ST_Transform(geom, 4326) AS geom
FROM meine_tabelle;
```

9. OpenStreetMap und PostPASS

Kernidee: OpenStreetMap (OSM) ist die grösste freie Geodatenbank der Welt — mit über 100 Mio. Nutzern, Milliarden Gebäuden und 70 Mio. Points-of-Interest. PostPASS ermöglicht SQL-Abfragen direkt auf OSM-Daten.

3.1.10.1. OSM-Datenstruktur

OSM speichert Geodaten als **Tags** (Key-Value-Paare):

- `amenity=restaurant` — ein Restaurant
- `amenity=fast_food + cuisine=kebab` — ein Kebab-Imbiss
- `amenity=drinking_water` — ein Trinkwasserbrunnen

3.1.10.2. PostPASS-Abfrage (Praxis-Rezept)

Rezept: OSM-Daten mit PostPASS abfragen

1. URL: <https://overpass-turbo.eu> mit PostPASS-Header
2. Header: `{{data:sql,server=https://postpass.geofabrik.de/api/0.2/}}`
3. SQL schreiben mit OSM-Tabellen und PostGIS-Funktionen

Allgemeines Query-Muster:

```

SELECT
    osm_id,
    tags->>'name'          AS name,
    tags->>'[tag-key]'      AS attribut,
    ST_PointOnSurface(geom) AS geom
FROM postpass_pointpolygon
WHERE tags->>'amenity' IN ('[kategorie1]', '[kategorie2]')
AND tags->>'[filter-key]' = '[filter-wert]'
AND geom && {{bbox}};

```

Erklärung der Schlüsselemente:

- tags->>'key' → extrahiert Wert aus JSON-Tags-Spalte
- ST_PointOnSurface(geom) → konvertiert Polygone zu Punkten (für einheitliche Darstellung)
- {{bbox}} → Template-Variable für sichtbaren Kartenausschnitt (wird zu ST_MakeEnvelope(...))
- geom && {{bbox}} → Bounding-Box-Filter (schnell, nutzt Index)
- string_to_array(tags->>'cuisine', ';') @> '{kebab}' → prüft ob 'kebab' in einer ';' getrennten Liste vorkommt
- Kein Semikolon am SQL-Ende!

Beispiel: Kebab-Restaurants in Sichtbereich

```

SELECT
    osm_id,
    tags->>'name'      AS name,
    tags->>'cuisine' AS cuisine,
    ST_PointOnSurface(geom) AS geom
FROM postpass_pointpolygon
WHERE tags->>'amenity' IN ('fast_food', 'restaurant')
AND string_to_array(tags->>'cuisine', ';') @> '{kebab}'
AND geom && {{bbox}}

```

Tip: OSM-Tags für die Übung (Brunnen):

- amenity=drinking_water — Trinkwasserbrunnen
- amenity=fountain — Zierbrunnen
- amenity=water_point — Wasserstelle
- man_made=water_well — Brunnen/Quelle
- man_made=water_tap — Wasserhahn

OSM-Tags finden: tagfinder.osm.ch

10. Lösungsrezept für die Übungsaufgabe “Raumanalyse Vektordaten: Autobahn”

Kernidee der Übung: Vektordaten (Autobahnen aus OSM) werden geladen, visualisiert und räumlich analysiert — typische Operationen: Filtern, Puffer erzeugen, Verschneiden, Attribute berechnen.

Allgemeines Rezept: Räumliche Vektoranalyse in QGIS

Schritt 1: Daten beschaffen

- OSM-Daten über Overpass API oder PostPASS herunterladen
- Format prüfen (GeoJSON, GeoPackage, Shapefile)
- CRS prüfen (meist WGS84/EPSC:4326 aus OSM)

Schritt 2: Daten in QGIS laden und prüfen

- Layer hinzufügen (Layer > Datei hinzufügen)
- Attributtabelle öffnen — welche Spalten gibt es?
- CRS des Layers prüfen (rechts unten im QGIS-Fenster)

Schritt 3: Daten filtern/selektieren

- Nach Attributen filtern: Rechtsklick Layer > Eigenschaften > Abfrage-Generator
- Beispiel Autobahn: "highway" = 'motorway'

Schritt 4: Räumliche Analyse

- **Puffer:** Vektor > Geoverarbeitungswerkzeuge > Puffer
 - Eingabe: Linienlayer
 - Distanz: z.B. 500m (→ CRS muss metrisch sein!)
- **Verschneidung** (Intersection): Vektor > Geoverarbeitungswerkzeuge > Verschneidung
 - Welche Objekte liegen innerhalb des Puffers?
- **Längenberechnung:** Feldrechner → \$length (in CRS-Einheiten)

Schritt 5: Ergebnis visualisieren

- Styling anpassen (Farbe, Linienbreite, Beschriftung)
- Als Karte exportieren oder als GeoJSON speichern

Schritt 6: Ergebnis interpretieren

- Was zeigt die Analyse? Welche Schlüsse können gezogen werden?

Exam Trap: Häufige Fehler bei Vektordatenanalysen:

- **CRS-Mismatch:** Zwei Layer mit verschiedenen CRS werden nicht korrekt überlagert → immer auf gleiches CRS reprojizieren (in QGIS: Rechtsklick > Layer CRS > Auf CRS reprojizieren)
- **Distanz in Grad statt Meter:** Puffer mit 500 in WGS84 = 500 Grad = falsch! → Zuerst auf metrisches CRS (z.B. EPSG:2056 oder EPSG:3857) umprojizieren
- **Topologiefehler:** Ungültige Geometrien (self-intersecting Polygone) → Vektor > Geometriewerkzeuge > Geometrien reparieren
- **Attribut-Encoding:** Umlaute werden falsch dargestellt → UTF-8 Encoding sicherstellen

11. Datenquellen für Geodaten

Wichtige Geodatenquellen:

| Quelle | Inhalt | Lizenz | |—|—|—| | OpenStreetMap (OSM) | Weltweit; POIs, Strassen, Gebäude | ODbL (frei) | | swisstopo / map.geo.admin.ch | Schweiz; amtliche Geodaten |

meist frei | | Overture Maps | MS, Amazon, TomTom; Karten | offen | | Google Maps / Earth Engine | Weltweit; Raster + POIs | kommerziell | | Geoportal Kantone/Gemeinden | CH; Pläne, Zonen | variiert |

Zugang zu OSM-Daten:

- Overpass API / overpass-turbo.eu (für kleine Abfragen)
- PostPASS (für SQL-Abfragen, Kursumgebung)
- Geofabrik.de (Dump-Downloads nach Land/Region)
- OSMnx (Python-Library)

Zusammenfassung: Konzeptlandkarte

Week 4: [Topic Title]

Woche 4: Spatial Data Analytics — PostGIS, Funktionen & Geodatenbanken

Überblick

Diese Woche dreht sich alles um **PostGIS** als räumliche Datenbankerweiterung: Wie man eine Geodatenbank aufbaut, Daten lädt, räumliche Abfragen schreibt und Raumanalysen durchführt. Der rote Faden ist das **EVAP-Prinzip**: Eingabe → Verarbeitung → Analyse → Präsentation.

1. Das EVAP-Prinzip bei Geodaten

Kernidee: Jede Geodatenanalyse folgt demselben vierstufigen Ablauf — von der leeren Datenbank bis zur Visualisierung.

EVAP-Ablauf für Geodaten:

1. **Eingabe (E):** Leere Datenbank erstellen, PostGIS-Extension aktivieren, Tabellen mit Geometrie-Spalten anlegen
2. **Verarbeitung/Laden (V):** Geodaten importieren (QGIS DB-Manager, ogr2ogr, shp2pgsql, manuelle INSERTs)
3. **Analyse (A):** SQL-Abfragen mit PostGIS-Funktionen ausführen
4. **Präsentation (P):** Ergebnisse in QGIS, geojson.io oder anderen GIS-Tools visualisieren

Beispiel: Schweizer Gemeindegrenzen analysieren:

- E: `CREATE DATABASE gisdb; + CREATE EXTENSION postgis;`
- V: GeoPackage via `ogr2ogr` laden
- A: `SELECT name FROM gemeinden WHERE ST_Intersects(geom, fluss.geom);`
- P: Ergebnis in QGIS als Layer darstellen

Tip: Woran erkenne ich EVAP-Aufgaben?

- Aufgabe verlangt das Aufsetzen einer Geodatenbank → Schritt E
- Datei (.gpkg, .json, .shp) soll importiert werden → Schritt V
- Eine räumliche Frage soll beantwortet werden → Schritt A
- Ergebnis soll auf einer Karte erscheinen → Schritt P

Wichtig bei Geodaten speziell:

- Schritt E: `GEOMETRY`-Spalte braucht Type Modifier + EPSG-Code
- Schritt V: Bulk-Tools nötig (grosse Datenmengen)
- Schritt P: Koordinatenzahlen alleine sagen nichts — immer visualisieren!

2. PostGIS einrichten

Kernidee: PostGIS ist eine Erweiterung von PostgreSQL, die Geodatentypen, räumliche Funktionen und einen speziellen Index (GiST/R-Baum) hinzufügt.

PostGIS-Datenbank aufsetzen — Rezept:

1. Datenbank erstellen:

```
CREATE DATABASE gisdb;
```

oder in Shell: `createdb -U postgres gisdb`

2. Zur Datenbank wechseln und Extension aktivieren:

```
\c gisdb
CREATE EXTENSION postgis;
```

oder in Shell: `psql -U postgres -d gisdb -c "CREATE EXTENSION postgis;"`

3. Installation prüfen:

```
SELECT postgis_full_version();
```

(sollte eine Zeile mit Versionsinformation zurückgeben)

4. Tabelle mit Geometrie-Spalte erstellen:

```
CREATE TABLE tabellenname (
  id INT PRIMARY KEY,
  name TEXT,
  geom GEOMETRY('GEOMETRIETYP', EPSG_CODE)
);
```

5. Räumlichen Index erstellen:

```
CREATE INDEX idx_tabellenname_geom
ON tabellenname
USING gist(geom);
```

Beispiel — Eisenbahntabelle für CH:

```
CREATE TABLE eisenbahn (
  id INT4 PRIMARY KEY,
  name TEXT,
  geom GEOMETRY('LINESTRING', 2056)
);
CREATE INDEX idx_eisenbahn_geom ON eisenbahn USING gist(geom);
```

Tip: Woran erkenne ich, dass ich PostGIS-Setup brauche?

- Aufgabe sagt: “erstellen Sie eine Geodatenbank”
- Aufgabe arbeitet mit Geometrie-Typen in PostgreSQL
- Fehlermeldung “type geometry does not exist” → Extension fehlt!

Type Modifier Syntax:

```
GEOMETRY('POINT', 4326)      -- Punkt in WGS84
GEOMETRY('LINESTRING', 2056) -- Linie in CH1903+
GEOMETRY('POLYGON', 21781)   -- Fläche in CH1903
GEOMETRY                      -- ohne Einschränkung (flexibel, aber weniger
                              sicher)
```

Exam Trap: Häufige Fehler beim PostGIS-Setup:

- AddGeometryColumn(...) ist **veraltet** — stattdessen Type Modifier in CREATE TABLE verwenden
- Extension muss **pro Datenbank** aktiviert werden, nicht global
- Ohne räumlichen GiST-Index werden räumliche Abfragen bei grossen Datenmengen sehr langsam
- Spaltenname: oft geom, manchmal geometry oder the_geom — konsistent bleiben!

3. Geodatentypen (Vektoren)

Kernidee: Vektorgeometrien beschreiben Raumobjekte als Punkte, Linien oder Flächen — mit einem klaren Typ-Hierarchie-System.

3.1 Basis-Geometrietypen

```
GEOMETRY (abstrakter Obertyp)
├─ POINT
├─ LINESTRING
├─ POLYGON
├─ MULTIPOINT
├─ MULTILINESTRING
├─ MULTIPOLYGON
└─ GEOMETRYCOLLECTION
```

Tip: Welcher Typ für welches Objekt?

- Einzelner Ort, Adresse, Gipfel → POINT
- Strasse, Fluss, Grenze (Linie) → LINESTRING
- Gemeindegebiet, See, Gebäudefläche → POLYGON
- Mehrere Punkte zusammen → MULTIPOINT
- Mehrere Linien zusammen → MULTILINESTRING
- Mehrere Flächen zusammen (z.B. Kanton mit Enklaven) → MULTIPOLYGON
- Gemischte Geometrien → GEOMETRYCOLLECTION

3.2 WKT — Well Known Text

Kernidee: WKT ist die lesbare Textdarstellung von Geometrien, die PostGIS versteht.

```
POINT(lon lat)          -- Einzelpunkt
LINESTRING(x1 y1, x2 y2, x3 y3) -- Linie mit Stützpunkten
POLYGON((x1 y1, x2 y2, x3 y3, x1 y1)) -- Fläche (geschlossen!)
```

Exam Trap: Koordinatenreihenfolge — der häufigste Fehler überhaupt!

- Menschen sagen: “lat/lon” (Breitengrad/Längengrad), z.B. “47.22, 8.81” für Rapperswil
- PostGIS/WKT erwartet: **lon/lat** (Längengrad zuerst!), also `POINT(8.81 47.22)`
- Merkhilfe: “**Länge lang**” — Längengrade gehen von Pol zu Pol (lang), Breitengrade quer (breit)
- In EPSG:4326: `POINT(Längengrad Breitengrad) = POINT(8.818 47.226)` für Rapperswil

Beispiel für Rapperswil:

- GeoAdmin/Maps zeigt: 47.226°N, 8.818°E (lat zuerst)
- PostGIS WKT: `POINT(8.818 47.226)` (lon zuerst!)

4. Geodaten-Konstrukturen

Kernidee: Konstrukturen wandeln Text- oder Koordinaten-Eingaben in interne PostGIS-Geometrieobjekte um.

Wichtige Konstrukturen — Rezept:

```
-- Aus WKT-Text (mit SRID):
ST_GeomFromText('POINT(lon lat)', EPSG_CODE)
ST_GeometryFromText('LINESTRING(x1 y1, x2 y2)', EPSG_CODE)

-- Aus einzelnen Koordinaten:
ST_MakePoint(lon, lat)          -- dann mit ST_SetSRID kombinieren
ST_SetSRID(ST_MakePoint(lon, lat), EPSG_CODE)
ST_Point(lon, lat, EPSG_CODE) -- neuere Variante

-- Geography-Typ (für echte Kugelberechnungen):
ST_GeogFromText('SRID=4326;POINT(lon lat)')

-- Geometrie als Text ausgeben:
ST_AsText(geom)                -- gibt WKT zurück
ST_AsEWKT(geom)               -- gibt WKT mit SRID zurück
ST_AsGeoJSON(geom)            -- gibt GeoJSON zurück
```

Beispiel — Punkt für Rapperswil einfügen:

```
INSERT INTO ort(id, name, geom)
VALUES (
  100,
  'Rapperswil',
  ST_GeometryFromText('POINT(8.818 47.226)', 4326)
);
```

Beispiel — Linie einfügen:

```
INSERT INTO uster(id, name, geom)
VALUES (
  100,
  'Uster',
  ST_GeometryFromText('LINESTRING(8.81 47.22, 8.82 47.23)', 4326)
);
```

Tip: Wann welchen Konstruktor?

- Habe WKT-String + SRID → `ST_GeomFromText('...', srid)`
- Habe nur Koordinaten → `ST_SetSRID(ST_MakePoint(lon, lat), srid)`
- Will Geography (Kugelkoordinaten) → `ST_GeogFromText('SRID=4326;POINT(...)')`
- Will Ergebnis als lesbaren Text → `ST_AsText(geom)` in SELECT

5. Koordinatenreferenzsysteme (EPSG-Codes)

Kernidee: Jede Koordinate braucht ein Referenzsystem — ohne EPSG-Code sind Koordinaten bedeutungslos. Viele Fehler entstehen durch falsches oder gemischtes CRS.

Die wichtigsten EPSG-Codes:

EPSG	Name	Einheit	Verwendung	SRID	Geography
4326	WGS84	Grad (°)	GPS, GeoJSON, OpenStreetMap	4326	Ja
3857	Web-Mercator	Meter	Google Maps, OpenStreetMap Tiles	3857	Nein
2056	CH1903+/LV95	Meter	Aktuelle CH-Landeskarte	2056	Nein
21781	CH1903/LV03	Meter	Ältere CH-Daten	21781	Nein

Faustregel:

- Distanz in Metern berechnen → Daten in metrisches CRS transformieren (2056 für CH)
- Daten aus dem Web/GPS → wahrscheinlich EPSG:4326
- Schweizer Amtsdaten → oft EPSG:2056 oder 21781

Exam Trap: CRS-Fehler — häufigste Probleme:

- `ST_Distance` in EPSG:4326 gibt Ergebnis in **Grad**, nicht in Metern! → Immer erst `ST_Transform(geom, 2056)` für CH-Berechnungen in Metern
- Zwei Tabellen mit verschiedenem CRS können nicht direkt verglichen werden → Eine der Geometrien transformieren: `ST_Transform(geom, ziel_epsg)`
- Wenn `ST_Intersects` nichts findet obwohl es sollte → CRS prüfen!

CRS-Transformation — Rezept:

```
-- Geometrie in anderes CRS transformieren:
ST_Transform(geom, ziel_epsg)
```

```
-- Beispiel: Distanz in Metern berechnen (CH):
SELECT ST_Distance(
  ST_Transform(a.geom, 2056),
  ST_Transform(b.geom, 2056)
) AS distanz_meter
FROM tabelle_a a, tabelle_b b
WHERE a.name = 'Bern' AND b.name = 'Zürich';

-- Beispiel: Punkt in CH-Koordinaten aus WGS84:
SELECT ST_Transform(
  ST_GeomFromText('POINT(8.81642 47.22541)', 4326),
  21781
) AS geom_ch;
```

6. Metrische Funktionen (Zahl zurück)

Kernidee: Diese Funktionen messen geometrische Eigenschaften und geben Zahlen zurück — Distanz, Länge, Fläche.

6.1 Distanz

Distanz berechnen — Rezept:

```
-- Distanz in CRS-Einheiten (Meter wenn metrisches CRS):
ST_Distance(geom1, geom2)

-- Distanz auf Kugeloberfläche (immer Meter, für lat/lon geeignet):
ST_DistanceSphere(geom1, geom2)

-- Allgemeines Muster für Distanz zweier Orte (in Metern, CH):
SELECT
  a.name AS ort_a,
  b.name AS ort_b,
  ST_Distance(
    ST_Transform(a.geom, 2056),
    ST_Transform(b.geom, 2056)
  ) AS distanz_meter
FROM orte a, orte b
WHERE a.name = 'Bern' AND b.name = 'Zürich';
```

Beispiel — Übung 5.1, Distanz Bern–Zürich:

```
SELECT ST_Distance(
  ST_Transform(a.geom, 2056),
  ST_Transform(b.geom, 2056)
) AS distanz_m
```

```
FROM orte a, orte b
WHERE a.name = 'Bern' AND b.name = 'Zürich';
```

Variante mit festem Referenzpunkt (aus WKT):

```
SELECT name,
       round(
         ST_DistanceSphere(
           geom,
           ST_GeomFromText('POINT(8.816511 47.223064)', 4326)
         )::numeric, 0
       ) AS distanz_meter
FROM beers
ORDER BY distanz_meter;
```

6.2 Länge und Fläche

```
ST_Length(geom)    -- Länge einer Linie (in CRS-Einheiten)
ST_Area(geom)       -- Fläche eines Polygons (in CRS-Einheiten²)
```

Tip: Wann ST_Distance vs. ST_DistanceSphere?

- Daten in metrischem CRS (2056, 21781): ST_Distance direkt → Ergebnis in Metern
- Daten in EPSG:4326 (Grad): ST_DistanceSphere → Ergebnis immer in Metern
- Oder: ST_Transform auf metrisches CRS, dann ST_Distance
- Für präzise Berechnungen in CH: ST_Transform(..., 2056) + ST_Distance

7. Räumliche Filter-Funktionen (Boolean zurück)

Kernidee: Diese Funktionen testen topologische Beziehungen zwischen Geometrien und geben TRUE/FALSE zurück — sie werden in WHERE-Klauseln eingesetzt.

Wichtige räumliche Filter — Rezept:

```
-- Schneidet/überlappt (am häufigsten verwendet!):
ST_Intersects(g1, g2)          -- TRUE wenn gemeinsame Punkte

-- Innerhalb einer Distanz:
ST_DWithin(g1, g2, distanz)    -- TRUE wenn Abstand <= distanz

-- Gleichheit:
ST_Equals(g1, g2)              -- TRUE wenn geometrisch identisch

-- Berühren (nur Rand):
ST_Touches(g1, g2)             -- TRUE wenn nur Rand berührt
```



```
-- Schneidet (kreuzt):  
ST_Crosses(g1, g2)          -- TRUE wenn Linie durch Fläche  
  
-- Enthält/Liegt in:  
ST_Contains(g1, g2)         -- g1 enthält g2 vollständig  
ST_Within(g1, g2)           -- g2 liegt in g1  
  
-- Bounding-Box-Operator (schnell, für Index):  
g1 && g2                     -- TRUE wenn Bounding Boxes überlappen
```

Allgemeines Muster — “Alle X die Bedingung Y erfüllen”:

```
SELECT a.name  
FROM tabelle_a a  
JOIN tabelle_b b ON ST_Intersects(a.geom, b.geom)  
WHERE b.name = 'Referenzobjekt';
```

Tip: Welche Funktion wann?

- **Normalfall** (Überschneidung jeder Art): `ST_Intersects` ← bevorzugen!
- **Umkreissuche** (alle X innerhalb d Meter): `ST_DWithin`
- **Angrenzende Gebiete**: `ST_Intersects` (Grenzen berühren sich) oder `ST_Touches`
- **Punkt liegt in Polygon**: `ST_Intersects` oder `ST_Within`
- **Linie fließt durch Gemeinde**: `ST_Intersects`

`ST_Intersects` ist dem `ST_Within`, `ST_Contains`, `ST_Covers` **normalerweise vorzuziehen**, weil es den räumlichen Index nutzt und die häufigste topologische Beziehung prüft.

Exam Trap: Häufiger Denkfehler bei räumlichen Filtern:

- `ST_Intersects(a, b)` im JOIN ist **nicht dasselbe** wie ein kartesisches Produkt mit WHERE — JOIN ist effizienter
- Bei `ST_DWithin` muss die Distanz in den **CRS-Einheiten** der Geometrie angegeben werden! Mit EPSG:4326 sind das Grad, nicht Meter → transformieren!
- `ST_Disjoint` ist das Gegenteil von `ST_Intersects` — selten gebraucht

8. Räumliche Abfrage-Muster (Rezepte für Übungsaufgaben)

8.1 Umkreissuche: Alle Objekte innerhalb eines Radius

Kernidee: “Alle Orte im Umkreis von X km um Punkt P” ist eine klassische räumliche Abfrage.

Umkreissuche — Rezept:

```
-- Variante 1: ST_DWithin (empfohlen, nutzt Index)  
SELECT a.name
```

```
FROM tabelle a
WHERE ST_DWithin(
  a.geom,
  ST_Transform(ST_GeomFromText('POINT(lon lat)', 4326), ziel_epsg),
  radius_in_meter
);

-- Variante 2: Mit CTE für Referenzpunkt
WITH referenz AS (
  SELECT ST_Transform(
    ST_GeomFromText('POINT(lon lat)', 4326),
    ziel_epsg
  ) AS geom
)
SELECT a.name,
  ST_Distance(a.geom, r.geom)::int AS distanz_m
FROM tabelle a
JOIN referenz r ON ST_DWithin(a.geom, r.geom, radius_in_meter)
ORDER BY distanz_m;
```

Beispiel — Übung 5.3: Orte im Umkreis von 10 km um HSR:

```
-- HSR in CH1903: Easting=704472, Northing=231216
SELECT name
FROM orte
WHERE ST_DWithin(
  geom,
  ST_SetSRID(ST_MakePoint(704472, 231216), 21781),
  10000 -- 10 km in Metern (Daten in EPSG:21781)
);
```

Beispiel — Übung 4: Orte im Umkreis mit Distanzangabe:

```
WITH mylocation AS (
  SELECT ST_Transform(
    ST_GeomFromText('POINT(8.81642 47.22541)', 4326),
    21781
  ) AS geom
)
SELECT
  o.name,
  ST_AsText(o.geom, 0) AS koordinaten,
  ST_Distance(o.geom, m.geom)::int AS distanz_m
FROM orte o
JOIN mylocation m ON ST_DWithin(o.geom, m.geom, 10000)
ORDER BY distanz_m;
```

8.2 Angrenzende/Überlappende Objekte finden

Kernidee: “Alle Gemeinden die an X angrenzen” — zwei Tabellen räumlich verknüpfen.

Nachbarschaftsabfrage — Rezept:

```
-- Alle Objekte aus tabelle_a, die räumlich tabelle_b berühren/schneiden:  
SELECT a.name  
FROM tabelle_a a  
JOIN tabelle_b b ON ST_Intersects(a.geom, b.geom)  
WHERE b.name = 'Referenzname'  
      AND a.name != 'Referenzname'; -- Selbst ausschliessen!
```

Beispiel — Übung 5.2: Gemeinden angrenzend an Rapperswil-Jona:

```
SELECT a.name  
FROM gemeinden a  
JOIN gemeinden b ON ST_Intersects(a.geom, b.geom)  
WHERE b.name = 'Rapperswil-Jona'  
      AND a.name != 'Rapperswil-Jona';
```

Exam Trap: Selbstbezug vergessen: Wenn man eine Tabelle mit sich selbst join't (Nachbarn in derselben Tabelle), muss man `AND a.name != b.name` oder `AND a.id != b.id` einfügen — sonst wird das Objekt sich selbst als Nachbar zurückgegeben!

8.3 Objekte in Pufferzone

Kernidee: “Alle Orte innerhalb von X km um einen Fluss/eine Linie” — Buffer erzeugen, dann schneiden.

Puffer-Abfrage — Rezept:

```
-- Variante 1: ST_DWithin (direkter, effizienter)  
SELECT a.name  
FROM tabelle_a a  
JOIN tabelle_b b ON ST_DWithin(a.geom, b.geom, radius_meter)  
WHERE b.name = 'Referenzlinie';  
  
-- Variante 2: Expliziter Buffer (wenn Buffer-Geometrie auch benötigt wird)  
SELECT a.name  
FROM tabelle_a a  
JOIN tabelle_b b ON ST_Intersects(  
    a.geom,  
    ST_Buffer(b.geom, radius_meter)  
)  
WHERE b.name = 'Referenzlinie';
```

Beispiel — Übung 5.4: Orte in 2-km-Pufferzone um Emme:

```
SELECT o.name  
FROM orte o  
JOIN fluesse f ON ST_DWithin(o.geom, f.geom, 2000)  
WHERE f.name = 'Emme';
```

Beispiel — Übung 5.5: Gemeinden durch die Emme fließt:

```
SELECT g.name
FROM gemeinden g
JOIN fluesse f ON ST_Intersects(g.geom, f.geom)
WHERE f.name = 'Emme';
```

9. Geometrische Verarbeitungs-Funktionen (Geometrie zurück)

Kernidee: Diese Funktionen erzeugen neue Geometrien aus bestehenden — sie sind das Werkzeug der Raumanalyse.

9.1 Puffer (Buffer)

```
ST_Buffer(geom, distanz_in_crs_einheiten)
```

Erzeugt eine Pufferzone um eine Geometrie. Distanz in den Einheiten des CRS!

9.2 Verschneidung (Intersection/Overlay)

Verschneidungs-Funktionen — Rezept:

```
-- Gemeinsamer Teil (Schnittmenge):
ST_Intersection(g1, g2)

-- Vereinigung:
ST_Union(g1, g2)

-- Differenz (g1 minus g2):
ST_Difference(g1, g2)

-- Symmetrische Differenz (alles ausser Schnittmenge):
ST_SymDifference(g1, g2)
```

Wichtiger Hinweis: Immer `ST_Intersects` in WHERE-Bedingung kombinieren, damit keine leeren Geometrien erzeugt werden und der Index genutzt wird:

```
SELECT
  g1.name,
  ST_Intersection(g1.geom, g2.geom) AS schnitt_geom
FROM tabelle_1 g1, tabelle_2 g2
WHERE ST_Intersects(g1.geom, g2.geom);
```

Beispiel — Übung 6.5: Flugschneise (gepuffert) mit Gemeinden verschneiden:

```
SELECT
  g.name,
```

```

ST_Intersection(
  ST_Buffer(f.geom, 2000),
  g.geom
) AS schnitt_geom
FROM flugschneise f, gemeinden g
WHERE ST_Intersects(ST_Buffer(f.geom, 2000), g.geom);

```

Beispiel — Differenz (Teile ausserhalb eines Radius):

```

SELECT
  ST_Transform(
    ST_Difference(
      ST_Transform(geom, 2056),
      ST_Buffer(ST_Transform(ST_Centroid(geom), 2056), 3000)
    ),
    4326
  ) AS ausserhalb_geom
FROM gemeinden
WHERE name = 'Uster';

```

9.3 Weitere geometrische Funktionen

ST_Centroid(geom)	-- Schwerpunkt/Mittelpunkt
ST_Envelope(geom)	-- Minimales umgebendes Rechteck (Bounding Box)
ST_ConvexHull(geom)	-- Konvexe Hülle
ST_PointOnSurface(geom)	-- Garantierter Punkt auf/in der Fläche

Tip: ST_Centroid vs. ST_PointOnSurface:

- ST_Centroid kann **ausserhalb** der Geometrie liegen (z.B. bei Hufeisenform)
- ST_PointOnSurface liegt **immer innerhalb** der Geometrie — für Point-on-Polygon-Abfragen verwenden

Exam Trap: Geometrische Funktionen können unerwartete Typen zurückgeben!

- ST_Intersection zweier Polygone kann POINT, LINESTRING oder POLYGON zurückgeben
- ST_Union mehrerer Polygone kann MULTIPOLYGON zurückgeben
- Immer mit ST_GeometryType(geom) prüfen wenn unklar
- Cast mit ::geometry wenn nötig

10. Daten laden: Werkzeuge

Kernidee: Geodaten kommen in verschiedenen Formaten — die richtigen Import-Werkzeuge kennen ist Voraussetzung für jede Analyse.

Import-Werkzeuge im Überblick:

| Werkzeug | Format | Verwendung | |———|———|———| | QGIS DB-Manager | GeoPackage, Shapefile, GeoJSON | Kleine/mittlere Datenmengen, GUI | | **ogr2ogr** | Alle Formate (GDAL) | Grosse Daten, Kommandozeile | | **shp2pgsql** | Shapefile → SQL | Älteres Tool, immer noch verbreitet | | Manuelles INSERT | WKT | Einzelne Objekte, Tests | | Overpass API / PostPASS | OSM-Daten | OpenStreetMap-Abfragen |

ogr2ogr Befehl für GeoPackage → PostGIS:

```
ogr2ogr -f PostgreSQL PG:"dbname='gisdb' user='postgres' password='postgres'"
-overwrite Schweiz.gpkg
```

11. OpenStreetMap und PostPASS

Kernidee: OpenStreetMap ist die grösste freie Geodaten-Quelle — PostPASS bietet SQL-Zugang zu weltweiten OSM-Daten via PostGIS-Schema.

11.1 OSM-Datenmodell

Tip: OSM physisches Schema (Nodes, Ways, Relations):

- **Nodes** = Punkte mit Koordinaten + optionale Tags
- **Ways** = geordnete Liste von Nodes (ohne eigene Koordinaten)
- **Relations** = Gruppierung von Nodes/Ways/Relations
- **Tags** = Key-Value-Paare für Attribute (z.B. `amenity=restaurant`)

Vorsicht: Flächen in OSM = geschlossene Ways oder Relations — kein eigenständiger Flächentyp!

11.2 PostPASS-Schema

PostPASS bietet ein GIS-freundliches Schema:

```
postpass_point    -- OSM Nodes mit Tags
postpass_line     -- OSM Ways als Linien (Strassen, etc.)
postpass_polygon  -- Geschlossene Ways als Flächen
postpass_pointpolygon -- View: Points + Polygone kombiniert (für gemischte Suchen)
```

Gemeinsame Attribute aller PostPASS-Tabellen:

- `osm_type` & `osm_id` — interner OSM-Identifikator
- `tags` — vom Typ `json` (Key-Value-Paare)
- `geom` — vom Typ `GEOMETRY` in EPSG:4326

PostPASS SQL-Abfrage — Rezept:

```
-- Grundmuster: Objekte nach Tag filtern + Bounding Box
SELECT
  osm_id,
  coalesce(tags->>'name', '?') AS name,
  tags->>'attribut_key' AS attribut,
  ST_PointOnSurface(geom) AS geom
FROM postpass_pointpolygon
WHERE tags->>'amenity' = 'restaurant'
      AND geom && {{bbox}};

-- Tags-Array-Suche (wenn Wert mehrere mit ';' getrennte Werte hat):
WHERE string_to_array(tags->>'cuisine', ';') @> '{kebab}'
```

Beispiel — Restaurants mit Kebab-Küche in Bounding Box:

```
SELECT
  osm_id,
  coalesce(tags->>'name', '?') AS name,
  tags->>'cuisine' AS cuisine,
  ST_PointOnSurface(geom) AS geom
FROM postpass_pointpolygon
WHERE tags->>'amenity' IN ('fast_food', 'restaurant')
      AND string_to_array(tags->>'cuisine', ';') @> '{kebab}'
      AND geom && {{bbox}};
```

Tip: JSON-Tags in PostPASS abfragen:

- tags->>'key' gibt den Wert als Text zurück (null wenn nicht vorhanden)
- coalesce(tags->>'name', '?') gibt '?' zurück wenn kein Name vorhanden
- tags->>'amenity' IN ('restaurant', 'cafe') für mehrere Werte
- string_to_array(tags->>'key', ';') @> '{wert}' wenn Wert mit Semikolon getrennt ist (OSM-Konvention)

12. Spatial Relationships — Konzept

Kernidee: Räumliche Beziehungen zwischen Objekten können direkt aus der Geometrie abgeleitet werden — ohne explizite Modellierung einer Beziehungstabelle.

12.1 Typen räumlicher Beziehungen

Nähe (Proximity):	Distanz zwischen zwei Objekten
Nachbarschaft:	Grenzen aneinander (touching/adjacent)
Containment:	Ein Objekt liegt in einem anderen
Überschneidung:	Objekte haben gemeinsame Fläche

12.2 Klassifikation von Raumanalysefunktionen (nach Keller)

Funktionsklassen und Rückgabetypen:

Klasse	Rückgabe	Beispiele	Management/Constructor
Geometrie	ST_MakePoint, ST_GeomFromText	Output/Conversion	Text/JSON
ST_AsText, ST_AsGeoJSON	Measurement	Zahl	ST_Length, ST_Area, ST_Distance
Relationship/Filter	Boolean	ST_Intersects, ST_DWithin, ST_Equals	Spatial Aggregate
Geometrie	ST_Union (aggregiert mehrere)	Processing	Geometrie
ST_Buffer, ST_Intersection, ST_Difference			

Entscheidungsregel:

- Brauche ich ein JA/NEIN? → Relationship-Funktion (WHERE-Klausel)
- Brauche ich eine Zahl? → Measurement-Funktion (SELECT oder ORDER BY)
- Brauche ich eine neue Geometrie? → Processing-Funktion (SELECT)

13. Räumliche Indexe

Kernidee: Ohne räumlichen Index muss PostgreSQL bei jeder Abfrage alle Geometrien prüfen — mit GiST-Index werden nur geometrisch nahe Objekte berücksichtigt.

GiST-Index erstellen — Rezept:

```
CREATE INDEX indexname
ON tabellenname
USING gist(geom);

-- Konvention für Indexname: idx_tabellenname_geom
CREATE INDEX idx_gemeinden_geom
ON gemeinden
USING gist(geom);
```

Wie der Index funktioniert:

- GiST = Generalized Search Tree
- Implementiert als R-Baum (R-tree): mehrdimensionale, dynamische, balancierte Datenstruktur
- Speichert Bounding Boxes der Geometrien hierarchisch
- ST_Intersects, ST_DWithin, &&-Operator nutzen den Index automatisch

Tip: Wann wird der Index genutzt?

- ST_Intersects(a.geom, b.geom) → nutzt Index ✓
- a.geom && b.geom → nutzt Index, nur Bounding Box ✓ (schneller, weniger präzise)
- ST_Distance(a.geom, b.geom) < 1000 → nutzt Index **nicht** automatisch → ST_DWithin bevorzugen!

Beste Praxis: Immer ST_DWithin statt ST_Distance(...) < d für gefilterte Abfragen.

14. Geo-Normen und Datenformate

Kernidee: Geodaten kommen in vielen Formaten — die wichtigsten kennen und einordnen können.

Vektor-Formate im Überblick:

Format	Typ	Status	Hinweis	Shapefile	Vektor	Veraltet
Immer noch verbreitet; viele Einschränkungen				GeoJSON	Vektor	Aktuell
W3C-Standard; direkt in Browser nutzbar				GeoPackage	Vektor+Raster	Aktuell
SQLite-basiert; OGC-Standard; empfohlen				KML	Vektor	Aktuell
XML; Google-Ursprung; OGC				Interlis	Vektor	CH-spezifisch
Amtliche CH-Daten				WFS	Webservice	Veraltet (→ OGC API)
XML-basiert				OGC API Features	Webservice	Aktuell
REST + GeoJSON; WFS-Nachfolger						

Raster-Formate:

Format	Hinweis	GeoTIFF	Standard für Rasterdaten; enthält CRS-Info
WMS	Web Map Service; Bilder als Kacheln	OGC API Maps	WMS-Nachfolger

15. Offene Geodatenquellen

Kernidee: Der “Raumbezug als universeller Fremdschlüssel” erlaubt die Verknüpfung beliebiger Datensätze über Koordinaten — freie Datenquellen kennen ist prüfungsrelevant.

Wichtige freie Geodatenquellen:

Schweiz:

- map.geo.admin.ch — Swisstopo, offizielle CH-Geodaten
- opendata.swiss — Schweizer Open Government Data Portal
- Bundesamt für Statistik — statistische Geodaten
- GeoHarvester.ch — aggregiert CH-Geodaten

International:

- osm.org / Overpass Turbo — OpenStreetMap, weltweit, frei
- PostPASS (postpass.geofabrik.de) — SQL-Zugang zu OSM-Daten
- NASA SRTM — Höhenmodell weltweit
- datasetsearch.research.google.com — Google Dataset Search
- giswiki.ch/Freie_Geodaten — Sammlung freier Geodaten

API (teils Freemium):

- Google Maps API
- OpenMapTiles API

16. Vollständige Beispiel-Queries (Übungsreferenz)

Übung 4: Orte im Umkreis von 10 km (mit CTE)

```
WITH mylocation AS (  
  SELECT ST_Transform(  
    ST_GeomFromText('POINT(8.81642 47.22541)', 4326),  
    21781  
  ) AS geom  
)  
SELECT  
  o.name,  
  ST_AsText(o.geom, 0) AS koordinaten,  
  ST_Distance(o.geom, m.geom)::int AS distanz_m  
FROM orte o  
JOIN mylocation m ON ST_DWithin(o.geom, m.geom, 10000)  
ORDER BY distanz_m;
```

Was passiert hier:

1. CTE mylocation: Punkt aus WGS84-Koordinaten wird in CH1903 (21781) transformiert
2. ST_DWithin: filtert Orte innerhalb 10 km (= 10000 m)
3. ST_Distance: berechnet genaue Distanz in Metern
4. ::int: castet auf Integer (keine Dezimalstellen)
5. ORDER BY distanz_m: sortiert nach Nähe

Übung 5.2: Angrenzende Gemeinden

```
SELECT a.name  
FROM gemeinden a  
JOIN gemeinden b ON ST_Intersects(a.geom, b.geom)  
WHERE b.name = 'Rapperswil-Jona'  
      AND a.name != 'Rapperswil-Jona';
```

Übung 5.3: Orte im Umkreis (feste Koordinaten)

```
SELECT name  
FROM orte  
WHERE ST_DWithin(  
  geom,  
  ST_SetSRID(ST_MakePoint(704472, 231216), 21781),  
  10000  
);
```

Übung 5.4: Orte in Pufferzone um Fluss

```
SELECT DISTINCT o.name  
FROM orte o  
JOIN fluesse f ON ST_DWithin(o.geom, f.geom, 2000)  
WHERE f.name = 'Emme';
```

Übung 5.5: Gemeinden durch die der Fluss fließt

```
SELECT DISTINCT g.name
FROM gemeinden g
JOIN fluesse f ON ST_Intersects(g.geom, f.geom)
WHERE f.name = 'Emme';
```

Übung 6.5: Flugschneise gepuffert × Gemeinden verschneiden

```
SELECT
  g.name,
  ST_Intersection(
    ST_Buffer(f.geom, 2000),
    g.geom
  ) AS schnitt_geom
FROM flugschneise f
JOIN gemeinden g ON ST_Intersects(ST_Buffer(f.geom, 2000), g.geom);
```

Tip: Attribute die SQL-Keywords sind — immer in Anführungszeichen:

```
SELECT "name", "date", "type" FROM ...
--      ↑ name ist kein Keyword aber Konvention
--      ↑ date ist SQL-Keyword → muss gequotet werden!
```

DISTINCT nicht vergessen wenn JOIN mehrere Treffer pro Objekt erzeugen kann!

17. Zusammenfassung: Entscheidungsbaum für räumliche Abfragen

Wenn die Aufgabe lautet... dann benutze...

- “Alle X **in einem Umkreis** von d km um P” → ST_DWithin(x.geom, p.geom, d_in_meter) + CRS prüfen!
- “Alle X **die an Y angrenzen**” → ST_Intersects(x.geom, y.geom) + Selbstbezug ausschliessen
- “Alle X **durch die Y fließt/geht**” → ST_Intersects(x.geom, y.geom)
- “Alle X **in einer Pufferzone** von d km um Y” → ST_DWithin(x.geom, y.geom, d) oder ST_Intersects(x.geom, ST_Buffer(y.geom, d))
- “**Distanz** zwischen A und B” → ST_Distance(ST_Transform(a.geom, 2056), ST_Transform(b.geom, 2056))
- “**Schnittfläche** von A und B” → ST_Intersection(a.geom, b.geom) WHERE ST_Intersects(a.geom, b.geom)
- “**Distanz in Metern** bei EPSG:4326-Daten” → Entweder ST_DistanceSphere(g1, g2) oder ST_Distance(ST_Transform(g1, 2056), ST_Transform(g2, 2056))
- “Koordinaten aus **WKT-Text** erstellen” → ST_GeomFromText('POINT(lon lat)', epsg)

- “Geometrie in **anderen EPSG** umrechnen” → `ST_Transform(geom, ziel_epsg)`

Week 5: [Topic Title]

Woche 5: Big Data Analytics und Cloud Platforms

1. Big Data: Grundbegriffe und Definitionen

1.1 Was ist Big Data?

Kernidee: Big Data bezeichnet Datenmengen, die mit herkömmlichen Methoden nicht mehr verarbeitet werden können — zu gross, zu komplex, zu schnelllebig oder zu schlecht strukturiert.

Tip: Woran erkenne ich eine Big-Data-Frage?

- Aufgabe fragt nach Definition, Merkmalen oder Einordnung von Datenmengen
- Stichwörter: “4 V’s”, “5 V’s”, “Volume”, “Velocity”, “Variety”, “Veracity”, “Value”
- Frage: “Ist das Big Data?” → Einordnung in Small/Medium/Big

1.2 Die 4 (bzw. 5) V’s von Big Data

Definition durch die vier Dimensionen:

Volume — Datenmenge: Die schiere Grösse der Daten (Terabyte bis Petabyte).

Variety — Vielfalt: Strukturierte, semi-strukturierte und unstrukturierte Daten (Tabellen, JSON, Bilder, Videos, Sensordaten).

Velocity — Geschwindigkeit: Wie schnell Daten entstehen und verarbeitet werden müssen (Echtzeit-Streams, Batch).

Veracity — Wahrhaftigkeit/Qualität: Wie zuverlässig und korrekt die Daten sind (Rauschen, Fehler, Inkonsistenzen).

Value (seit 2015, 5. V) — Geschäftswert: Der Erkenntnisgewinn, der zu besseren Entscheidungen oder Profit führt.

Entscheidungshilfe: Welches V ist gemeint?

- Daten wachsen auf TB/PB → **Volume**
- Bilder + JSON + CSV gemischt → **Variety**
- Echtzeit-Sensorstream → **Velocity**
- Datenfehler, Rauschen → **Veracity**
- Nutzen für das Unternehmen → **Value**

1.3 Small / Medium / Big Data (Keller, 2020)

Eine pragmatische Einteilung nach physikalischen Grenzen:

Small Data: Daten passen vollständig in den Arbeitsspeicher (RAM) eines einzelnen Rechners. Typische Tools: R, Python/Pandas DataFrames.

Medium Data: Daten passen auf die Festplatte eines einzelnen Laptops/Servers, aber nicht in den RAM. Typische Tools: Apache Spark (lokal), Python mit DuckDB.

Big Data: Daten passen weder in RAM noch auf die Disk eines einzelnen Rechners. Verlangen nach **verteilten Systemen** (Distributed Systems).

Exam Trap: Häufiger Denkfehler: “Alles ist Big Data” Jordan Tigani (“Big Data is Dead”, 2023) zeigt: Die meisten Daten sind kleiner als 100 GB. Nicht jedes Datenproblem braucht ein verteiltes System. Zuerst prüfen: Passt es in RAM? → Small Data reicht. Passt es auf Disk? → Medium Data-Tools (DuckDB, Spark lokal) reichen. Nur wenn nicht: verteilte Systeme.

1.4 Big Data-Lösungsansätze: Scale Up vs. Scale Out

Lösungsansatz 1 — Scaling Up (vertikal): Mehr RAM, schnellere CPUs in einem einzelnen Rechner. → Technologie: **In-Memory Column Stores**

Lösungsansatz 2 — Scaling Out (horizontal): Viele normale Rechner zusammenschalten (“Shared Nothing“-Architektur mit Parallel Processing). → Technologie: **Verteilte Systeme** (MapReduce, Apache Spark)

2. Analytical Processing-Technologien

2.1 Column Stores (Spaltenorientierte DBMS)

Kernidee: Statt Daten zeilenweise zu speichern (wie in klassischen OLTP-Datenbanken), werden sie spaltenweise abgelegt — das ermöglicht enorme Lese-Performance bei analytischen Abfragen.

Vergleich Row Store vs. Column Store:

- **Row Store:** Alle Attribute einer Zeile liegen zusammen auf Disk. Gut für: `SELECT * FROM t WHERE id = 5` (einzelner Datensatz).
- **Column Store:** Alle Werte einer Spalte liegen zusammen. Gut für: `SELECT AVG(preis) FROM verkauf` (aggregieren über viele Zeilen, wenige Spalten).

Vorteile Column Store:

- Schnelle OLAP-Lese-Anfragen (10 bis 100× schneller als Row Stores)
- Bessere Komprimierung pro Spalte (gleiche Werte → Run-Length-Encoding etc.)
- Bessere Indexierung möglich

Nachteile Column Store:

- Langsame Updates (alle Spalten-Dateien müssen geändert werden)
- Nicht geeignet für write-heavy OLTP-Workloads

2.2 In-Memory Stores

Kernidee: Das DBMS nutzt ausschliesslich den Arbeitsspeicher als Datenspeicher — kein Disk-I/O bei Abfragen, daher extrem schnell.

Problem: Persistenz! RAM ist flüchtig. **Lösungen:**

- Logisches Write-Ahead-Log (WAL) auf Disk schreiben

- Snapshots (“eventuelle Persistenz”)

Oft kombiniert: In-Memory + Column Store zusammen = maximale OLAP-Performance.

Bekannte Systeme (Auswahl):

- **SAP HANA** — Spaltenorientiert, In-Memory (min. 500 GB RAM)
- **Oracle 12+** — In-Memory-Erweiterung spaltenorientiert
- **MS SQL Server** — Spaltenorientierte Erweiterung (nicht In-Memory)
- **PostgreSQL** — Erweiterungen: pg_duckdb, Citus, TimescaleDB
- **DuckDB** — Spaltenorientiert, In-Process, Open Source (kann über RAM hinaus skalieren)

2.3 Weitere Query-Optimierungen

Materialized Views: Vorberechnete Abfrageergebnisse als eigene Struktur gespeichert (Caching auf Abfrageebene).

Bitmap-Indexe: Besonders geeignet für Data Warehouses mit niedrig-kardinalen Spalten (z.B. Geschlecht, Status). Jeder Wert bekommt ein Bit-Array über alle Zeilen.

Parallel Query Processing: Nutzung mehrerer CPU-Kerne oder GPUs gleichzeitig.

Volcano Style vs. Vectorized Style:

Volcano Style (Tupel-at-a-Time):

- Jede Operation verarbeitet ein Tupel vollständig, bevor das nächste kommt
- Einfach zu implementieren
- Overhead: viele Funktionsaufrufe und Kontextwechsel

Vectorized Style (Batch-Verarbeitung):

- Verarbeitet Daten in Batches (Arrays/Vektoren von Werten)
- Passt natürlich zu Column Stores
- Weniger Funktionsaufrufe → weniger Overhead
- Nutzt moderne CPU-Architekturen: **SIMD (Single Instruction, Multiple Data)** — eine CPU-Instruktion auf viele Datenwerte gleichzeitig angewendet

Exam Trap: Achtung: Verwechslungsgefahr “Vectorized Processing” hat **nichts** mit “Vector Stores” (für AI/Embeddings bei ChatGPT) zu tun! Es geht um CPU-effiziente Batch-Verarbeitung von Spalten-Arrays.

3. Überblick: Data Processing Tools

Entscheidungsrahmen: Welches Tool für welche Datengröße?

1. Daten passen in RAM → **Pandas** (einfach, eager), **Polars** (schnell, lazy+eager), **DuckDB** (SQL, in-process)
2. Daten passen nicht in RAM, aber auf einen Rechner → **DuckDB** (skaliert über RAM), **Dask/Modin** (Pandas-API, erweiterbar)

3. Daten brauchen Cluster → **Apache Spark** (Batch+Streaming), **Apache Flink** (Stream-first)
4. Cloud-Managed → **Databricks** (Spark-basiert), **Snowflake**, **BigQuery** (Google), **Redshift** (AWS)

Vergleichstabelle der wichtigsten Eigenschaften:

Tip: Relationale Tabelle vs. Pandas vs. Polars vs. DuckDB — Erkennungsmerkmale:

- **Schema:** Relationale DB und DuckDB strikt vorab definiert; Pandas inferiert und änderbar; Polars strikt und unveränderlich
- **Auswertung:** Relationale DB und DuckDB deklarativ (SQL); Pandas eager (sofort); Polars lazy + eager
- **Parallelität:** Pandas single-thread; Polars, DuckDB, relationale DB multi-thread
- **Fehlende Werte:** SQL/DuckDB → NULL; Pandas → NaN/None; Polars → Null (typisiert)
- **Skalierung:** Pandas RAM-limitiert; DuckDB skaliert über RAM; Relationale DB vertikal/horizontal

4. MapReduce

4.1 Grundidee und Herkunft

Kernidee: MapReduce ist ein einfaches, funktionales Programmiermodell für die verteilte Verarbeitung sehr grosser Datenmengen auf vielen Standard-Rechnern.

Ursprung: Google (Dean & Ghemawat, 2004). Später: Apache Hadoop-Implementierung.

4.2 Die drei Phasen

Map-Phase:

- Eingabe: Schlüssel/Wert-Paare (key/value pairs)
- Ausgabe: Zwischen-Schlüssel/Wert-Paare (intermediate key/value pairs)
- Jeder Map-Worker verarbeitet einen Teil der Eingabe unabhängig

Shuffle/Partition-Phase:

- Zwischen-Paare werden nach Schlüssel gruppiert
- Standard-Partitionierung: $\text{hash}(\text{key}) \bmod N$ (wobei N = Anzahl Reducer)
- Ziel: Gleiche Schlüssel landen beim selben Reducer

Reduce-Phase:

- Eingabe: Ein Schlüssel + alle zugehörigen Zwischenwerte
- Ausgabe: Aggregiertes Ergebnis (z.B. Summe, Anzahl)
- Fasse alle Zwischenwerte mit demselben Schlüssel zusammen

MapReduce-Rezept: Wörter zählen (Word Count)

Aufgabe: Zähle wie oft jedes Wort in einem grossen Textkorpus vorkommt.

1. **Map:** Für jedes Wort w im Input $\rightarrow$ emittiere $(w, 1)$
 - Eingabe: "the cat and the dog" $\rightarrow$ Ausgabe: $[("the", 1), ("cat", 1), ("and", 1), ("the", 1), ("dog", 1)]$
2. **Shuffle:** Gruppiere nach Schlüssel
 - "the" $\rightarrow [1, 1]$; "cat" $\rightarrow [1]$; "and" $\rightarrow [1]$; "dog" $\rightarrow [1]$
3. **Reduce:** Summiere die Werte pro Schlüssel
 - "the" $\rightarrow 2$; "cat" $\rightarrow 1$; "and" $\rightarrow 1$; "dog" $\rightarrow 1$

Allgemeines Muster: Map emittiert (k, v) -Paare; Reduce aggregiert alle v mit gleichem k .

4.3 Vor- und Nachteile MapReduce

Vorteile:

- Einfaches Programmiermodell
- Automatische Parallelisierung und Fehlerbehandlung
- Läuft auf vielen Standard-Rechnern (commodity hardware)
- Hohe Verfügbarkeit

Nachteile:

- **Master ist Single Point of Failure** und Skalierungsengpass
- Latenz beim Öffnen von tausenden Dateien (disk-intensiv)
- Nicht geeignet für Echtzeit-Verarbeitung
- Inzwischen durch Apache Spark weitgehend ersetzt

MapReduce heute noch relevant für:

- Massive Batch-Jobs (Petabyte-Verarbeitung)
- Log-Analysen
- Verteilte Berechnungen ohne Echtzeitanforderung

Nicht geeignet für: iterative Algorithmen (ML), interaktive Abfragen, Streaming

5. Apache Spark

5.1 Was ist Apache Spark?

Kernidee: Apache Spark ist ein Open-Source Framework für verteilte Datenverarbeitung, das durch **In-Memory Computing** 10–100× schneller als Hadoop MapReduce ist.

Stärken von Spark:

- **Iterative Programme** (Machine Learning) — Daten bleiben im RAM, kein wiederholtes Disk-Lesen
- **Interaktive Abfragen** — schnelles Explorieren grosser Daten
- **Unified Engine** — Batch + Streaming + SQL + ML + Graph in einem Framework

5.2 Spark-Architektur

Driver-Prozess: Zentrales Steuerelement. Koordiniert alle Aufgaben, erstellt den Execution Plan, verteilt Arbeit an Executors.

Executors (Worker Nodes): Führen die eigentlichen Berechnungen aus. Verarbeiten Partitionen der Daten parallel.

Ressourcenmanagement: Primär über **Kubernetes (K8s)**; früher auch YARN/Mesos.

Catalyst Optimizer: Spark's interner Query-Optimierer. Analysiert den logischen Plan und erzeugt einen optimierten physischen Ausführungsplan.

Separation of Compute & Storage: Rechenleistung von Speicher getrennt. Storage in Objektspeichern (AWS S3, Azure ADLS) oder Lakehouse-Formaten (Delta Lake, Apache Iceberg).

5.3 Spark-Komponenten

Spark SQL:

- SQL-Abfragen direkt auf Spark-Daten
- JDBC-API für BI-Tools
- ETL aus verschiedenen Formaten
- Aufruf: `spark.sql("SELECT ...")`

Unterstützte Datenformate:

- **Parquet** — spaltenorientiertes Dateiformat (bevorzugt für Analytics)
- JSON, Protobuf, Avro, ORC (ebenfalls column-based wie Parquet)

DataFrame/Dataset-API:

- Unified API seit Spark 2.0
- `Dataset[T]` (Scala/Java) mit Compile-Zeit-Typsicherheit
- `DataFrame` = `Dataset` ohne Typsicherheit (Python/R)

5.4 Spark DataFrames — Kernkonzepte

Immutabilität: DataFrames sind unveränderlich. Jede Transformation erzeugt einen **neuen** DataFrame. Der ursprüngliche bleibt unberührt.

Lazy Evaluation: Transformationen (z.B. `filter()`, `select()`, `groupBy()`) werden **nicht sofort ausgeführt**, sondern als DAG (Directed Acyclic Graph) von Operationen gespeichert. Ausführung erst bei einer **Action**.

Actions lösen die Ausführung aus:

- `show()` — erste N Zeilen anzeigen
- `collect()` — alle Daten zum Driver bringen
- `count()` — Anzahl Zeilen zählen
- `write()` — Ergebnis speichern

Partitionierung: DataFrames sind intern in Partitionen aufgeteilt, die parallel auf dem Cluster verarbeitet werden.

- `repartition(n)` — Neupartitionierung auf n Partitionen (shuffle)
- `coalesce(n)` — Reduziert Partitionen ohne shuffle (effizienter)

Exam Trap: Häufiger Denkfehler: Pandas $\neq$ Spark DataFrames "DataFrame" ist ein **Konzept** (strukturierte Tabelle mit Schema). Pandas und Spark **implementieren** dieses Konzept unterschiedlich:

- Pandas: eager, single-machine, mutable, RAM-limitiert
- Spark: lazy, distributed, immutable, cluster-skalierbar

Der Code sieht ähnlich aus, aber das Verhalten ist fundamental verschieden!

5.5 PySpark — Praxis-Rezept

PySpark Session starten und DataFrame erstellen

```
from pyspark.sql import SparkSession

# 1. SparkSession erstellen (Entry Point)
spark = SparkSession.builder \
    .appName("MeinProjekt") \
    .getOrCreate()

# 2. DataFrame aus Python-Liste erstellen
data = [(1, "Alice", 85), (2, "Bob", 92)]
df = spark.createDataFrame(data, ["ID", "Name", "Score"])

# 3. DataFrame anzeigen (Action!)
df.show()

# 4. SQL: Temporäre View registrieren
df.createOrReplaceTempView("studenten")
result = spark.sql("SELECT Name, Score FROM studenten WHERE Score > 90")
result.show()

# 5. DataFrame-API verwenden
from pyspark.sql.functions import col, avg
df.filter(col("Score") > 80) \
    .groupBy("Name") \
    .agg(avg("Score").alias("Durchschnitt")) \
    .show()
```

Schritte allgemein:

1. SparkSession erstellen
2. Daten laden (CSV, Parquet, Datenbank, Python-Liste)
3. Transformationen definieren (lazy)
4. Action aufrufen → Ausführung
5. Ergebnis anzeigen oder speichern

5.6 Spark SQL vs. DataFrame-API

Tip: Wann Spark SQL, wann DataFrame-API?

- **Spark SQL:** Wenn du komplexe SQL-Abfragen kennst; gut für ad-hoc Exploration; `spark.sql("SELECT ...")`
- **DataFrame-API:** Wenn du programmatisch transformieren willst; besser für ML-Pipelines; `df.filter(...).groupBy(...)`
- **Beide sind funktional äquivalent!** Der Catalyst Optimizer optimiert beide gleich.

5.7 Spark vs. Hadoop MapReduce

Merkmal	Hadoop MapReduce	Apache Spark
Speicher	Disk-basiert	In-Memory
Geschwindigkeit	Langsam (Disk-I/O)	10-100× schneller
Echtzeit	Nein	Ja (Streaming)
ML/Iterativ	Schlecht	Gut
API	Java-only	Scala, Java, Python, R
Zustand	Legacy	Aktuell Standard

6. Cloud Computing und Cloud Platforms

6.1 Definition Cloud Computing

NIST-Definition: Ein Modell für den bequemen, bedarfsorientierten Netzzugang zu einem gemeinsam genutzten Pool konfigurierbarer Rechenressourcen (Netze, Server, Speicher, Anwendungen, Dienste), die mit minimalem Verwaltungsaufwand schnell bereitgestellt und freigegeben werden können.

Kernprinzip: Elastizität — Ressourcen nach Bedarf hochfahren und wieder freigeben. Zahlen nur für das, was genutzt wird.

6.2 Historische Entwicklung Cloud & Datenbanken

- **2003–2006:** Google-Paper (GFS, MapReduce, Bigtable) → Grundlage von Hadoop; AWS-Gründung
- **2007–2010:** NoSQL-“Revolution”, CAP-Theorem, erste DBaaS (Amazon RDS)
- **2010–2015:** Cloud Data Warehouses (Redshift, BigQuery), Apache Spark
- **2015–2019:** Compute/Storage-Trennung als Architekturprinzip, Snowflake, Serverless, HTAP
- **2019–2022:** Data Lakehouse, Open Formats (Apache Iceberg, Delta Lake), Data Mesh
- **2022–2024:** Vector Databases, RAG, Edge-DBs, AI-Integration
- **2025+:** Rückkehr zu On-Premises?

Fazit der Geschichte: SQL wurde mehrfach totgesagt und lebt jedes Mal stärker wieder auf. Die eigentliche Revolution war die **Trennung von Compute und Storage** als Architekturprinzip.

6.3 Technologischer Wandel seit 2000

- **“RAM is King”:** Arbeitsspeicher ist die primäre Datenquelle; Festplatte = Fallback
- Netzwerk-Durchsatz wird kontinuierlich besser
- Multi-Core CPUs sind Commodity (normale Ware)
- Volume **und** Velocity werden immer wichtiger

6.4 Wichtige Cloud Platforms

Grosse Anbieter:

- **Amazon Web Services (AWS)** — Marktführer
- **Microsoft Azure** — starke Enterprise-Präsenz
- **Google Cloud Platform (GCP)** — ML/AI-Stärken, BigQuery
- **Google Earth Engine (GEE)** — Geo/Raster-Analyse

Spezialisierte Plattformen:

- **Databricks** — kommerzielle Plattform auf Apache Spark

- **Snowflake** — kommerzielles Cloud Data Warehouse
- **Google BigQuery** — serverless, columnar, Google Cloud
- **Amazon Redshift** — AWS Data Warehouse, columnar
- **ClickHouse** — Open Source, columnar

6.5 Cloud vs. On-Premises: Vor- und Nachteile

Entscheidungsrahmen Cloud vs. On-Prem

Vorteile Cloud gegenüber On-Premises:

1. Skalierbarkeit und Flexibilität (elastisch)
2. Geringere Initialkosten (kein Hardware-Kauf)
3. Schnelles Prototyping (hochfahren, testen, wegwerfen)
4. Weniger IT-spezialisiertes Personal nötig
5. Grosse Unternehmen: erzwingt Modernisierung von Architektur und Prozessen

Nachteile Cloud gegenüber On-Premises:

1. Man muss sich trotz Werbung weiterhin um Technik, Integration und Betrieb kümmern
2. Verlust physischer Kontrolle über Daten
3. Datenschutz-Risiken (GDPR-Compliance schwerer sicherzustellen)
4. “Lift and Shift” ist kurzfristig günstiger, langfristig aber teurer (Cloud-native Anpassungen nötig)
5. Fehlende Standards für Daten-Migration zwischen Cloud-Anbietern
6. **Vendor Lock-in** — Abhängigkeit vom Cloud-Anbieter

Exam Trap: Häufiger Denkfehler: “Cloud = kein Aufwand” Die Cloud-Werbung suggeriert, man müsse sich um nichts kümmern. In der Praxis: Technik, Integration, Sicherheit und Betrieb bleiben Aufgaben des Nutzers. Nur die Hardware-Verwaltung entfällt.

Häufiger Denkfehler: “Lift and Shift spart langfristig Geld” Schnelle Migration in die Cloud (“Lift and Shift”) ist anfangs günstig, aber ohne Anpassung an Cloud-native Prinzipien (z.B. AWS Lambda, Container) nutzt man die Cloud-Vorteile nicht aus — und zahlt mehr als nötig.

Wann Cloud, wann On-Prem?

- Prototyping, schnell starten → **Cloud**
- Sensible Daten, GDPR-kritisch → **On-Prem** bevorzugen
- Unregelmässige Lastspitzen → **Cloud** (Elastizität)
- Stabile, vorhersagbare Last → **On-Prem** oft günstiger
- Startup ohne IT-Team → **Cloud**
- Multi-Cloud oder Wechsel geplant → **Vendor Lock-in** beachten!

Die Kernfrage ist nicht Cloud vs. On-Prem, sondern: die richtige Workload am richtigen Ort — das ist die eigentliche Kernkompetenz im modernen Data Engineering.

7. PySpark Praxis: Vollständiges Lösungsrezept für Übungsaufgaben

Allgemeines Rezept für PySpark-Aufgaben (Databricks)

Schritt 1: Umgebung aufsetzen

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Aufgabe").getOrCreate()
```

In Databricks ist die SparkSession oft schon als spark vorhanden.

Schritt 2: Daten laden

```
# CSV laden
df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("/pfad/zur/datei.csv")

# Parquet laden
df = spark.read.parquet("/pfad/zur/datei.parquet")
```

Schritt 3: Daten erkunden

```
df.printSchema()      # Schema anzeigen
df.show(5)             # erste 5 Zeilen
df.count()             # Anzahl Zeilen
df.describe().show()   # Statistiken
```

Schritt 4a: Abfragen mit Spark SQL

```
df.createOrReplaceTempView("meine_tabelle")
result = spark.sql("""
    SELECT spalte1, COUNT(*) as anzahl
    FROM meine_tabelle
    WHERE spalte2 > 100
    GROUP BY spalte1
    ORDER BY anzahl DESC
""")
result.show()
```

Schritt 4b: Abfragen mit DataFrame-API

```
from pyspark.sql.functions import col, count, avg, desc

result = df \
    .filter(col("spalte2") > 100) \
    .groupBy("spalte1") \
    .agg(count("*").alias("anzahl")) \
    .orderBy(desc("anzahl"))

result.show()
```

Schritt 5: Ergebnis speichern (optional)

```
result.write.format("parquet").save("/ausgabe/pfad")
```

Tip: Woran erkenne ich, ob Spark SQL oder DataFrame-API verlangt ist?

- Aufgabe zeigt `spark.sql(...)` → Spark SQL
- Aufgabe zeigt `.filter()`, `.groupBy()`, `.agg()` → DataFrame-API
- Beide sind äquivalent! Wenn nicht vorgegeben: wähle die, die du besser kennst
- In Databricks: beide im selben Notebook verwendbar

Exam Trap: Typische Fehler in PySpark-Aufgaben:

1. **Lazy Evaluation vergessen:** `df.filter(...)` allein gibt nichts aus. Immer Action (`show()`, `count()`, etc.) aufrufen!
2. **createOrReplaceTempView vergessen:** Vor `spark.sql(...)` muss der DataFrame als temporäre View registriert sein.
3. **Pandas-Syntax verwenden:** `df["spalte"]` kann in Spark funktionieren, aber `col("spalte")` ist idiomatisch und sicherer.
4. **Schema-Fehler:** Bei `inferSchema=False` sind alle Typen Strings. Dann: `.cast("integer")` etc. nötig.
5. **Grossen collect() auf Cluster:** `collect()` bringt ALLE Daten zum Driver — bei Big Data gefährlich (OutOfMemory)!

8. Zusammenfassung: Konzept-Landkarte**Schnellübersicht: Was ist was?****Big Data Merkmale:**

- 4 V's → Volume, Variety, Velocity, Veracity
- 5. V → Value (Geschäftswert)

Datengrößen:

- Small → RAM → Pandas, R
- Medium → Disk → DuckDB, lokaler Spark
- Big → Cluster → Apache Spark, Flink

Scale Up vs. Scale Out:

- Scale Up → In-Memory Column Stores (SAP HANA, DuckDB)
- Scale Out → MapReduce, Apache Spark

Query Processing Styles:

- Volcano → ein Tupel nach dem anderen
- Vectorized → Batches/Arrays (SIMD) → moderner, schneller

MapReduce = Map + Shuffle + Reduce:

- Map: $(k_{in}, v_{in}) \rightarrow (k_{mid}, v_{mid})$

- Shuffle: Gruppierung nach k_{mid}
- Reduce: $(k_{\text{mid}}, [v_1, v_2, \dots]) \rightarrow (k_{\text{out}}, v_{\text{out}})$

Apache Spark Kernbegriffe:

- Driver + Executors
- Lazy Evaluation $\rightarrow$ Action löst Ausführung aus
- Immutable DataFrames
- Catalyst Optimizer
- Spark SQL $\equiv$ DataFrame-API (gleiche Performance)

Cloud:

- Vorteile: Elastizität, niedrige Einstiegskosten, Prototyping
- Nachteile: Vendor Lock-in, GDPR, langfristig teurer, Schein-Einfachheit
- Kernfrage: richtige Workload am richtigen Ort

Week 6: [Topic Title]

Woche 6: Big Data Analytics, Cloud Platforms & Google Earth Engine

Überblick

Diese Woche behandelt drei Themenblöcke:

1. **Cloud vs. On-Premise** — Entscheidungskriterien, Architekturmodelle (IaaS/PaaS/SaaS), Auswahlkriterien für Cloud-Plattformen
2. **Vergleich von Data Analytics Services** — Snowflake, Amazon Redshift, Databricks, Data Lakes/Lakehouses
3. **Spatial Data Analytics & Google Earth Engine (GEE)** — Rasterdaten, GEE-Architektur, praktisches Programmieren mit der GEE JavaScript API

Teil 1: Cloud vs. On-Premise

Grundbegriffe

On-Premise bedeutet: Hardware steht beim Kunden, der Kunde betreibt und wartet alles selbst.

Cloud bedeutet: Der Anbieter hostet die Infrastruktur, der Nutzer zahlt per Abonnement oder nach Nutzung.

Tip: Woran erkenne ich die Frage? Aufgaben fragen nach Vor-/Nachteilen, nach Entscheidungskriterien oder nach der Klassifikation eines Services (IaaS/PaaS/SaaS).

Schnell-Entscheidung Cloud vs. On-Premise:

- Hohe Nutzungsschwankungen → Cloud (Elastizität)
- Geringe Nutzung, klar definiert → Cloud oft besserer ROI
- Sensible Daten, strenge Compliance → eher On-Premise
- Hohe Rechenanforderungen mit Latenz-Kritikalität → On-Premise oder Hybrid

Wenn X, dann Y:

- Wenn Kostenkontrolle wichtig + variable Last → Cloud (pay-per-use)
- Wenn Datenschutz / DSGVO / Finanz/Gesundheitswesen → On-Premise oder private Cloud
- Wenn schnelle Skalierung nötig → Cloud
- Wenn volle Kontrolle über Updates → On-Premise
- Wenn KI-Workloads mit Echtzeit-Anforderungen → Performance-Probleme möglich in Cloud

Aktuelle Marktlage (2025):

- AWS: 30 % Marktanteil

- Microsoft Azure: 23 %
- Google Cloud: 13 %

Trend: Nicht mehr “Cloud-first”, sondern “**Hybrid-first**” oder “**Workload-first**” — je nach Workload entscheiden, wo er am besten läuft (Kosten, Performance, Jurisdiktion).

Exam Trap: Häufiger Denkfehler: “Cloud ist immer günstiger als On-Premise.”

In der Praxis können Cloud-Kosten bis zu 50 % der Umsatzkosten ausmachen. Cloud ist **bei geringer oder variabler Nutzung** oft günstiger — bei konstant hoher Last kann On-Premise besser sein. KI-Workloads treiben Cloud-Kosten zusätzlich in die Höhe.

XaaS — Infrastructure, Platform, Software as a Service

Drei Schichten der Cloud-Abstraktion:

IaaS (Infrastructure as a Service):

- Anbieter stellt Rechenleistung, Speicher, Netzwerk bereit
- Kunde verwaltet OS, Middleware, Anwendungen
- Beispiel: Amazon EC2, Google Compute Engine

PaaS (Platform as a Service):

- Anbieter stellt Laufzeitumgebung und Entwicklungsplattform bereit
- Kunde entwickelt und deployt Anwendungen
- Beispiel: Google App Engine, **Google Earth Engine**

SaaS (Software as a Service):

- Vollständige Anwendung wird geliefert
- Kunde nutzt nur die Software
- Beispiel: Snowflake (als SaaS nutzbar), Microsoft 365

Tip: Klassifikation eines Services:

- Muss der Nutzer noch OS/Middleware konfigurieren? → IaaS
- Stellt der Dienst eine Entwicklungsplattform mit APIs bereit, aber keine fertige App? → PaaS
- Nutzt der Nutzer eine fertige Anwendung ohne eigene Infrastruktur? → SaaS
- GEE ist **Function- und Platform-as-a-Service (PaaS)** mit bereits aufbereiteten Daten (Data Warehouse)

Anforderungen an Cloud-Plattformen

Die sechs wichtigsten Auswahlkriterien (prüfungsrelevant!):

1. **Einfache Einrichtung und Betrieb** — dedizierte Systeme statt monolithischer Allzwecklösungen
2. **Elastizität** — autonomes Provisionieren/Deprovisionieren von Ressourcen je nach Last
3. **Unabhängige Skalierung** von Rechenleistung und Speicher (Shared Nothing)
4. **Gute SQL-Unterstützung**
5. **Verwaltung vielfältiger Daten** — strukturiert, semistrukturiert (JSON/XML), spaltenbasiert
6. **Backup-Möglichkeiten**

Definition Elastizität: “Das Ausmass, in dem sich ein System an Änderungen der Arbeitslast anpassen kann, indem es Ressourcen auf autonome Weise bereitstellt und entfernt, so dass zu jedem Zeitpunkt die verfügbaren Ressourcen so gut wie möglich dem aktuellen Bedarf entsprechen.”

Scale Up vs. Scale Out

Scale Up (vertikal): Leistungsfähigere Hardware einsetzen — mehr RAM, schnellere CPU in einem Knoten.

Scale Out (horizontal): Mehr Knoten hinzufügen — **Shared Nothing**-Architektur: Jeder Knoten hat eigenen Speicher, keine geteilte Ressource als Flaschenhals.

- Scale Up → begrenzt durch Hardwaremaximum, einfacher zu verwalten
- Scale Out → nahezu unbegrenzt skalierbar, Shared Nothing, besser für Big Data
- Cloud-native Systeme (Snowflake, Redshift) → bevorzugen Scale Out

Spaltenbasierte Datenspeicherung

Idee: Daten werden nicht zeilenweise sondern spaltenweise auf Disk gespeichert.

Vorteil für Analytics: Analytische Abfragen lesen typischerweise nur wenige Spalten über viele Zeilen. Bei spaltenbasiertem Speicher müssen nur die relevanten Spalten gelesen werden → weniger I/O.

Warum viele Spalten in Data Warehouses? Data Warehouses speichern stark denormalisierte Fakten- und Dimensionstabellen mit vielen Attributen (z.B. alle Eigenschaften eines Verkaufs in einer Zeile).

Data Warehouse vs. Data Lake vs. Data Lakehouse

Data Warehouse: Strukturierte, aufbereitete Daten; Schema-on-Write; optimiert für SQL-Abfragen; z.B. Snowflake, Redshift.

Data Lake: Rohdaten aller Typen (strukturiert, semistrukturiert, unstrukturiert); Schema-on-Read; kostengünstige Speicherung; z.B. S3 + Parquet.

Data Lakehouse (Trend): Kombination: Offenes Tabellenformat (Apache Iceberg) über einem Data Lake, mit Data-Warehouse-Abfragefähigkeiten.

Tip: Schlüsselbegriffe Open-Source Data Lake Formate:

- **Apache Iceberg** (Snowflake-bevorzugt): ACID-Transaktionen, Schema-Evolution, Time Travel; meiste GitHub-Stars; “hat gewonnen”
- **Delta Lake** (Databricks): Am schnellsten; fast nur Databricks-Contributors
- **Apache Hudi** (ex-Uber): Meiste Contributors insgesamt; gut für Streaming

Teil 2: Vergleich Cloud Analytics Services

Amazon Redshift

- Verfügbar seit 2013, Teil von AWS

- Vollständig verwalteter Service (Backup, Upgrades automatisch)
- **Spaltenorientierte Datenbank**
- Workload Manager (WLM): Priorisierung von Abfragen, verhindert Blockierung durch lange Queries
- Neu: “on-the-fly”-Elastizität (Serverless)
- Schwäche: Begrenzte Unterstützung semistrukturierter JSON-Daten

Snowflake

- 2012 gegründet, cloud-native (nicht von On-Premise abgeleitet)
- **Kernmerkmal: Trennung von Rechenleistung und Speicher** — kann unabhängig skaliert und bezahlt werden
- Spaltenorientierte SQL-Datenbank darunter (Apache Iceberg)
- SaaS: einfache Einrichtung ohne eigene Infrastruktur
- **Virtuelle Warehouses:** Jede Benutzergruppe/jeder Prozess läuft auf isolierter Hardware
- **Zero Copy Clone:** Schnelle Kopie ganzer Datenbanken ohne Datenduplizierung
- Data Warehouse + Data Lake = “Data Lakehouse”

Snowflake-Architektur — drei Schichten:

- **Storage Layer:** Zentraler, skalierbarer Objektspeicher (S3/Azure Blob/GCS)
- **Compute Layer:** Virtuelle Warehouses — mehrere unabhängig skalierbar
- **Cloud Services Layer:** Metadaten, Query Optimizer, Transaktions-Management

Dieses Modell ist weder rein “Shared Disk” noch rein “Shared Nothing” — es ist ein Hybrid: Shared Disk auf Storage-Ebene + Shared Nothing auf Compute-Ebene.

DuckDB (Erwähnung)

- Verarbeitet out-of-the-box bis 100 GB
- Via Parquet-Dateien und S3-Integration auch Terabyte-grosse Datensätze
- Als Ergänzung zu PostgreSQL: `pg_duckdb` bzw. `pg_lake`
- Relevant für lokale Analytics ohne Cloud

Teil 3: Spatial Data Analytics & Rasterdaten

GIS-Grundbegriffe: Zwei Datenmodelle

Vektor-Datenmodell:

- Repräsentiert Geometrien als Punkt, Linie, Polygon
- Beispielformat: GeoJSON
- Eignet sich für diskrete Objekte (Strassen, Gebäude, Grenzen)

Raster-Datenmodell:

- Repräsentiert Information als Gitter (Grid) aus Pixeln
- Jeder Pixel hat einen oder mehrere Zahlenwerte (“Bänder”)
- Beispielformate: GeoTIFF, HDF5, 2D-Matrix
- Eignet sich für kontinuierliche Phänomene (Temperatur, Höhe, Niederschlag)

Layer (Ebene): Sachdaten mit einer Geometrie — ein thematischer Datensatz.

Band: Ein Wertekanal eines Rasterbildes. Ein Satellitenbild hat typischerweise mehrere Bänder (rot, grün, blau, infrarot, ...).

Rasterdaten — Begriffe

- **Satellitenbild:** Bild aufgenommen von einem Satelliten
- **Luftbild:** Bild aufgenommen aus der Luft (Flugzeug, Drohne) nahezu senkrecht auf die Erde
- **Orthophoto:** Verzerrungsfreie, massstabsgetreue Abbildung der Erdoberfläche — abgeleitet aus Luft-/Satellitenbildern durch photogrammetrische Verfahren
- **Auflösung (Resolution):** Grösse eines Pixels in der Realität — z.B. 100 m bedeutet, ein Pixel repräsentiert $100\text{ m} \times 100\text{ m}$

Tip: Warum Rasterdaten besonders anspruchsvoll?

- Massive Datenmenge (Petabytes für globale, multi-temporale Bilder)
- Teuer in der Beschaffung grosser Gebiete
- Erfordern spezialisierte GIS-/Fernerkundungssoftware
- Georeferenzierung nötig (Pixel haben reale Weltkoordinaten)

Bildsensoren und Spektralbänder

Satellitensensoren erfassen mehr als das menschliche Auge sieht:

- **Sichtbares Licht (Visible):** Rot, Grün, Blau
- **Nahes Infrarot (NIR):** Vegetation stark reflektierend
- **Thermisches Infrarot:** Wärmestrahlung → Oberflächentemperatur
- **UV, Radar (SAR):** Weitere Anwendungen

Anwendungsbeispiel Urbane Hitzeinseln: Gebiete (v.a. städtisch) die wärmer sind als ihre Umgebung — erkennbar durch thermische Infrarotbänder in Satellitenbildern. Verwendung für Klimakarten, Stadtplanung, Gesundheitsstrategien.

Rasterdatenanalyse vs. klassische Bildverarbeitung

Gemeinsamkeiten: Beide arbeiten mit digitalen Bildern, nutzen Algorithmen zur Bildbearbeitung, haben ähnliche Vorverarbeitungsschritte.

Unterschiede:

Merkmal	Rasterdatenanalyse	Klassische Bildverarbeitung	— — —	Räumlicher
Bezug	Ja (georeferenziert)	Nein/optional		Datenmenge
	Gross (Bildreihen, Mosaik)	Klein (Einzelbilder)		Ziel
	Abbildung der Realität	Objekterkennung, Verarbeitung		Software
	GIS-Software + Bildverarbeitung	CV-Bibliotheken		

Teil 4: Google Earth Engine (GEE)

Was ist GEE?

Google Earth Engine ist eine **Cloud-Computing-Plattform** für das Auffinden, Zugreifen und Analysieren von Satellitendaten und raumbezogenen Beobachtungen.

Kernfähigkeiten:

- Zugriff auf **90 Petabytes** Satellitendaten über **45 Jahre** (NASA, ESA, etc.)
- Massive Rechenleistung direkt bei den Daten (keine Daten herunterladen nötig)
- Statistik- und ML-Funktionen in **JavaScript** und **Python**
- Geo-Visualisierung (Google Maps Mashups)
- Teilen als “GEE Apps”

Klassifikation: GEE ist **Function- und Platform-as-a-Service (PaaS)** mit bereits aufbereiteten Daten in der Cloud.

Wichtig: GEE ist nicht dasselbe wie Google Earth! Kommerzielle Nutzung kostet (Vertrag mit Google nötig). Für Lehre/Forschung kostenlos.

GEE Datenkatalog — Wichtige Datensätze

Datensatz — Auflösung — Zeitraum:

- Landoberflächentemperatur: ASTER AG100 → 100 m, 2000–2008
- Niederschlag: GSMaP → 11 km, alle 3h seit 2014
- Urbane Hitzeinseln: YCEO Surface Urban Heat Islands → 300 m, 2003–2018
- Höhenmodell: SRTM v4 → 90 m, 2000
- Bodenbedeckung: Copernicus CORINE Land Cover → 100 m, 2018
- Siedlungsflächen: World Settlement Footprint → 10 m, 2015
- Bevölkerungsdichte: GPWv411 → 1 km, 2000/2005/2010/2015/2020

GEE JavaScript API — Kernkonzepte

7.5.3.1. Datentypen in GEE

- `ee.Image` — Ein einzelnes Rasterbild (ein oder mehrere Bänder)
- `ee.ImageCollection` — Eine Sammlung von Bildern (z.B. Zeitreihe)
- `ee.FeatureCollection` — Eine Sammlung von Vektorobjekten (Polygone, Punkte)
- `ee.Filter` — Filter für Kollektionen

7.5.3.2. Grundprinzip: Lazy Evaluation

GEE führt Berechnungen nicht sofort lokal aus — Operationen werden als Beschreibung an Googles Server geschickt, die dann die eigentliche Berechnung durchführen. Deshalb: Grosse Datenmengen können verarbeitet werden ohne lokalen Download.

GEE Code-Rezepte für die Übungsaufgaben

Rezept 1: Vektordaten laden und filtern

1. `ee.FeatureCollection('DATASET_ID')` — Datensatz laden
2. `.filter(ee.Filter.eq('Attribut', 'Wert'))` — auf ein Land/Region filtern
3. `Map.addLayer(feature)` — zur Karte hinzufügen
4. `Map.centerObject(feature, zoom)` — Karte auf Objekt zentrieren

Beispiel — Land laden:

```
var countries = ee.FeatureCollection('USDOS/LSIB_SIMPLE/2017');  
var switzerland = countries.filter(ee.Filter.eq('country_na', 'Switzerland'));
```

```
Map.addLayer(switzerland);  
Map.centerObject(switzerland, 7);
```

Rezept 2: Rasterdaten laden, Band auswählen, visualisieren

1. `ee.ImageCollection('DATASET_ID')` — Bildserie laden
2. `.select('BANDNAME')` — gewünschtes Band auswählen
3. Zeitfilter anwenden (Jahr, Monat)
4. `.mean()` — Durchschnittsbild über Zeitraum
5. `.clip(region)` — auf Region zuschneiden
6. `Map.addLayer(image, vizParams, 'Name')` — visualisieren

Visualisierungsparameter:

```
var vizParams = {  
  min: minWert,  
  max: maxWert,  
  palette: ['blau', ..., 'rot'],  
  opacity: 0.7  
};
```

Beispiel — NO₂-Konzentration:

```
var no2ic = ee.ImageCollection('COPERNICUS/S5P/NRTI/L3_NO2')  
  .select('NO2_column_number_density');  
var no2_2020 = no2ic  
  .filter(ee.Filter.calendarRange(2020, 2020, 'year'))  
  .filter(ee.Filter.calendarRange(2, 5, 'month'))  
  .mean()  
  .multiply(1e6)  
  .clip(italy);  
Map.addLayer(no2_2020, {min: 0, max: 200,  
  palette: ['black', 'purple', 'green', 'red']}, 'NO2 2020');
```

Rezept 3: Kartenausschnitt und Zoom setzen

```
Map.setCenter(laengengrad, breitengrad, zoom);
```

- Koordinaten in **WGS84** (Längengrad zuerst, dann Breitengrad — Achtung: oft umgekehrt!)
- Zoomlevel: 1 = global, 13 = Stadtebene, 18+ = Gebäudeebene

Beispiel Zürich:

```
Map.setCenter(8.5417, 47.3769, 13);
```

Rezept 4: Einheiten skalieren

Daten liegen oft in ungewöhnlichen Einheiten vor (z.B. hundertstel Kelvin). Skalieren vor der Analyse:

```
var temperature = dataset.select('temperature').divide(100);  
// Ergebnis: Kelvin mit Dezimalstellen statt hundertstel Kelvin
```

Verfügbare Methoden:

- `.divide(zahl)` — durch Zahl teilen
- `.multiply(zahl)` — mit Zahl multiplizieren
- `.add(zahl)` — addieren
- `.subtract(zahl)` — subtrahieren

Kelvin zu Celsius: `.subtract(273.15)`

Rezept 5: Umgebungstemperatur berechnen (reduceNeighborhood)

Idee: Hitzeinseln sind Orte, die **wärmer als ihre Umgebung** sind. Schritt 1: Lokalen Durchschnitt (Umgebungstemperatur) berechnen.

```
var kernel = ee.Kernel.square({  
  radius: 2000,  
  units: 'meters'  
});  
var tempNeighborhood = temperature.reduceNeighborhood({  
  reducer: ee.Reducer.mean(),  
  kernel: kernel  
});
```

- `radius`: Radius des Suchfensters in Metern (z.B. 2 km = 2000 m)
- `ee.Reducer.mean()`: Mittelwert im Fenster
- Ergebnis: “verschwommenes” Bild — jeder Pixel enthält Umgebungsdurchschnitt

Rezept 6: Hitze-/Kälteinsel-Differenz berechnen

Hitze-/Kälteinsel = lokale Temperatur minus Umgebungstemperatur:

Differenz = Temperatur_{lokal} – Temperatur_{Umgebung}

```
var heatIsland = temperature.subtract(tempNeighborhood);
```

Positive Werte = wärmer als Umgebung (Hitzeinsel) Negative Werte = kälter als Umgebung (Kälteinsel)

Visualisierung: Wertebereich symmetrisch um 0 wählen, z.B. min: -5, max: 5

Rezept 7 (Zusatz): Einschränkung auf besiedeltes Gebiet

Idee: Besiedeltes Gebiet emittiert nachts mehr Licht. Nacht-Licht-Datensatz als Maske verwenden:

1. Nacht-Licht-Datensatz laden (z.B. NOAA/VIIRS Nighttime Lights)
2. Schwellenwert anwenden: Pixel unter Schwellenwert = unbewohnt → auf 0 setzen
3. Mit Hitzeinsel-Daten multiplizieren (Pixel in unbewohnten Gebieten werden 0)

```
// Nachtlicht laden
var nightlights = ee.ImageCollection('NOAA/VIIRS/DNB/MONTHLY_V1/VCMSLCFG')
  .select('avg_rad')
  .mean();

// Maske: Pixel mit genug Licht = 1, sonst = 0
var settledMask = nightlights.gt(schwellenwert);

// Ergebnis nur für besiedeltes Gebiet
var heatIslandUrban = heatIsland.updateMask(settledMask);
Map.addLayer(heatIslandUrban, vizParams, 'Urban Heat Islands');
```

Häufige Fehlerquellen in GEE

Exam Trap: Häufige Fehler und warum sie passieren:

1. **Koordinaten-Reihenfolge falsch:** `Map.setCenter` erwartet (Längengrad, Breitengrad) — also erst Ost-West, dann Nord-Süd. Wikipedia zeigt oft Breitengrad zuerst. Immer kontrollieren!
2. **Falscher Wertebereich in vizParams:** Wenn min/max nicht zum tatsächlichen Datenbereich passen, erscheint das Bild einfarbig (alles zu hoch → alles rot; alles zu niedrig → alles blau). Lösung: Im Inspector-Tab auf Pixel klicken und Rohwert prüfen.
3. **Einheiten ignoriert:** Viele Datensätze verwenden Skalierungsfaktoren (z.B. hundertstel Kelvin). Ohne `.divide(100)` sind alle Berechnungen falsch. Immer die Dataset-Dokumentation lesen!
4. **ImageCollection statt Image:** Viele Operationen funktionieren nur auf `ee.Image`, nicht auf `ee.ImageCollection`. Erst `.mean()`, `.first()` oder `.mosaic()` aufrufen um ein einzelnes Bild zu erhalten.
5. **Kein .clip()** → globaler Layer, der sehr langsam ist. Immer auf Region zuschneiden.
6. **Opacity vergessen:** Nach jedem Code-Run ist der Layer wieder 100% opak. Opacity im Code mitgeben: `Map.addLayer(img, params, 'Name', true, 0.7)` — letzter Parameter ist Opacity.

GEE: Stärken und Grenzen

Stärken:

- Einzigartige Datenmenge (90 PB, 45 Jahre, täglich aktualisiert)
- Rechenleistung direkt bei den Daten — kein Download nötig
- Einfache JavaScript/Python-API
- Kostenlos für Forschung und Lehre

Grenzen:

- Upload eigener Daten mengenmässig beschränkt
- Privacy-Bedenken (Google-Konto zwingend, Tracking)
- Keine Garantie auf dauerhafte Kostenfreiheit
- Eigentumsrechte: Code und eigene Daten bleiben beim Nutzer (anders als bei Google Maps!)

Zusammenfassung: Entscheidungsbaum**Welches Cloud-Modell passt?**

Brauche ich fertige Software?

- Ja → SaaS (z.B. Snowflake SaaS, Office 365)
- Nein: Entwickle ich eigene Apps?
 - Ja, brauche Plattform/APIs → PaaS (z.B. GEE, App Engine)
 - Nein, brauche nur Infrastruktur → IaaS (z.B. EC2, GCE)

Welche Analytics-Plattform?

Sehr variable Last + einfache Einrichtung?

- Snowflake (beste Elastizität, SaaS)

Bereits in AWS-Ökosystem?

- Amazon Redshift

Grosse Datenmengen + Spark/ML-Workflows?

- Azure Databricks

Satellitendaten / Geo-Analyse?

- Google Earth Engine

Lokale Analyse bis 100 GB?

- DuckDB

GEE-Analyse-Pipeline (für Hitzeinseln):

1. Datensatz suchen (Data Catalog)
2. ImageCollection laden → Band auswählen
3. Zeitfilter → `.mean()` → `.clip(region)`
4. Skalieren (`.divide()` / `.multiply()`)
5. Visualisieren (min, max, palette, opacity)
6. Umgebungstemperatur: `.reduceNeighborhood(mean, kernel)`
7. Differenz: `temperature.subtract(neighborhood)`
8. (Optional) Maske mit Nachtlcht für urbane Gebiete

Week 7: [Topic Title]

Woche 7: Data Integration — ETL/ELT

Überblick und Lernziele

Diese Woche behandelt die **Datenintegration** als zentrales Handwerk der Data Analytics. Der Student soll verstehen, was Datenintegration bedeutet, welche Konflikte dabei auftreten, wie ETL- und ELT-Prozesse funktionieren, und wie Tools wie OpenRefine, Apache Hop und dbt eingesetzt werden.

1. Informations- und Datenintegration: Grundbegriffe

1.1 Definition und Motivation

Kernidee: Datenintegration bedeutet, Daten aus verschiedenen, heterogenen Quellen korrekt, vollständig und effizient zu einer einheitlichen Informationsmenge zusammenzuführen.

Tip: Woran erkenne ich dieses Konzept in einer Aufgabe?

- Mehrere Datenquellen mit unterschiedlichen Formaten/Strukturen sollen zusammengeführt werden
- Frage nach “Vorteilen/Nachteilen” von Integrationsansätzen
- Frage nach typischen Problemen bei der Datenintegration

Die **Zeitrealität der Praxis:** Data Scientists verbringen mindestens 60–80% ihrer Zeit mit Daten finden, bereinigen und integrieren — nicht mit Modellierung. Das zeigt die zentrale Bedeutung dieses Themas.

Bestandteile der Datenintegration (alle prüfungsrelevant):

Funktionen der Datenintegration — Vollständige Liste:

1. **Datenimport / Datenkonversion** — Einlesen aus verschiedenen Formaten (CSV, JSON, XML, XLSX)
2. **Datenvalidierung** — Prüfen ob Daten gültig und vollständig sind
3. **Datenbereinigung** — Entfernen von Duplikaten, ungültigen oder fehlenden Werten
4. **Datentransformation** — Schema-Abbildung, Typkonvertierung, Spalten aufteilen/zusammenführen
5. **Datenfusion (Conflation/Conciliation)** — Zusammenführen widersprüchlicher Darstellungen derselben Entität
6. **Datenaggregation** — Zusammenfassen aus mehreren Quellen (GROUP BY, SUM, COUNT)
7. **Datenfilterung** — Auswahl relevanter Datensätze (WHERE)
8. **Datenberechnung** — Hinzufügen berechneter Spalten

9. **Datenanreicherung** — Ergänzung durch externe Quellen (z.B. Geocoding, Web Scraping)
10. **Datenanonymisierung** — Schutz sensibler Daten
11. **Datengenerierung** — Synthetische Daten erzeugen

1.2 Datenintegration: DWH vs. Machine Learning

Wenn die Aufgabe fragt: “Unterschied DWH vs. ML bei der Datenintegration?”

Aspekt	Data Warehouse	Machine Learning	Ziel
Exploration, Modelltraining	Fixiert, strukturiert	Flexibel, agil, Feature-Engineering	Reporting, Historisierung
Aktualität	Periodisch (Batch)	Batch + Streaming/Near-Realtime	
Datenqualität	Konsistenz, historische Genauigkeit	Genauigkeit, Aktualität, Bias, Feature-Qualität	
Quellen	Primär strukturiert, intern	Divers, oft unstrukturiert, extern/intern	
Technologien	Relationale DBs, ETL-Tools, SQL	Python/Pandas, Pipeline-Tools, SQL	

2. Vorgehensweisen und Konflikttypen bei der Datenintegration

2.1 Top-Down vs. Bottom-Up

Top-Down: Das Zielschema ist vorgegeben (z.B. durch Standards). Man sucht Korrespondenzen zu den Quell-Schemata.

Bottom-Up: Ausgangspunkt sind die existierenden Quell-Schemata. Daraus wird ein integriertes Zielschema konstruiert, das möglichst den vollständigen Informationsgehalt aller Quellen enthält.

2.2 Kriterien für Integrationsmethoden

Vier Qualitätskriterien nach Batini et al. (1986):

1. **Vollständigkeit** — Das integrierte Schema muss alle Konzepte aller lokalen Schemata beinhalten; keine Information darf verloren gehen.
2. **Minimalität** — Reale Konzepte, die in mehreren lokalen Schemata vorkommen, dürfen nur **einmal** im integrierten Schema repräsentiert sein (keine Redundanz).
3. **Verständlichkeit** — Das integrierte Schema muss leicht verständlich und nachvollziehbar sein.
4. **Korrektheit** — Alle Informationen im integrierten Schema müssen in mindestens einem lokalen Schema semantisch äquivalent vorhanden sein; neu hinzugefügte Beziehungen dürfen nicht im Widerspruch zu bestehenden Schemata stehen.

2.3 Klassifikation von Integrationskonflikten

Tip: Woran erkenne ich welche Konfliktart vorliegt?

- **Semantischer Konflikt** → Unterschiedliche Mengen von Real-Welt-Objekten sind repräsentiert (z.B. “öffentliche Sitzbank” bedeutet in Quelle A und B verschiedene Dinge)
- **Beschreibungskonflikt** → Gleiche Objekte, aber unterschiedliche Attribute/Namen; **Homonyme** (gleicher Name, verschiedene Bedeutung) oder **Synonyme** (verschiedene Namen, gleiche Bedeutung); unterschiedliche Masseinheiten
- **Heterogenitätskonflikt** → Unterschiedliche Schemata, Modellierungskonstrukte oder Ausdruckskraft → Auflösung durch Schema-Transformation
- **Struktureller Konflikt** → Unterschiedliche Modellierung (z.B. Vererbung vs. flache Tabelle), fehlende Attribute, unterschiedliche Default-Werte
- **Datenkonflikt** → Falsche/veraltete Daten, unterschiedlich codierte Aufzählwerte

2.4 Heterogenitätsarten

Semantische Heterogenität: Begriffe und Objektkonzepte meinen in verschiedenen Systemen verschiedenes.

Modellierungsheterogenität: Unterschiedliche Datenstrukturen, Datentypen, Regeln:

- Texttyp vs. spezifischer Datentyp
- Zeilenorientiert vs. geschachtelt (JSON)
- Unterschiedliche Attributnamen (Homonyme)

Technisch-strukturelle Heterogenität: Unterschiedliche Zugriffsmethoden, Datenformate (CSV, XML, JSON, binär), Kodierung (Little/Big Endian, EOL, UTF-8, String Escaping).

3. Data Warehouse: Architektur und Wartung

3.1 Pro und Contra Data Warehouses

Wenn gefragt: “Warum DWH?” oder “Warum nicht DWH?”

Pro DWH:

- Abfragen stören die Quellsysteme nicht
- Anspruchsvolle Berechnungen (Data Mining, Bereinigung) möglich
- Zentraler, konsistenter Datenbestand

Contra DWH:

- Aufwändiges Design des physischen und logischen Datenbankschemas
- Daten sind oft nicht aktuell (träge oder Offline-Aktualisierung)
- Datensynchronisation bei Änderungen in den Quellen ist ein Problem

Alternative: Virtuelle Datenintegration — Daten bleiben in den Quellen, werden erst bei Abfrage abgerufen (kein Kopieren, aber höhere Abfragelatenz).

3.2 DWH Wartungsansätze

Aktualisieren-Kopieren (Full Refresh): Gesamtaustausch aller Dateninhalte. Einfach, aber Aufwand wächst mit Datenvolumen.

Inkrementelle Kopie: Nur geänderte Daten werden aktualisiert. Effizienter, aber: Es müssen Vorkehrungen für **Update-Anomalien** getroffen werden, die durch asynchrone Verarbeitung entstehen.

Exam Trap: Update-Anomalien: Wenn Quellsystem und DWH nicht synchron sind, können Änderungen in der Quelle im DWH nicht oder falsch ankommen. Das DWH zeigt dann veraltete oder inkonsistente Daten — ein klassischer Fehler bei inkrementellen Kopien ohne Transaktionsschutz.

4. ETL — Extract, Transform, Load

4.1 Grundidee und Prozessschritte

Kernidee: ETL ist der Standardprozess für die physische Zusammenführung von Daten aus heterogenen Quellen in ein Data Warehouse. Die Transformation geschieht **ausserhalb** des Ziel-DBMS, in einem separaten Tool oder Skript.

ETL-Prozess — Allgemeines Rezept:

1. **Extract (Extrahieren):**

- Selektion eines Ausschnitts der Daten aus den Quellsystemen
- Bereitstellung in einer Staging Area (Zwischenspeicher)
- Quellen können sein: relationale DBs, CSV/Excel, APIs, JSON/XML-Dateien

2. **Transform (Transformieren):**

- Anpassung der Daten an Zielschema und Qualitätsanforderungen
- Durchgeführt im ETL-Tool (z.B. mit JavaScript bei Kettle, Java bei Talend)
- Alle Transformationsfunktionen werden hier angewendet (siehe Abschnitt 4.2)

3. **Load (Laden):**

- Physisches Einbringen der transformierten Daten in das Data Warehouse
- Inklusive eventuell notwendiger Aggregationen
- Bei Rohdaten in Data Lakehouses: "Schema-on-Read" statt Schema-on-Write

Merke: ETL kann in jeder Programmiersprache selbst implementiert werden, aber Unternehmen nutzen meist spezialisierte ETL-Tools.

4.2 Transformationsfunktionen — Vollständige Übersicht

Grundlegende Transformationen (alle mit SQL lösbar):

- **String-Operationen:** Trimmen, Auffüllen, Gross-/Kleinschreibung, Tausendertrennzeichen löschen
- **Berechnungen:** Arithmetische/logische Operationen → neue Spalten ableiten (z.B. Alter aus Geburtsdatum)

- **Typumwandlung:** String → Integer, String → Datum, etc.
- **Felder hinzufügen:** Konstanten, eindeutige Bezeichner, Zeitstempel
- **Feldaufteilung (Split):** Ein Feld mit mehreren Werten → mehrere Spalten/Zeilen
- **Filtern und Löschen:** WHERE-Bedingungen; nicht benötigte Zeilen entfernen
- **Umbenennen und Löschen von Feldern:** Schema anpassen
- **Gruppieren und Aggregieren:** GROUP BY + SUM, AVG, COUNT, MAX, MIN
- **Kodierung/Dekodierung:** Kategorische → numerische Werte (für ML)
- **Deduplizierung:** Doppelte Datensätze identifizieren und entfernen
- **Datenmaskierung/Anonymisierung:** Sensible Daten schützen

Anspruchsvollere Transformationen:

- **Sortieren (SORT):** Zeilen nach einem oder mehreren Feldern ordnen
- **Verbinden (JOIN):** Zeilen aus zwei oder mehr Tabellen über gemeinsame Schlüssel verbinden
- **Pivotieren (PIVOT/UNPIVOT):** Zeilen → Spalten oder Spalten → Zeilen umstrukturieren
- **Verflachung:** Hierarchische Strukturen (JSON/XML) → flaches relationales Format
- **Zusammenführen/Merge:** Datensätze kombinieren ohne vollständige Schlüssel-Übereinstimmung
- **Datenanreicherung (Lookup):** Daten aus Nachschlagetabellen oder externen Quellen ergänzen

Für ML-Datenmodelle zusätzlich:

- **Fehlerbehandlung und Bereinigung:** Ausreisser identifizieren und korrigieren/entfernen
- **Normalisierung/Standardisierung:** Werte auf gemeinsame Skala bringen
- **Imputation:** Fehlende Werte durch Mittelwert/Median/Modus ersetzen
- **Fehlende Daten markieren:** Explizit als fehlend kennzeichnen statt löschen

4.3 SQL-Rezepte für häufige ETL-Transformationen

Rezept: Tabelle kopieren

```
CREATE TABLE ziel_tabelle AS  
SELECT * FROM quell_tabelle;
```

Rezept: Datentyp einer Spalte ändern (z.B. TEXT zu INT)

```
ALTER TABLE meine_tabelle  
  ALTER COLUMN spalte1 TYPE INT  
  USING CAST(NULLIF(spalte1, '') AS INT);
```

Achtung: `NULLIF(spalte1, '')` wandelt leere Strings zuerst in NULL um, damit CAST nicht fehlschlägt.

Rezept: Spalte aufteilen (Split)

```
ALTER TABLE meine_tabelle
  ADD COLUMN vorname VARCHAR(255),
  ADD COLUMN nachname VARCHAR(255);

UPDATE meine_tabelle
SET vorname = split_part(vollname, ' ', 1),
    nachname = split_part(vollname, ' ', 2);
```

split_part(string, trennzeichen, position) — Position 1 = erstes Element.

Rezept: CSV in PostgreSQL laden (COPY)

```
COPY personen(Kunden_ID, Nachname, Vorname, Geburtsdatum)
FROM 'C:/daten/adressliste.csv'
DELIMITER ',' CSV HEADER;
```

Voraussetzung: Das Zielschema muss **exakt** zur CSV-Struktur passen.

Exam Trap: Häufiger Fehler beim CSV-Import mit COPY: Das Datenbankschema muss **vor** dem Import angelegt werden und genau zu den Spalten der CSV-Datei passen — Reihenfolge, Datentyp und Anzahl der Spalten. Wenn das Schema nicht passt, schlägt COPY mit einem Fehler fehl. Lösung: Tools wie DBeaver, CSVKit oder Python/Pandas nutzen, die das Schema automatisch ableiten können.

5. ELT — Extract, Load, Transform

5.1 Unterschied ETL vs. ELT

Kernidee: Bei ELT werden die Rohdaten zuerst in die Datenbank geladen (Staging DB), und die Transformation erfolgt **innerhalb** des DBMS mit SQL — kein separates Transformations-Tool nötig.

Wenn Aufgabe fragt: “ETL oder ELT — was wählen und warum?”

Kriterium	ETL	ELT	— — —	Wo wird transformiert?	Im ETL-Tool (ausserhalb DBMS)	Im DBMS (mit SQL)	Sprache	JavaScript, Java, Python	SQL	Architektur
Komplexer (separates Tool)	Einfacher (nur DBMS)	Speicherplatz	Staging Area + Ziel	On-demand im DBMS	Delay	Kleiner Delay (Transform vor Load)	Transform verzögert bis Abfrage	Wann bevorzugt?	Legacy DWH, komplexe Transformationen	Cloud-DBMS, grosse Datenmengen, SQL-affine Teams

5.2 Reverse ETL

Reverse ETL ist der umgekehrte Prozess: Daten werden aus einem zentralen Data Warehouse zurück in operative Systeme (CRM, Marketing-Tools etc.) kopiert. Typisch wenn analytisch aufbereitete Daten operativ genutzt werden sollen.

6. Tools für die Datenintegration

6.1 Übersicht der Tool-Kategorien

Welches Tool für welche Aufgabe?

- **Python/Pandas, Java, R:** Maximale Flexibilität, aber hoher Implementierungsaufwand
- **SQL / dbt:** Das “T” in ETL/ELT — Transformation mit SQL, Versionierung, Wiederverwendung
- **OpenRefine:** Interaktive Datenbereinigung und -exploration, No-Coding, ideal für CSV/Excel/JSON
- **Apache Hop:** Vollständiges ETL-Tool (Fork von Kettle), GUI-basiert, Java, Open Source
- **KNIME:** ETL + Machine Learning, visuelle Pipelines
- **AWS Glue / Azure Data Factory:** Cloud-native ETL-Plattformen
- **Apache AirFlow:** Schwerpunkt DataOps und Pipeline-Orchestrierung
- **FME:** Kommerziell, spezialisiert auf Geodaten
- **GDAL/OGR:** Open Source für Geodaten

6.2 OpenRefine — Detaillierte Beschreibung

Kernidee: OpenRefine ist ein Desktop-Tool für interaktives Sichten, Bereinigen, Integrieren und Anreichern von Daten — ohne Programmierung, mit einer tabellenkalkulationsähnlichen GUI.

OpenRefine — Allgemeines Lösungsrezept für Datenbereinigung:

1. **Daten importieren:** CSV, XLS, JSON, XML oder Datenbank → Projekt erstellen
2. **Daten sichten (Facets):** Text-Facets, Zahlen-Facets, Timeline-Facets einsetzen um Verteilungen und Ausreisser zu erkennen
3. **Inkonsistente Werte identifizieren:** Cluster-Funktion (“Cluster and Edit”) nutzt Ähnlichkeitsalgorithmen um Varianten desselben Wertes zu finden (z.B. “Zürich”, “Zuerich”, “zürich”)
4. **Werte bereinigen:** Direkt in der Zelle oder über Transformationen (GREL-Ausdrücke: `value.trim()`, `value.toLowerCase()`, etc.)
5. **Spalten transformieren:** Umbenennen, aufteilen, zusammenführen, Typ zuweisen
6. **Filtern:** Text-Filter oder Facets zur Selektion von Teilmengen
7. **Datenanreicherung:** Reconciliation gegen Wikidata oder andere externe Quellen
8. **Export:** CSV, Excel, JSON, etc. exportieren

Wichtige Eigenschaften:

- Verändert **niemals** die Quelldaten (ausser man überschreibt explizit)
- Spalten haben **keinen** Datentyp — ausser man weist einen explizit zu
- Memory-/RAM-basiert → limitiert durch verfügbaren RAM
- Alle Operationen werden als **History** gespeichert und können als JSON-Skript exportiert/importiert werden (Reproduzierbarkeit!)

Tip: Woran erkenne ich, dass OpenRefine die richtige Wahl ist?

- Aufgabe: “Bereinige diese CSV/Excel-Datei interaktiv”
- Aufgabe: “Finde und entferne Duplikate oder Tippfehler in den Werten”
- Aufgabe: “Standardisiere Schreibweisen (z.B. Ländernamen, Kategorien)”
- Aufgabe: “Reichere Daten mit externen Quellen an”
- Kein Coding gewünscht, einmalige oder explorative Bereinigung

Exam Trap: Häufige Denkfehler bei OpenRefine:

- OpenRefine hat **keinen** Undo über Sessions hinaus — aber die History-Funktion speichert alle Operationen als JSON. Diese JSON-Operationsliste kann auf neuen Daten wiederholt angewendet werden.
- Spalten haben **keinen automatischen Datentyp** — wenn man mit Zahlen rechnen will, muss man den Typ explizit zuweisen.
- OpenRefine ist RAM-basiert: bei sehr grossen Dateien (> wenige GB) gibt es Performanceprobleme.

6.3 Apache Hop — Ausgewählte Transforms

Kernidee: Apache Hop ist ein GUI-basiertes ETL-Tool für visuelle Pipeline-Erstellung. Daten fließen als Zeilen-Streams durch verkettete Transformations-Schritte.

Wichtige Transforms in Apache Hop:

- **Filter Rows:** Leitet Zeilen basierend auf einer Bedingung zu “true”- oder “false”-Ausgabe
- **Select Values:** Auswahl, Umbenennung, Konvertierung von Feldern (Regex möglich)
- **Database Lookup:** Abfrage einer DB-Tabelle und Hinzufügen neuer Felder (= JOIN-ähnlich)
- **String Operations:** Kleinschreibung, Zeichenketten-Ersetzen
- **Value Mapper:** Ersetzt Werte basierend auf einer definierten Mapping-Tabelle
- **CSV Input / Text Output:** Einlesen und Schreiben von CSV-Dateien
- **Parquet Input / Output:** Einlesen und Schreiben von Parquet-Dateien

6.4 dbt — data build tool

Kernidee: dbt ist ein Command-Line-Tool (und Cloud-Service) das **nur** das “T” in ETL/ELT übernimmt: Datentransformation durch SQL-Abfragen, mit Unterstützung für Testen, Versionierung und Wiederverwendung.

Kernmerkmale:

- Transformation rein durch SQL (+ Jinja-Templates für Parametrisierung)
- YAML für Konfiguration und Metadaten
- Unterstützt “SQL-Query-Dependencies” (welche Query hängt von welcher ab)
- Versionierung mit Git (im Gegensatz zu GUI-Tools)
- Als Desktop-Tool und als “dbt cloud” verfügbar
- Eingebettet in grössere Plattformen wie AirByte

7. Klassifikation von Datenfehlern

Datenfehler können auf verschiedenen Ebenen auftreten (nach Leser und Naumann 2006):

- **Syntaktische Fehler:** Falsche Datentypen, falsche Formatierung (z.B. Datum als "32.13.2024")
- **Semantische Fehler:** Wert ist syntaktisch korrekt, aber bedeutungslos (z.B. Alter = -5)
- **Strukturelle Fehler:** Fehlende Pflichtfelder, falsche Relationen zwischen Tabellen
- **Duplikate:** Dieselbe Entität mehrfach gespeichert, eventuell mit leichten Abweichungen

8. Datenfluss: Schematische Übersicht

Typischer Datenfluss in ein Data Warehouse:

```

Quellsystem 1 ┘
Quellsystem 2 ┘ → [Extract] → [Staging Area] → [Transform] → [Load] → Data
Warehouse
Quellsystem 3 ┘
  
```

Bei Data Lakehouse:

```

Quellsystem → [Extract] → [Load RAW] → Data Lake (Rohdaten)
                                   ↓ (Schema-on-Read bei Abfrage)
                                   Analyseschicht
  
```

Reverse ETL (umgekehrt):

```

Data Warehouse → [Extract] → [Transform] → [Load] → Operative Systeme (CRM,
etc.)
  
```

9. Zusammenfassung: Entscheidungsmatrix

Schnell-Entscheidungen für Aufgaben und Prüfung:

- Wenn **mehrere heterogene Quellen** → Datenintegration nötig, Konflikttypen prüfen
- Wenn **interaktive Bereinigung**, kein Code → **OpenRefine**
- Wenn **automatisierte Pipelines**, SQL-Transformation → **ELT mit dbt oder SQL**
- Wenn **komplexe Pipelines mit GUI** → **Apache Hop** (Open Source) oder Talend/SSIS (kommerziell)
- Wenn **Cloud-native** → AWS Glue, Azure Data Factory
- Wenn **Transformation ausserhalb DB** (mit Java/JS/Python) → **ETL**
- Wenn **Transformation innerhalb DB** (mit SQL) → **ELT**
- Wenn **Daten aus DWH zurück in operative Systeme** → **Reverse ETL**
- Wenn Frage nach Qualitätskriterien: Vollständigkeit, Minimalität, Verständlichkeit, Korrektheit (Batini et al.)
- Wenn Frage nach Konflikten: Semantisch, Beschreibung, Heterogenität, Strukturell, Daten

Week 8: [Topic Title]

Woche 8: Data Integration — Datenaufbereitung, Duplikaterkennung & Apache Hop

Überblick der Themen

Diese Woche behandelt die **praktische Seite der Datenintegration**: Wie werden Daten bereinigt, Duplikate erkannt, fehlende Werte ergänzt und Daten angereichert? Als praktisches Tool steht **Apache Hop** im Mittelpunkt.

Teil 1: Tools für die Datenintegration

Toolkategorien und Auswahlkriterien

Tip: Woran erkenne ich welche Toolkategorie gemeint ist?

- Aufgabe fragt nach **Steuerung von Abläufen/Prozessen** → Datenflussmanagement (Airflow, NiFi)
- Aufgabe fragt nach **Transformation/Bereinigung** → Dataprocessing (OpenRefine, dbt, KNIME)
- Aufgabe fragt nach **beidem** → Hybrid (Apache Hop, FME, Informatica PowerCenter)
- Aufgabe fragt nach **Geodaten** → FME, Apache Hop, KNIME, Looker Studio

Wenn Tool X, dann Kategorie Y:

- Apache Airflow → Datenflussmanagement, Python, OSS, Batch + teilweise Echtzeit
- Apache NiFi → Datenflussmanagement, Java, OSS, Echtzeit + Batch, Web-GUI
- KNIME → Dataprocessing/Hybrid, Java, OSS+komm., Desktop+Web, Batch
- OpenRefine → Dataprocessing, Java, OSS, Desktop, Batch, No-Code, interaktiv
- dbt → Transformation in Data Warehouses, Python, OSS+komm., CI/CD-Integration
- IBM InfoSphere DataStage → Dataprocessing, kommerziell, Echtzeit+Batch, skalierbar
- Microsoft SSIS → Dataprocessing, .NET, kommerziell, Windows, Batch
- Apache Hop → Hybrid, Java, OSS, Desktop+Web+Batch, grafische ETL-Entwicklung
- FME by Safe Software → Hybrid, C++/Python, kommerziell, Geodaten-Spezialist
- Informatica PowerCenter → Hybrid, Java, kommerziell, Enterprise-Umgebungen
- Looker Studio → Transformation+Visualisierung, kommerziell, Cloud-only, Google

Auswahl-Kriterien in der Praxis (Reihenfolge):

1. **Kategorie** bestimmen: Datenflussmanagement, Dataprocessing oder Hybrid?
2. **Ökosystem** prüfen: Microsoft? IBM? Google? Open Source?
3. **Know-how** im Team vorhanden?
4. **Spezifische Anforderungen**: Geodaten? Echtzeit? Skalierung?

Exam Trap: Kommerzieller Support existiert auch für Open-Source-Produkte (z.B. KNIME). OSS $\neq$ kein Support.

Teil 2: Datenaufbereitung und Datenreinigung

Begriffsabgrenzung

Datenintegration (Data Integration)

- Verschiedene Daten von vielen Quellen zu einem gemeinsamen Schema zusammenführen

Datenaufbereitung (Data Wrangling) — Teil der Datenintegration

- Reinigung, Transformation und Anreicherung von Rohdaten vor der Analyse

Datenreinigung (Data Cleaning / Cleansing / Scrubbing) — Teil der Datenaufbereitung

- Fehlerhafte, inkonsistente oder unvollständige Daten erkennen und korrigieren

Tip: Unterschied DW/DL vs. ML bei der Datenaufbereitung:

Merkmal	Data Warehouse / Data Lake	Machine Learning
Ziel	Bereitstellung für Analysen	Vorbereitung für Modelle
Richtung	Top-down (Zielschema vorgegeben)	Bottom-up (gemeinsamer Nenner)
Schwerpunkt	Aggregation, Schema-Optimierung	Feature Engineering, Dimensionsreduktion
Prozess	Einmalig, regelmässige Updates	Iterativ, kontinuierlich

Datenfehler-Klassifikation

Einfache vs. schwierige Fehler:

- **Einfach:** Durch Betrachtung eines einzelnen Tupels erkennbar (z.B. falsches Datumsformat)
- **Schwierig:** Erst durch Vergleich mehrerer Tupel erkennbar (z.B. Duplikate)

Systematische vs. unsystematische Fehler:

- **Systematisch:** Regelmässig, daher leicht (maschinell) korrigierbar
- **Unsystematisch:** Unregelmässig, oft schwierig zu korrigieren

Data-Cleaning-Techniken

1. **Konvertierung und Normalisierung** — unterschiedliche Formate vereinheitlichen (Datum, Währung)
2. **Domänenunabhängige Bereinigung** — Schlüssel- und Attributvergleich, Deduplizierung
3. **Domänenspezifische Bereinigung** — z.B. mit Thesauri (Synonymwörterbücher)
4. **Regelbasierte Bereinigung** — Regeln definieren welche Daten geprüft/gefiltert/korrigiert werden

Regelbasierte Bereinigung — zwei Entscheidungsfragen:

- Auf welche Daten zielt die Regel? → Quelldaten oder Zieldaten?
- Welche Aktion? → Prüfung (Validation) / Filterung (Removal) / Korrektur (Correction)

Teil 3: Duplikaterkennung und Zeichenkettenvergleich

Was sind Duplikate?

Duplikate = verschiedene Tupel in einer Relation, die dasselbe Realweltobjekt repräsentieren.

Synonyme: Dublettenerkennung, Record Linkage, Object Identification, Named Entity Resolution, Reference Reconciliation.

Zwei zentrale Herausforderungen:

- **Genauigkeit (Accuracy):** Echte Duplikate finden, keine False Positives
- **Skalierbarkeit:** Effizient bei grossen Datenmengen

Exakte Duplikaterkennung mit SQL

Rezept: Exakte Duplikate finden und löschen

1. Mit `ROW_NUMBER() OVER (PARTITION BY ...)` jedem Tupel eine Zeilennummer zuweisen — gleiche Werte in den Partition-Spalten erhalten Nummern 1, 2, 3, ...
2. Tupel mit `rownum > 1` sind Duplikate (der erste Treffer bleibt erhalten)
3. Duplikate löschen mit `DELETE ... USING ...`

Beispiel (PostgreSQL):

```
WITH duplicates AS (  
  SELECT  
    id,  
    ROW_NUMBER() OVER(  
      PARTITION BY spalte1, spalte2, spalte3  
      ORDER BY id DESC  
    ) AS rownum  
  FROM tabelle  
)  
DELETE FROM tabelle USING duplicates  
WHERE tabelle.id = duplicates.id  
  AND duplicates.rownum > 1  
RETURNING *;
```

- `PARTITION BY` definiert, welche Spalten für die Gleichheit verglichen werden
- `ORDER BY id DESC` bestimmt, welcher Datensatz behalten wird (höchste ID bleibt)
- `RETURNING *` zeigt die gelöschten Zeilen

Exam Trap: Exaktes Matching versagt bei Tippfehlern! "Müller" und "Muller" werden nicht als Duplikate erkannt. → Lösung: Ähnlichkeitsmasse verwenden.

Ähnlichkeitsmasse (Similarity Measures)

9.4.3.1. Kategorie 1: Edit-/Transpositionsbasierte Masse

Levenshtein-Distanz (Edit-Distanz)

Die Kernidee: Wie viele elementare Operationen brauche ich, um String x in String y umzuwandeln?

Erlaubte Operationen (je Kosten = 1):

- Löschen eines Zeichens
- Einfügen eines Zeichens
- Ersetzen eines Zeichens

$d(\text{Muster, Master}) = 1$ (ein Ersetzen: $u \rightarrow a$)

$d(\text{Haus, Maus}) = 1$ (ein Ersetzen: $H \rightarrow M$)

$d(\text{kitten, sitting}) = 3$

Interpretation: Kleinere Distanz = höhere Ähnlichkeit.

Tip: Wann Edit-Distanz?

- Kurze Strings mit Tippfehlern (Vor-/Nachnamen, Produktcodes)
- Wenn Zeichenreihenfolge wichtig ist
- PostgreSQL: `levenshtein()` Funktion verfügbar (fuzzystrmatch-Extension)

Smith-Waterman

- Sucht die **ähnlichsten Teilstrings** (lokale Ausrichtung statt globaler)
- Gut für: "Prof. John R. Smith, Univ of Wisconsin" vs. "John R. Smith, Professor" → findet "John R. Smith" als gemeinsamen Kern

Jaro / Jaro-Winkler

- Speziell für kurze Strings (Vor- und Nachnamen)
- Berücksichtigt Transpositionen und gemeinsame Zeichen

9.4.3.2. Kategorie 2: Token-/Mengenbasierte Masse

Grundidee: String nicht als Zeichenfolge, sondern als **Menge von Tokens** behandeln.

Tokenisierung:

- Nach Wörtern: "david smith" → {"david", "smith"}
- Nach n-Grammen (Teilstrings Länge n, z.B. n=3): "david" → {##d, #da, dav, avi, vid, id#, d##}
- Optional: Stoppwörter entfernen (das, und, von), Stemming anwenden

Jaccard-Koeffizient — sehr verbreitet!

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Wobei A und B die Token-Mengen der beiden Strings sind.

- $J = 1$: identisch
- $J = 0$: kein gemeinsames Token
- $J = 0.5$: Hälfte der Tokens gemeinsam

Beispiel:

- $A = \{\text{david, smith}\}$, $B = \{\text{dave, smith}\}$
- $A \cap B = \{\text{smith}\}$, $A \cup B = \{\text{david, smith, dave}\}$
- $J = \frac{1}{3} \approx 0.33$

TF/IDF (Term Frequency / Inverse Document Frequency)

- Mass aus dem Information Retrieval
- Häufige Tokens werden weniger gewichtet als seltene, charakteristische Tokens

9.4.3.3. Hybride und phonetische Masse

- **Generalized Jaccard, Soft TF/IDF, Monge-Elkan:** Kombinieren mehrere Ansätze
- **Soundex:** Phonetisches Mass — klingt es gleich? ("Schmidt" $\approx$ "Schmid")

Welches Ähnlichkeitsmass wählen?

- Kurze Strings, Tippfehler in einzelnen Zeichen → **Levenshtein / Edit-Distanz**
- Kurze Namen, Transpositionen → **Jaro-Winkler**
- Lange Strings mit gemeinsamen Teilbereichen → **Smith-Waterman**
- Mehrwörtige Strings, Wortmengen → **Jaccard**
- Wenn häufige Wörter weniger zählen sollen → **TF/IDF**
- Klang wichtiger als Schreibweise → **Soundex**

Teil 4: Datenabgleich (Data Matching)

Abgrenzung zu String Matching

Merkmal	String Matching	Data Matching	Einheit
Strings Tupel / Datensätze			
Beispiel "D. Smith" vs. "David Smith"	(D. Smith, 245-4367, Madison) vs. (David Smith, 608-245-4367, Madison WI)	Technik Ähnlichkeitsmasse auf Strings Kombination von Feldern, domänenspezifisches Wissen	

Typische Szenarien:

- Zusammenführen zweier Datenbanken mit gleichem Schema
- Zusammenführen aus Quellen mit unterschiedlichen Schemata
- Abgleich einer Benutzerabfrage mit einem Datenelement

Vier Techniken zum Datenabgleich

1. **Rule-based Matching** — Experten definieren Regeln (z.B. IF name_similarity > 0.8 AND zip = zip THEN match)
2. **Learning-based Matching** — Maschinelles Lernen trainiert auf Beispiel-Matches
3. **Matching by Clustering** — Ähnliche Datensätze werden in Cluster gruppiert
4. **Probabilistic Matching (Bayes)** — Wahrscheinlichkeitsbasiert: Wie wahrscheinlich ist ein Match gegeben die Übereinstimmung bei Feld X?

Teil 5: Datenanreicherung (Data Enrichment)

Kernidee

Datenanreicherung = bestehende Daten mit Daten Dritter aus externen Quellen zusammenführen, verknüpft über gemeinsame **Schlüsselattribute** (z.B. PLZ, Ortsname, Koordinaten).

Quellen für Anreicherungsdaten

1. **Open Government Data** — offene Behördendaten (Beispiel: PLZ-Liste der Schweizerischen Post)
2. **Open Public Data** (crowdsourced) — Wikipedia, Wikidata, OpenStreetMap, Flickr

3. Open Corporate Data — Firmendaten, z.B. Google Earth Engine

Anwendungsbeispiele

Bereinigung via Anreicherung:

- Gegeben: Kundenliste mit PLZ+Ort
- Referenz: Gebäudeadressen von Post / OpenStreetMap
- Methode: Fuzzy Match von Kundenliste mit Referenzdaten
- Resultat: Kontrollierte, korrigierte Kundenliste

Geocodierung (eigentliche Anreicherung):

Rezept: Geocodierung

1. Ausgangsdaten haben Adressen (Strasse, PLZ, Ort) aber keine Koordinaten
2. Referenzdatenbank mit Adressen+Koordinaten bereitstellen (GWR, OpenStreetMap)
3. Adress-Standardisierer normalisiert beide Seiten (einheitliches Format)
4. Matcher verknüpft Quelladressen mit Referenzadressen (exakt oder fuzzy)
5. Koordinaten (Latitude/Longitude) werden dem Quelldatensatz hinzugefügt

Ergebnis:

Vorher: Hauser, Pascal, Röschibachstr. 77, 8037 Zürich

Nachher: Hauser, Pascal, Röschibachstr. 77, 8037 Zürich, lat=47.393, lon=8.529

Die Koordinaten erlauben anschliessend Kartenvisualisierungen.

Teil 6: Synthetische Datengenerierung

Warum synthetische Daten?

- Echte Daten haben Lücken
- Zugang zu echten Daten eingeschränkt (Datenschutz, DSGVO)
- Echte Daten noch nicht vorhanden (neue Systeme)
- Tests benötigen reproduzierbare Daten

Anwendungen: ML-Modelltraining, Softwaretests, Datenbankoptimierung, Veröffentlichung offener Daten

Drei Typen synthetischer Daten

1. **Anonymized Data** — echte Daten, aber persönliche Merkmale maskiert/ersetzt
2. **Artificial Data** — künstlich erzeugt, basiert nicht auf echten Einzeldaten
3. **(Pure) Synthetic Data** — vollständig generiert, kein Bezug zu echten Daten

Synthetische Datengenerierung mit PostgreSQL SQL

Rezept: Synthetische Testdaten mit SQL generieren

```
SELECT
  -- Zufällige ganze Zahl zwischen lower_bound und upper_bound
```

```

round(random() * (upper_bound - lower_bound) + lower_bound)::int AS i,
-- Zufällige Dezimalzahl zwischen 0 und 1
random() AS d,
-- Zufälliger String (via MD5-Hash)
upper(substr(md5(random()::text), 1, upper_bound)) AS str,
-- Zufälliger Boolean
random() > 0.5 AS bool,
-- Zufälliger Timestamp in einem Bereich
'2023-04-04'::timestamp
  + (random() * interval '1 month')
  - interval '1 months' AS dt
FROM generate_series(1, 5) AS series, -- 5 Zeilen erzeugen
LATERAL (SELECT 1 AS lower_bound, 4 AS upper_bound) AS bound
ORDER BY dt DESC;

```

Schlüsselfunktionen:

- `generate_series(start, end)` — erzeugt N Zeilen
- `random()` — Zufallszahl $\in [0, 1)$
- `md5(random()::text)` — zufälliger Hash-String
- `LATERAL (SELECT ...)` — definiert benannte Konstanten (lower/upper bound)

PostgreSQL Anonymizer

Gehört zum Typ “Anonymized Data”:

```

SECURITY LABEL FOR anon
ON COLUMN customer.lastname
IS 'MASKED WITH FUNCTION anon.fake_last_name()';

```

Vier Maskierungsmodi:

1. **Statische Maskierung** — PII dauerhaft aus der Datenbank entfernen (irreversibel)
2. **Dynamische Maskierung** — PII nur für bestimmte Benutzer verstecken (View-basiert)
3. **Generalisierung** — “verschwommene” Ansichten für Statistik und ML
4. **Anonyme Dumps** — maskierte Daten als SQL-Datei exportieren (gut für CI/CD)

Tools für synthetische Datengenerierung

Tool	Sprache	Lizenz	Beschreibung
Faker	Python	MIT	Fake-Daten: Namen, Adressen, E-Mails
SDV (Synthetic Data Vault)	Python	Business Source	Tabellarische synthetische Daten
PostgreSQL Anonymizer	Ruby	PostgreSQL Lic.	Anonymisierung in PostgreSQL
PGFaker	TypeScript	MIT	Faker für PostgreSQL
pgsynthdata (OST)	Python	MIT	Syntax via <code>COMMENT ON COLUMN</code>
Plain SQL	SQL	—	<code>generate_series()</code> + <code>random()</code>

Teil 7: Apache Hop — Praktische ETL-Pipelines

Was ist Apache Hop?

Apache Hop ist ein **grafisches ETL/ELT-Tool** (Hybrid-Kategorie):

- Open Source, Java, sehr aktiv
- Desktop-GUI + Web-Variante + Batch-Tool
- Kombiniert Datenflussmanagement mit Dataprocessing
- Unterstützt zahlreiche Quellen und Ziele (CSV, JSON, Datenbanken, ...)

Grundkonzepte

Projekt = organisatorische Einheit, hat ein eigenes Verzeichnis (`Home folder`). Variable `${PROJECT_HOME}` zeigt auf dieses Verzeichnis.

Pipeline = gerichteter Graph von **Blöcken** (Transforms), verbunden durch **Hops** (Pfeile).

Hops:

- **Grüner Hop** = Datensätze, bei denen Bedingung erfüllt (True)
- **Oranger Hop** = Datensätze, bei denen Bedingung nicht erfüllt (False/Fehler)

Apache Hop Projekt erstellen

Rezept: Neues Projekt in Apache Hop anlegen

1. Icon unten links anklicken → “Add project...” wählen
2. Name eingeben (z.B. Sales)
3. “Browse” neben “Home folder” → neues Verzeichnis erstellen, gleich wie Projektname benennen
4. Dialog “Create project lifecycle environment?” mit “No” schliessen
5. CSV-Dateien in das Projektverzeichnis kopieren

Wichtige Apache Hop Blöcke (Transforms)

9.8.4.1. CSV File Input — Daten einlesen

Rezept: CSV-Datei einlesen

1. Block “CSV file input” in Pipeline einfügen
2. Einstellungen:
 - Filename: `${PROJECT_HOME}/dateiname.csv`
 - Delimiter: `,` (oder `;` je nach Datei)
 - Enclosure: `"`
 - “Lazy conversion?” Häkchen **entfernen** (sonst werden Typen nicht korrekt erkannt)
3. Button “Get Fields” klicken → Felder und Datentypen automatisch ermitteln
4. Datentypen prüfen und ggf. korrigieren (z.B. POSTALCODE → **String** statt Integer!)
5. Trim type auf **both** setzen für Felder, die als Lookup-Schlüssel dienen

Wichtig: Postleitzahlen immer als **String** einlesen (führende Nullen gehen sonst verloren, z.B. “0815” wird zu 815).

Exam Trap: Häufiger Fehler: Lazy Conversion aktiviert lassen. Dann werden Datentypen nicht korrekt erkannt und spätere Lookups schlagen fehl. Immer deaktivieren!

Häufiger Fehler: Trim type vergessen. Felder mit führenden/nachfolgenden Leerzeichen matchen beim Lookup nicht. Bei allen Join-/Lookup-Schlüsseln `Trim type = both` setzen.

9.8.4.2. Filter Rows — Datensätze aufteilen

Rezept: Datensätze nach Bedingung aufteilen

1. Block “Filter rows” einfügen
2. Bedingung definieren (z.B. `POSTALCODE IS NULL` oder `POSTALCODE = ""`)
3. Grünen Hop → für erfüllte Bedingung (True)
4. Orangen Hop → für nicht erfüllte Bedingung (False)
5. Dummy-Blöcke am Ende zum Testen verwenden

Anwendung: Fehlende Werte erkennen, Datensätze in zwei Streams aufteilen

9.8.4.3. Stream Lookup — Fehlende Werte ergänzen

Rezept: Lookup (fehlende Werte aus Referenztabelle ergänzen)

1. Referenzdaten (z.B. `zipsorted.csv`) separat einlesen
2. Block “Stream lookup” einfügen
3. Einstellungen:
 - “Lookup step”: Welcher Block liefert die Referenzdaten?
 - “The key(s) to look up the value(s)”: Schlüsselfelder aus Hauptdaten (z.B. `CITY`, `STATE`)
 - “Fields to retrieve”: Welches Feld aus der Referenz soll hinzugefügt werden (z.B. `POSTALCODE` → als `ZIP_RESOLVED`)
4. Der Block fügt ein neues Feld hinzu (z.B. `ZIP_RESOLVED`)

Achtung: Datentypen der Schlüsselfelder müssen auf beiden Seiten identisch sein!

9.8.4.4. Select Values — Felder umbenennen und ersetzen

Rezept: Felder umbenennen, ersetzen, neu anordnen

1. Block “Select values” einfügen
2. Reiter “Select & Alter”:
 - “Get Fields to select” klicken → alle Felder werden eingetragen
 - Feld verschieben: `ZIP_RESOLVED` an Position von `POSTALCODE` schieben
 - `POSTALCODE` aus Liste entfernen
3. Reiter “Meta-data”:
 - `ZIP_RESOLVED` umbenennen zu `POSTALCODE`

Merke: Select values kann drei Dinge:

- Felder auswählen/ausschliessen (Reiter “Select & Alter”)
- Felder umbenennen (Reiter “Meta-data”)
- Datentypen ändern (Reiter “Meta-data”)

9.8.4.5. Append Streams — Streams zusammenführen

Zweck: Zwei Datenströme (z.B. Datensätze mit und ohne PLZ nach Ergänzung) wieder zusammenführen.

- “Head step”: Datensätze die zuerst kommen sollen
- “Tail step”: Datensätze die am Ende angefügt werden sollen

Exam Trap: Append streams setzt voraus, dass beide Streams **dasselbe Schema** haben (gleiche Felder, gleiche Typen). Bei unterschiedlichen Schemata zuerst mit **Select values** angleichen!

9.8.4.6. Value Mapper — Werte ersetzen

Rezept: Einzelne Feldwerte ersetzen

1. Block “Value mapper” einfügen
2. Field name: Zu bereinigendes Feld (z.B. COUNTRY)
3. Mapping:
 - Source value: United States
 - Target value: USA
4. Default: Wert für nicht gemappte Einträge (leer = unverändert lassen)

Anwendung: Inkonsistente Bezeichnungen vereinheitlichen (klassische Datenbereinigung)

9.8.4.7. Number Range — Numerische Kategorisierung

Rezept: Numerischen Wert in Kategorie umwandeln

1. Block “Number range” einfügen
2. Input field: Numerisches Quellfeld (z.B. ORDER_AMOUNT)
3. Output field: Name des neuen Feldes (z.B. ORDER_SIZE)
4. Bereiche definieren:
 - Lower bound | Upper bound | Value (Kategorie-Label)
 - z.B.: 0 | 100 | “small”, 100 | 1000 | “medium”, 1000 | ∞ | “large”

Anwendung: Bestellgrößen kategorisieren, Altersgruppen bilden, Preisklassen zuweisen

9.8.4.8. Table Output — In Datenbank schreiben

Rezept: Daten in Datenbank-Tabelle schreiben

1. Datenbank vorher manuell erstellen (Apache Hop kann keine DB erstellen!):

```
CREATE DATABASE sampledata;
```

2. Block “Table output” in Pipeline einfügen (ersetzt Dummy am Ende)
3. Datenbankverbindung konfigurieren:
 - Datenbank-Icon klicken
 - Typ, Host, Port, DB-Name, Benutzername, Passwort eingeben
 - “Test” klicken um Verbindung zu prüfen
4. Target Table: Tabellennamen eingeben (z.B. SALES_DATA)
5. Button “SQL” klicken → zeigt CREATE TABLE Statement

6. “Execute” klicken → Tabelle wird erstellt
7. Pipeline ausführen

Exam Trap: Apache Hop kann **keine Datenbank erstellen** — nur Tabellen in bestehenden Datenbanken. Datenbank vorher im DBMS anlegen!

9.8.4.9. Formula — Berechnungen durchführen

Rezept: Neue berechnete Felder hinzufügen

1. Block “Formula” einfügen
2. Felder ohne Sonderzeichen im Namen verwenden (erst mit “Select values” umbenennen, Klammern entfernen!)
3. Neues Feld definieren:
 - New field: Name des Ergebnisfeldes (z.B. `temp_celsius`)
 - Formula: mathematischer Ausdruck (z.B. `[kelvin] - 273.15`)
 - Return type: Datentyp des Ergebnisses

Temperaturumrechnung (Beispiel Aufgabe 8):

- Kelvin → Celsius: $T_C = T_K - 273.15$
- Kelvin → Fahrenheit: $T_F = (T_K - 273.15) \times 1.8 + 32$

Wichtig: Falls nicht alle Datensätze einen Wert haben (NULL), Formel entsprechend absichern (NULL-Werte führen zu NULL im Ergebnis — das ist korrekt).

Exam Trap: Häufiger Fehler: Feldnamen mit Klammern () im Namen (z.B. `Boiling Point (K)`) verursachen Fehler in Formeln. **Zuerst** mit “Select values” → Reiter “Meta-data” umbenennen (z.B. `boiling_K`), **dann** Formula-Block verwenden!

9.8.4.10. Zusammenführen ungleicher Datensätze (Aufgabe 7)

Rezept: Zwei CSV-Dateien mit unterschiedlichen Schemata zusammenführen

Wenn zwei Dateien nicht dasselbe Schema haben, vor `Append streams` angleichen:

Für fehlende Felder:

- Block “Add constants” oder “Set field value” → Standardwert hinzufügen
- Beispiel: Feld `ranking` fehlt in Datei A → mit Wert `0.0` ergänzen

Für NULL-Werte:

- Block “If field value is null” → NULL durch Standardwert ersetzen
- Beispiel: `alter` ist NULL → durch `0` ersetzen

Für Feldumbenennung:

- Block “Select values” → Reiter “Meta-data” → Feld umbenennen
- Beispiel: Feld `name` in Datei B → umbenennen zu `nachname`

Reihenfolge:

1. Datei A einlesen → fehlende Felder ergänzen + NULLs ersetzen
2. Datei B einlesen → Felder umbenennen

3. Beide Streams mit `Append streams` zusammenführen
4. Ausgabe als CSV (Block “Text file output”) und JSON (Block “Json output”)

Übersicht: Apache Hop Blöcke und ihre Verwendung

Wenn Aufgabe X, dann Block Y:

- CSV-Datei einlesen → **CSV file input**
- Datensätze nach Bedingung aufteilen → **Filter rows**
- Fehlenden Wert aus Referenztabelle nachschlagen → **Stream lookup**
- Feld umbenennen / entfernen / Typ ändern → **Select values**
- Zwei Streams zusammenführen (gleiches Schema) → **Append streams**
- Einzelne Feldwerte ersetzen (z.B. “United States” → “USA”) → **Value mapper**
- Numerischen Wert kategorisieren (Bereiche) → **Number range**
- Berechnung / Formel auf Feld anwenden → **Formula**
- Fehlenden Feldwert mit Konstante füllen → **Add constants**
- NULL durch Standardwert ersetzen → **If field value is null**
- Daten in DB-Tabelle schreiben → **Table output**
- Daten als CSV schreiben → **Text file output**
- Daten als JSON schreiben → **Json output**
- Pipeline-Endpunkt zum Testen → **Dummy**

Typischer Pipeline-Aufbau in Apache Hop

Tip: Erkennungsmuster für Pipeline-Strukturen:

Einfaches Einlesen + Ausgabe:

CSV file input → Dummy

Filtern + Aufteilen:

[illegible]

Lookup + Zusammenführen:

```

CSV file input (Hauptdaten) → Filter rows → (grün OK) → Append streams → Dummy
                                |
                                → (orange fehlend) → Stream lookup → Select values
└─┘
CSV file input (Referenz) → Stream lookup

```

Bereinigen + Kategorisieren:

CSV file input → Value mapper → Number range → Table output

Unterschiedliche Quellen zusammenführen:

CSV file input (A) → Select values / Add constants / If null → Append streams
 → Output
 CSV file input (B) → Select values (umbenennen) _____

Zusammenfassung: Konzepte auf einen Blick**Schnellreferenz: Welches Konzept bei welcher Aufgabenstellung?**

- “Finde gleiche Einträge in einer Tabelle” → **Exakte Duplikaterkennung** mit ROW_NUMBER + PARTITION BY
- “Finde ähnliche Strings trotz Tippfehlern” → **Levenshtein-Distanz** (kurze Strings) oder **Jaccard** (Wortmengen)
- “Gleiche Entität in zwei verschiedenen Datenbanken” → **Data Matching** (Rule-based, Learning-based, Clustering, Bayes)
- “Fehlende Spalten aus externer Quelle ergänzen” → **Datenanreicherung**, Lookup via gemeinsames Schlüsselattribut
- “Adresse → Koordinaten” → **Geocodierung** (Adress-Standardisierer + Referenz-DB + Matcher)
- “Testdaten ohne Datenschutzproblem erzeugen” → **Synthetische Datengenerierung** (SQL generate_series, Faker, SDV)
- “Echte Daten anonymisieren” → **PostgreSQL Anonymizer** (statisch, dynamisch, Generalisierung, anonyme Dumps)
- “ETL-Pipeline grafisch bauen” → **Apache Hop** (Hybrid-Tool, OSS, Java)
- “Welches ETL-Tool für Geodaten?” → **FME**, alternativ Apache Hop, KNIME, Looker Studio
- “Workflow orchestrieren (kein starkes Transformieren)” → **Apache Airflow** (Python, OSS)
- “Echtzeit-Datenfluss mit Web-GUI” → **Apache NiFi** (Java, OSS)
- “Transformation nur innerhalb Data Warehouse / Cloud” → **dbt** (Python, OSS+komm.)

Exam Trap: Die häufigsten Fehler in Prüfung und Übung:

1. **POSTALCODE als Integer einlesen** → führende Nullen gehen verloren, Lookup schlägt fehl. Immer String!
2. **Lazy Conversion aktiviert lassen** → falsche Datentypen, Lookup-Fehler. Immer deaktivieren!
3. **Trim type vergessen** → Leerzeichen in Schlüsselfeldern verhindern Matches. Bei allen Join-Keys both setzen.
4. **Felder mit Sonderzeichen (Klammern) in Formula verwenden** → Syntaxfehler. Erst umbenennen!

5. **Append streams ohne Schemaangleich** → Felder fehlen oder haben falsche Typen.
Erst mit Select values / Add constants angleichen.
6. **Datenbank nicht vorher erstellen** → Apache Hop kann keine Datenbank anlegen,
nur Tabellen darin.
7. **Edit-Distanz für lange Texte verwenden** → sehr langsam und ungenau. Besser:
Jaccard oder TF/IDF.
8. **Data Matching und String Matching verwechseln** → Matching arbeitet auf
Tupeln (mehrere Felder zusammen), String Matching nur auf einzelnen Strings.

Week 9: [Topic Title]

Woche 9: Introduction and Preprocessing (1)

Data Science & Data Mining — Grundlagen

Was ist Data Science und Data Mining?

Kernidee: Data Science ist das übergeordnete Feld, das den gesamten Lebenszyklus der Datenarbeit umfasst — von der Datensammlung bis zur Präsentation von Erkenntnissen. Data Mining ist ein Teilbereich davon, der sich speziell auf das automatische Extrahieren von Mustern und Wissen aus (grossen) Datensätzen konzentriert.

Tip: Wie unterscheide ich Data Science von Data Mining?

- **Data Mining** = Teilprozess: Muster aus Rohdaten extrahieren, Algorithmen anwenden, KDD-Schritt (Knowledge Discovery in Databases)
- **Data Science** = Oberbegriff: umfasst zusätzlich Datensammlung, Bereinigung, Visualisierung, Kommunikation, Domänenwissen, ML-Integration
- Analogie: Data Mining ist wie das Graben nach Gold — Data Science umfasst auch den Bau der Mine, den Transport und den Verkauf des Goldes.

Beziehung zu Machine Learning:

- Viele DM-Algorithmen sind identisch mit ML-Algorithmen
- Data Mining liefert oft Trainingsdaten und Feature-Verständnis für ML-Modelle
- ML ist ein Werkzeugkasten, den Data Mining benutzt

CRISP-DM — Der Standard-Prozess

Kernidee: CRISP-DM (Cross-Industry Standard Process for Data Mining) beschreibt 6 Phasen, die *iterativ* und nicht strikt sequenziell durchlaufen werden. Rücksprünge sind normal und gewollt.

Die 6 CRISP-DM Phasen im Überblick:

1. **Business Understanding** — Ziele definieren: Was will das Unternehmen erreichen? Welche Fragen sollen beantwortet werden? Grober Plan wird erstellt.
2. **Data Understanding** — Daten sammeln und erstmalig sichten: Welche Daten existieren? Gibt es Qualitätsprobleme? Erste Exploration.
3. **Data Preparation** — Rohdaten bereinigen, transformieren, in das richtige Format bringen, für Modellierung vorbereiten. *Dieser Schritt benötigt am meisten Zeit (60–80% des Aufwands).*
4. **Modeling** — Geeignete Data-Mining-Methoden auswählen und anwenden. Parameter optimieren. Mehrere Modelle ausprobieren.
5. **Evaluation** — Bestes Modell auswählen. Prüfen ob das Modell die Geschäftsfrage wirklich beantwortet. Genauigkeit beurteilen. *Wenig Aufwand dank automatisierter Metriken.*

6. Deployment — Ergebnisse präsentieren oder Modell in Entscheidungsprozesse integrieren.

Merkhilfe: Bu-Du-Da-Mo-Ev-De → “Businessleute Denken Daran, Modelle Endlich Darzustellen”

Tip: Warum ist Data Preparation so aufwändig?

- Daten kommen aus heterogenen Quellen mit verschiedenen Formaten
- Fehlende, fehlerhafte, doppelte Werte müssen manuell oder halbautomatisch korrigiert werden
- Grosse Datensätze erfordern viel Rechenleistung
- “Garbage in, garbage out” — schlechte Daten → schlechte Modelle

Warum ist Evaluation weniger aufwändig?

- Standardisierte, automatisierte Metriken verfügbar (Accuracy, F1, RMSE, ...)
- Strukturierter Prozess, klare Vergleichskriterien

Die vier Data-Mining-Aufgabentypen

Kernidee: Jede Data-Mining-Aufgabe fällt in eine von vier Kategorien. Die Erkennung des Typs bestimmt, welche Methode eingesetzt wird.

Wenn X, dann Y — Welcher Aufgabentyp ist es?

| Fragestellung | Typ | Methoden | Beispiel | |—|—|—|—| | Welche Dinge treten *gemeinsam* auf? | **Association** | Market Basket Analysis, Apriori | Bier + Windeln | | In welche *Gruppen* lassen sich Daten einteilen? | **Clustering** | k-Means, hierarchisch, DBSCAN | Kundensegmente | | Zu welcher *vordefinierten Kategorie* gehört X? | **Classification** | Decision Trees, Naive Bayes, kNN | Spam/kein Spam | | Welchen *numerischen Wert* wird X haben? | **Regression** | Lineare/multiple Regression | Eiscremeverkäufe | | Ist X *ungewöhnlich*? | **Anomaly Detection** | Outlier Detection | Kreditkartenbetrug |

Schlüsselunterschied Classification vs. Clustering:

- Classification: Labels sind *bekannt* (supervised), neuen Datenpunkt einordnen
- Clustering: Labels sind *unbekannt* (unsupervised), Gruppen selbst entdecken

Schlüsselunterschied Regression vs. Classification:

- Regression → numerischer, kontinuierlicher Output (Alter, Preis, Umsatz)
- Classification → kategorischer Output (ja/nein, Spam/Ham, Klasse A/B/C)

Was sind Daten? — Datentypen und Attribute

Datenstruktur: Objekte und Attribute

Kernidee: Ein Dataset besteht aus Data Objects (Zeilen), die durch Attribute (Spalten) beschrieben werden. Jedes Attribut hat einen Typ, der bestimmt, welche Operationen sinnvoll sind.

Begrifflichkeiten:

- Data Object = Sample, Instance, Data Point, Tuple (z.B. ein Kunde, ein Patient)
- Attribut = Feature, Variable, Dimension (z.B. Alter, Farbe, Temperatur)
- Beobachtung = konkreter Wert eines Attributs (z.B. “rot”, 37.2)
- Feature Vector = alle Attribute eines Objekts zusammen

Attributtypen — Das wichtigste Klassifikationssystem

Kernidee: Der Attributtyp bestimmt, welche statistischen Operationen (Mean, Median, Mode) sinnvoll sind und welche Mathe erlaubt ist.

Attributtypen — Übersicht und Entscheidungsregeln:

1. Nominal (kategorisch)

- Werte sind Namen/Kategorien ohne sinnvolle Reihenfolge
- Beispiele: Haarfarbe (schwarz, braun, blond), Städtenamen, Länder, Firmenname
- Spezialfall **Binary**: nur 2 mögliche Werte (ja/nein, männlich/weiblich)
- Erlaubte Operationen: nur Mode (häufigster Wert)
- Mean und Median: *nicht sinnvoll*

2. Ordinal

- Werte haben eine sinnvolle Reihenfolge, aber Abstände sind nicht messbar
- Beispiele: Schulnoten (1–6), T-Shirt-Größen (S, M, L, XL), Kundenzufriedenheit (schlecht/mittel/gut)
- Erlaubte Operationen: Mode und Median sinnvoll
- Mean: *nicht sinnvoll* (der Abstand zwischen S und M ist nicht definiert)

3. Cardinal (numerisch) — Untertypen:

Interval-skaliert:

- Gleiche Abstände zwischen Werten, aber kein absoluter Nullpunkt
- “0” bedeutet nicht “nichts vorhanden”
- Beispiele: Temperatur in °C (0°C = nicht “keine Temperatur”), Kalenderjahr
- Erlaubt: Mean, Median, Mode, Addition/Subtraktion
- *Kein Verhältnis*: Man kann nicht sagen “20°C ist doppelt so warm wie 10°C”

Ratio-skaliert:

- Wie Interval, *aber* mit echtem absolutem Nullpunkt
- “0” bedeutet wirklich “nichts”
- Beispiele: Gewicht (0 kg = kein Gewicht), Grösse, Einkommen, Alter

- Erlaubt: Mean, Median, Mode, alle arithmetischen Operationen
- Verhältnisse sind sinnvoll: “60 kg ist doppelt so schwer wie 30 kg”

Tip: Woran erkenne ich den Attributtyp in einer Aufgabe?

Frage dich:

1. Kann ich die Werte sinnvoll sortieren? → Nein: **Nominal**; Ja: weiter zu 2
2. Sind die Abstände zwischen Werten gleichmässig und messbar? → Nein: **Ordinal**; Ja: weiter zu 3
3. Gibt es einen echten Nullpunkt (0 = “nichts”)? → Ja: **Ratio**; Nein: **Interval**

Schnelltable — Was ist erlaubt?

Operation	Nominal	Ordinal	Interval	Ratio	— — — —	Mode	✓	✓	✓	✓				
Median	X	✓	✓	✓	Mean	X	X	✓	✓	Verhältnisse (×, ÷)	X	X	X	✓

Exam Trap: Häufiger Fehler: Temperatur in °C vs. Kelvin

- Temperatur in °C: *Interval-skaliert* (kein echter Nullpunkt)
- Temperatur in Kelvin: *Ratio-skaliert* (0 K = absoluter Nullpunkt)
- “20°C ist doppelt so warm wie 10°C” ist FALSCH (in Kelvin wäre das 293 K vs. 283 K)

Häufiger Fehler: IQ-Werte

- IQ ist *Interval-skaliert*, nicht Ratio
- IQ 0 bedeutet nicht “keine Intelligenz”
- Man kann nicht sagen “IQ 120 ist doppelt so intelligent wie IQ 60”

Datenqualität und Preprocessing

Datenqualität — Dimensionen

Kernidee: Schlechte Datenqualität führt direkt zu schlechten Mining-Ergebnissen (“Garbage in, garbage out”). Datenqualität wird durch mehrere Dimensionen definiert.

Dimensionen der Datenqualität:

- **Accuracy** — Sind die Werte korrekt? (z.B. negativer Preis −100 CHF)
- **Completeness** — Fehlen Werte? (Missing Values)
- **Consistency** — Widersprüche oder verschiedene Formate für dasselbe? (01/02/2025 vs. 2025/02/01)
- **Timeliness** — Sind die Daten aktuell genug?
- **Believability** — Sind die Daten glaubwürdig?
- **Interpretability** — Sind die Daten verständlich dokumentiert?

Preprocessing-Techniken

Kernidee: Vier Haupttechniken verbessern gemeinsam die Datenqualität vor dem eigentlichen Mining.

Preprocessing-Techniken — Wann welche?

- **Data Integration** → Wenn Daten aus mehreren Quellen kommen und zusammengeführt werden müssen
- **Data Cleaning** → Wenn fehlende Werte, Rauschen, Ausreisser oder Inkonsistenzen vorhanden sind
- **Data Reduction** → Wenn der Datensatz zu gross ist oder irrelevante Features entfernt werden sollen
- **Data Transformation** → Wenn Werte in andere Skalierungen oder Formate gebracht werden müssen (z.B. Normalisierung)

Statistische Grundlagen — Lagemasse

Arithmetisches Mittel (Mean)

Kernidee: Der Durchschnitt aller Werte — misst das “Zentrum” eines Datensatzes, ist aber empfindlich gegenüber Ausreißern.

Formel und Rezept:

Gegeben: N Beobachtungen $x_1, x_2, \dots, x_N$ eines Attributs X

$$\bar{x} = \frac{\sum_{i=1}^N x_i}{N}$$

Gewichtetes Mittel (wenn jeder Wert ein Gewicht w_i hat):

$$\bar{x} = \frac{\sum_{i=1}^N w_i x_i}{\sum_{i=1}^N w_i}$$

Rezept:

1. Alle Werte aufsummieren: $S = x_1 + x_2 + \dots + x_N$
2. Durch Anzahl der Werte dividieren: $\bar{x} = \frac{S}{N}$

Beispiel (Temperaturen aus Vorlesung, $N = 14$): $S = 28.3 + 27.2 + 27.4 + 22.7 + 14.3 + 11.9 + 13.8 + 19.8 + 9.6 + 21.1 + 20.8 + 19.8 + 25.3 + 22.8 = 284.8$ $\bar{x} = \frac{284.8}{14} = 20.34$

Übungsaufgabe 9a (Alter, $N = 27$): $S = 809$, $\bar{x} = \frac{809}{27} \approx 29.96$

Exam Trap: Mean ist sensitiv gegenüber Ausreißern!

- Ein einzelner extrem hoher/niedriger Wert verschiebt den Mean stark
- Lösung: **Trimmed Mean** — die obersten und untersten $k\%$ der Werte entfernen, dann Mean berechnen
- Bei schiefen Verteilungen ist Median robuster als Mean

Median

Kernidee: Der mittlere Wert in einer sortierten Reihe — robuster als der Mean bei Ausreißern.

Rezept zur Median-Berechnung:

1. **Daten sortieren** (aufsteigend) — dieser Schritt wird oft vergessen!
2. Anzahl der Werte N bestimmen
3. Falls N **ungerade**: Median = Wert an Position $\frac{N+1}{2}$
4. Falls N **gerade**: Median = Durchschnitt der Werte an Position $\frac{N}{2}$ und $\frac{N}{2} + 1$

Beispiel ungerade ($N = 27$, Alters-Daten sortiert): Position des Medians = $\frac{27+1}{2} = 14 \rightarrow$ 14. Wert in sortierter Reihe Sortiert: 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, **25**, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70 Median = **25**

Beispiel gerade ($N = 14$, Temperaturen): Sortiert: 9.6, 11.9, 13.8, 14.3, 19.8, 19.8, **20.8**, **21.1**, 22.7, 22.8, 25.3, 27.2, 27.4, 28.3 Positionen 7 und 8: 20.8 und 21.1 Median = $\frac{20.8+21.1}{2} = 20.95$

Exam Trap: Häufigster Fehler beim Median: Vergessen zu sortieren! Die Musterlösung warnt explizit: “wenn nicht sortiert, dann kommt man aus 32.9 (typischer Fehler!)”

- **Immer** erst sortieren, dann Median berechnen
- Für %fat-Daten aus Aufgabe 10: sortiert ergibt sich Median 30.7, nicht 32.9

Mode (Modalwert)

Kernidee: Der am häufigsten vorkommende Wert — der einzige Lage-Kennwert, der für alle Attributtypen (auch nominal) sinnvoll ist.

Rezept:

1. Häufigkeit jedes Wertes zählen
2. Wert(e) mit der höchsten Häufigkeit = Mode

Unimodal: Ein Mode $\rightarrow$ z.B. Mode = 19.8 (in Temperaturdaten, kommt $2\times$ vor)

Bimodal: Zwei Modi $\rightarrow$ z.B. Mode = {25, 35} (in Altersdaten, beide kommen $4\times$ vor)

Multimodal: Mehr als zwei Modi

Beispiel (Alters-Daten):

- 25 kommt $4\times$ vor: 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, **25, 25, 25, 25**, 30, ...
- 35 kommt $4\times$ vor: ..., **35, 35, 35, 35**, 36, 40, ...
- Alle anderen Werte kommen seltener vor
- Ergebnis: Mode = {25, 35} $\rightarrow$ **bimodal**

Statistische Grundlagen — Streumasse

Range (Spannweite)

Kernidee: Einfachstes Mass für die Streuung — Differenz zwischen Maximum und Minimum.

$$R = \max(X) - \min(X)$$

Beispiel (Temperaturen): $R = 28.3 - 9.6 = 18.7$ **Beispiel** (Alter): $R = 70 - 13 = 57$

Nachteil: Sehr empfindlich gegenüber Ausreissern — ein einzelner extremer Wert verändert den Range stark.

Quantile und Quartile

Kernidee: q -Quantile teilen die sortierten Daten in q gleichgrosse Gruppen auf. Die wichtigsten sind Quartile (4 Gruppen) und Perzentile (100 Gruppen).

Quartile berechnen — Rezept:

1. Daten sortieren (aufsteigend)
2. **Median (Q_2)** berechnen (teilt Daten in zwei Hälften)
3. Q_1 = Median der *unteren* Hälfte (Werte unter dem Gesamtmedian)
4. Q_3 = Median der *oberen* Hälfte (Werte über dem Gesamtmedian)

Wichtig: Bei ungerader Anzahl von Werten wird der Median-Wert selbst typischerweise von beiden Hälften **ausgeschlossen** (Methode variiert je nach Software/Konvention — bei Handrechnung: Median aus den Werten links/rechts des Gesamtmedians).

Beispiel ($N = 27$ Alterswerte): Sortiert: 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, | **25** | 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70

Untere Hälfte (13 Werte links): 13, 15, 16, 16, 19, 20, **20**, 21, 22, 22, 25, 25, 25 $\rightarrow Q_1$ = Median = Wert an Position 7 = **20** Aber: 7. und 8. Wert = 20 und 21 $\rightarrow Q_1 = \frac{20+21}{2} = 20.5$ (*gemäss Musterlösung*)

Obere Hälfte (13 Werte rechts): 30, 33, 33, 35, 35, 35, **35**, 36, 40, 45, 46, 52, 70 $\rightarrow Q_3$ = Median = Wert an Position 7 = **35**

Interquartile Range (IQR):

$$IQR = Q_3 - Q_1$$

Beispiel: $IQR = 35 - 20.5 = 14.5$

Exam Trap: Verschiedene Quartilmethoden geben leicht unterschiedliche Ergebnisse!

- In der Prüfung: bei Handrechnung den Median der unteren/oberen Hälfte verwenden
- Bei gerader Anzahl von Werten in jeder Hälfte: Durchschnitt der zwei mittleren Werte
- Konsistenz ist wichtiger als die exakte Methode — solange du erklärst was du tust

Fünf-Zahlen-Zusammenfassung (Five-Number Summary)

Kernidee: Fünf Kennzahlen beschreiben die Verteilung vollständig — von den Extremen bis zur Mitte.

$$\text{Five-Number Summary} = \{\text{Minimum}, Q_1, \text{Median}(Q_2), Q_3, \text{Maximum}\}$$

Beispiel (Altersdaten):

$$\{13, 20.5, 25, 35, 70\}$$

Boxplot

Kernidee: Visuelle Darstellung der Five-Number Summary — zeigt Verteilung, Streuung und Ausreisser auf einen Blick.

Boxplot zeichnen — Rezept:

1. Box zeichnen von Q_1 bis Q_3 (Länge = IQR)
2. Linie in der Box bei Q_2 (Median)
3. **Whisker-Grenzen berechnen:**
 - Untere Grenze: $Q_1 - 1.5 \times IQR$
 - Obere Grenze: $Q_3 + 1.5 \times IQR$
4. Whisker reichen bis zum **letzten Datenpunkt innerhalb** der Grenzen (nicht bis zur Grenze selbst!)
5. Datenpunkte **ausserhalb** der Whisker-Grenzen = **Ausreisser** → einzeln einzeichnen (oft als Punkte/Sterne)

Beispiel (Altersdaten):

- $Q_1 = 20.5$, $Q_3 = 35$, $IQR = 14.5$
- Untere Whisker-Grenze: $20.5 - 1.5 \times 14.5 = 20.5 - 21.75 = -1.25$ → letzter Datenpunkt über -1.25 ist **13** → unterer Whisker bis 13
- Obere Whisker-Grenze: $35 + 1.5 \times 14.5 = 35 + 21.75 = 56.75$ → letzter Datenpunkt unter 56.75 ist **52** → oberer Whisker bis 52
- Wert $70 > 56.75$ → **Ausreisser**, einzeln markieren

Tip: Boxplot-Interpretation — Was sehe ich?

- Box weit oben im Gesamtbereich → rechtsschiefe Verteilung (viele kleine Werte, wenige grosse)
- Median nahe an Q_1 → mehr Werte im oberen Bereich
- Langer oberer Whisker → Ausreisser nach oben
- Einzelne Punkte = Ausreisser = potentielle Datenfehler oder echte Extremwerte

Varianz und Standardabweichung

Kernidee: Varianz misst die durchschnittliche quadratische Abweichung vom Mittelwert. Standardabweichung ist die Wurzel davon — leichter interpretierbar da in den Origineleinheiten.

Formeln:

Varianz (σ^2):

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

Standardabweichung (σ):

$$\sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Rezept:

1. Mean $\bar{x}$ berechnen
2. Für jeden Wert x_i : Abweichung $(x_i - \bar{x})$ berechnen
3. Abweichungen quadrieren: $(x_i - \bar{x})^2$
4. Alle quadrierten Abweichungen summieren
5. Durch N dividieren $\rightarrow$ Varianz σ^2
6. Wurzel ziehen $\rightarrow$ Standardabweichung σ

Beispiel (Temperaturdaten, $\bar{x} = 20.34$, $N = 14$): $\sigma^2 = \frac{1}{14} \cdot [(28.3 - 20.34)^2 + (27.2 - 20.34)^2 + \dots] \approx 35.67 \rightarrow$ Musterlösung gibt $\sigma = \sqrt{35.67} \approx 5.97$ als Varianz an (*hier als σ^2 notiert, Werte aus Originalfolie*)

Aufgabe 10 — Alters-Daten ($N = 18$, $\bar{x} = 46.4$): $\sigma^2 = 165.0247$, $\sigma = 12.846$

Aufgabe 10 — %fat-Daten ($N = 18$, $\bar{x} = 28.783$): $\sigma^2 = 80.886$, $\sigma = 8.994$

Tip: Interpretation der Standardabweichung:

- Kleine $\sigma \rightarrow$ Werte liegen nahe am Mean (geringe Streuung)
- Grosse $\sigma \rightarrow$ Werte sind weit verteilt (hohe Streuung)
- **68-Regel** (Normalverteilung): ca. 68% aller Werte liegen im Bereich $[\bar{x} - \sigma, \bar{x} + \sigma]$
- **Einheit:** σ hat dieselbe Einheit wie die Daten (z.B. Jahre, kg, °C)
- **Einheit von σ^2 :** Quadrat der Origineleinheit (z.B. Jahre²) $\rightarrow$ weniger intuitiv

Welches Lagemass / Streumass bei welchem Datentyp?

	Nominal	Ordinal	Interval/Ratio	Mean	Median	Mode	Range	Varianz / σ
Mean	X	X	✓	✓	X	X	X	X
Median	X	✓	✓	✓	✓	✓	✓	✓
Mode	✓	✓	✓	✓	✓	✓	✓	✓
Range	X	X	✓	✓	✓	✓	✓	✓
Varianz / σ	X	X	✓	✓	✓	✓	✓	✓

Robustheit gegen Ausreisser:

- Robust: Median, IQR, Trimmed Mean
- Nicht robust: Mean, Range, Varianz (durch quadrieren werden Ausreisser stark gewichtet)

Datenvisualisierung — Überblick

Histogramm

Kernidee: Zeigt die Häufigkeitsverteilung eines Attributs — für nominale Daten als Balkendiagramm (mit Lücken), für kardinale Daten als Histogramm (ohne Lücken, mit Bins).

Für nominale Attribute: Ein Balken pro Kategorie, Höhe = Häufigkeit **Für kardinale Attribute:**

- Wertebereich in gleichbreite **Bins** aufteilen (equiwidth-binning)
- Höhe eines Bins = Anzahl Werte in diesem Bereich
- Gibt intuitiven Eindruck der Verteilungsform (symmetrisch, schief, bimodal?)

Scatter Plot (Streudiagramm)

Kernidee: Visualisiert Beziehungen zwischen zwei Attributen — jeder Datenpunkt wird als Punkt im 2D-Raum dargestellt.

Tip: Wann verwende ich einen Scatter Plot?

- Wenn ich wissen will: Gibt es eine Korrelation zwischen Attribut A und Attribut B?
- Erkennung von: Cluster, Ausreisser, lineare/nichtlineare Trends
- Funktioniert für alle Attributtypen (auch gemischt)

Interpretation:

- Punkte bilden eine Linie (aufwärts/abwärts) → positive/negative Korrelation
- Punkte ohne Muster → keine/schwache Korrelation
- Korrelationskoeffizient nahe ± 1 → starke Korrelation (Aufgabe 10: 0.82 = starke positive Korrelation zwischen Alter und %fat)

Weitere Visualisierungstypen

Scatterplot Matrix: $n \times n$ Matrix aller paarweisen Scatter Plots für n -dimensionale Daten. Diagonale oft mit Histogrammen, oberes Dreieck mit Korrelationskoeffizienten. Wird bei höherer Dimensionalität unübersichtlich.

Liniendiagramm (Line Graph): Zeigt wie sich eine kontinuierliche Variable über eine andere verändert (meist Zeitreihen). Weit verbreitet in Finanzen, Marketing, Medizin.

Q-Q-Plot (Quantile-Quantile Plot): Prüft ob Daten normalverteilt sind.

- Punkte liegen auf der 45°

Week 10: [Topic Title]

Week 10: Preprocessing (2) & (3) — Noisy/Missing Data, Correlation, Normalization, Discretization, Data Reduction

1. Fehlende Daten (Missing Data)

Kernidee: Fehlende Werte können Analysen verzerren oder ineffizient machen — man muss verstehen *warum* sie fehlen, bevor man sie behandelt.

1.1 Typen von fehlenden Daten

Tip: Woran erkenne ich den Typ?

- **MCAR** (Missing Completely At Random): Fehlende Werte hängen weder von beobachteten noch von fehlenden Werten ab. Zufällig wie ein Geräteausfall. Test: Little's MCAR-Test.
- **MAR** (Missing At Random): Das Fehlen lässt sich durch *andere beobachtete* Variablen erklären. z.B. Männer geben seltener Gewicht an — aber wenn man Geschlecht kennt, kann man das modellieren.
- **MNAR** (Missing Not At Random): Das Fehlen hängt vom fehlenden Wert selbst ab. z.B. sehr reiche Personen geben Einkommen nicht an. Schwierigster Fall — systematischer Bias.

Konsequenzen:

- **MCAR:** Kein Bias, nur Effizienzverlust (weniger Daten)
- **MAR:** Bias möglich, aber korrigierbar
- **MNAR:** Systematischer Bias — schwer zu korrigieren

1.2 Umgang mit fehlenden Werten

Strategien (von einfach zu komplex):

1. **Löschen** — Tupel/Attribute mit fehlenden Werten entfernen (nur sinnvoll wenn wenige fehlen und MCAR)
2. **Manuelle Befüllung** — zeitaufwändig, erfordert Domänenwissen
3. **Globale Konstante** — ersetze durch “unknown” oder 0 (einfach, aber kann Algorithmen täuschen)
4. **Lagemass** — ersetze durch Mittelwert (numerisch), Median (bei Ausreißern), Modus (nominal)
5. **Regression** — schätze fehlenden Wert durch Regressionsmodell auf anderen Attributen
6. **Maximum Likelihood** — schätze Verteilungsparameter und impute basierend darauf
7. **Machine Learning Methoden** — modernster Ansatz (z.B. KNN-Imputation)

Exam Trap: Häufiger Fehler: Immer den Mittelwert nehmen. Bei schiefer Verteilung oder Ausreißern ist der **Median** besser. Bei nominalen Daten ist nur der **Modus** sinnvoll.

2. Noisy Data & Binning zur Glättung

Kernidee: Reale Daten = wahres Signal + zufälliger Fehler (Rauschen). Glättung reduziert Rauschen, indem lokale Mittelwerte oder Grenzen verwendet werden.

2.1 Binning-Methoden zur Glättung

Rezept: Smoothing by Bin Means

1. Daten sortieren (aufsteigend)
2. In Bins der Tiefe k aufteilen (equidepth: je k Werte pro Bin)
3. Für jeden Bin: Mittelwert berechnen
4. Alle Werte im Bin durch den Bin-Mittelwert ersetzen

Rezept: Smoothing by Bin Boundaries

1. Daten sortieren
2. In Bins aufteilen
3. Jeden Wert durch die nächste Bin-Grenze (min oder max des Bins) ersetzen — je nachdem welche näher liegt

Beispiel (Vorlesungsdaten, Bin-Tiefe 3):

Daten: 4, 8, 15, 21, 21, 24, 25, 28, 34

- Bin 1: $\{4, 8, 15\} \rightarrow \text{Mittelwert} = \frac{4+8+15}{3} = 9 \rightarrow \{9, 9, 9\}$
- Bin 2: $\{21, 21, 24\} \rightarrow \text{Mittelwert} = \frac{21+21+24}{3} = 22 \rightarrow \{22, 22, 22\}$
- Bin 3: $\{25, 28, 34\} \rightarrow \text{Mittelwert} = \frac{25+28+34}{3} = 29 \rightarrow \{29, 29, 29\}$

Grenzen-Glättung Bin 1: $4 \rightarrow 4$ (näher an min=4), $8 \rightarrow 4$ (näher an min=4), $15 \rightarrow 15$ (näher an max=15) Resultat Bin 1: $\{4, 4, 15\}$

Übungsaufgabe 1a — Rezept anwenden:

Daten: 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70 (27 Werte, Bin-Tiefe 3 $\rightarrow$ 9 Bins)

$$\text{Bin 1: } \{13, 15, 16\} \rightarrow \text{Mittel} = \frac{44}{3} \approx 14.67 \rightarrow \{14.67, 14.67, 14.67\}$$

$$\text{Bin 2: } \{16, 19, 20\} \rightarrow \text{Mittel} \approx 18.33$$

... und so weiter für alle 9 Bins.

Effekt: Ausreisser wie “70” wird gedämpft (Bin 9: $\{46, 52, 70\} \rightarrow \text{Mittel} = 56$).

Tip: Wann brauche ich Binning?

- Aufgabe erwähnt “smoothing”, “Rauschen reduzieren” oder “Ausreisser abmildern”
- Aufgabe gibt eine Liste von Zahlen und eine Bin-Tiefe (equidepth) oder Anzahl Bins
- Schlüsselwörter: “bin means”, “bin boundaries”, “equidepth”, “equiwidth”

2.2 Ausreisser erkennen und behandeln

Ausreisser = Werte die weit vom erwarteten Bereich abweichen. Sie sind eine spezielle Form von Noise.

Erkennungsmethoden:

- **Statistisch (univariat):** Boxplot — Ausreisser ausserhalb $1.5 \times \text{IQR}$ ($\text{IQR} = Q3 - Q1$)
- **Multivariat:** Cluster-Methoden, Distanzmetriken
- **Visuell:** Scatter Plots, Histogramme

Behandlungsstrategien:

- **Analysieren:** Sind Ausreisser das Ziel? (z.B. Betrugsdetekton)
- **Löschen:** Wenn sie einen kleinen Teil ausmachen und keinen Informationswert haben
- **Binning:** Neue Kategorie erstellen z.B. " ≥ 300 "
- **Imputation:** Wie fehlende Werte behandeln (nur bei Eingabefehler)

Exam Trap: Niemals Ausreisser blind löschen ohne zu verstehen woher sie kommen! Sie können der wertvollste Teil der Daten sein (z.B. Anomalie-Erkennung).

3. Korrelationsanalyse

3.1 Pearson Korrelationskoeffizient (für numerische Daten)

Kernidee: Misst die Stärke und Richtung des linearen Zusammenhangs zwischen zwei numerischen Variablen. Resultat immer zwischen -1 und $+1$.

Formel (Sample Pearson):

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Variablen:

- x_i, y_i : Beobachtungspaare
- $\bar{x}, \bar{y}$: Mittelwerte der jeweiligen Variablen
- n : Anzahl Beobachtungen

Rezept:

1. Mittelwerte $\bar{x}$ und $\bar{y}$ berechnen
2. Für jedes Paar (x_i, y_i) : Abweichungen $(x_i - \bar{x})$ und $(y_i - \bar{y})$ berechnen
3. Zähler: Summe der Produkte der Abweichungen
4. Nenner: Wurzel aus (Summe der quadrierten Abweichungen von x) $\times$ (Summe der quadrierten Abweichungen von y)
5. Dividieren

Beispiel (Übungsaufgabe 2):

$$\bar{x} = \frac{43+21+25+42+57+59}{6} = \frac{247}{6} \approx 41.17$$

$$\bar{y} = \frac{99+65+79+75+87+81}{6} = \frac{486}{6} = 81$$

Tabelle der Abweichungen:

$ x_i $	$ y_i $	$ x_i - \bar{x} $	$ y_i - \bar{y} $	Produkt										
43	99	1.83	18	32.94	21	65	-20.17	-16	322.72	25	79	-16.17	-2	32.34
42	75	0.83	-6	-4.98	57	87	15.83	6	94.98	59	81	17.83	0	0

$$\text{Zähler} \approx 32.94 + 322.72 + 32.34 - 4.98 + 94.98 + 0 = 478$$

$r \approx 0.5298 \rightarrow$ moderate positive Korrelation

Tip: Interpretation:

- $r = 1$: perfekte positive lineare Korrelation
- $r = -1$: perfekte negative lineare Korrelation
- $r = 0$: kein linearer Zusammenhang
- $|r| \geq 0.7$: starke Korrelation
- $0.3 \leq |r| < 0.7$: moderate Korrelation
- $|r| < 0.3$: schwache Korrelation

Achtung: Korrelation $\neq$ Kausalität! (Eis-Konsum korreliert mit Ertrinkungstoten — weil beides im Sommer steigt.)

3.2 Chi-Quadrat-Test (für nominale Daten)

Kernidee: Prüft ob zwei nominale (kategoriale) Variablen statistisch unabhängig sind, indem beobachtete Häufigkeiten mit erwarteten Häufigkeiten (unter Unabhängigkeitsannahme) verglichen werden.

Formel:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

Variablen:

- O_i : beobachteter Wert in Zelle i
- E_i : erwarteter Wert in Zelle i (unter Unabhängigkeit)

Erwarteter Wert:

$$E_{ij} = \frac{\text{Zeilensumme}_i \times \text{Spaltensumme}_j}{\text{Gesamtsumme } N}$$

Rezept:

1. Kontingenztafel mit beobachteten Werten aufstellen
2. Randsummen (Zeile + Spalte) berechnen
3. Für jede Zelle: $E_{ij} = \frac{\text{Zeilensumme} \times \text{Spaltensumme}}{N}$ berechnen
4. Für jede Zelle: $\frac{(O_{ij} - E_{ij})^2}{E_{ij}}$ berechnen
5. Alles summieren $\rightarrow \chi^2$ -Wert
6. Freiheitsgrade: $\text{DOF} = (r - 1)(c - 1)$ wobei r = Zeilen, c = Spalten
7. Mit kritischem Wert aus Tabelle vergleichen: Wenn $\chi^2 > \chi^2_{\text{krit}}$ $\rightarrow$ abhängig

Beispiel (Übungsaufgabe 3):

Beobachtete Werte:

	Compact	Luxury	Summe
1 or less	30	0	30
2 or 3	20	10	30
more than 3	10	30	40
Summe	60	40	100

Erwartete Werte ($E_{ij} = \frac{ZS \times SS}{100}$):

	Compact	Luxury
1 or less	$\frac{30 \times 60}{100} = 18$	$\frac{30 \times 40}{100} = 12$
2 or 3	$\frac{30 \times 60}{100} = 18$	$\frac{30 \times 40}{100} = 12$
more than 3	$\frac{40 \times 60}{100} = 24$	$\frac{40 \times 40}{100} = 16$

Chi-Quadrat:

$$\chi^2 = \frac{(30 - 18)^2}{18} + \frac{(0 - 12)^2}{12} + \frac{(20 - 18)^2}{18} + \frac{(10 - 12)^2}{12} + \frac{(10 - 24)^2}{24} + \frac{(30 - 16)^2}{16}$$

$$= 8 + 12 + 0.222 + 0.333 + 8.167 + 12.25 \approx 40.97$$

$$\text{DOF} = (3 - 1)(2 - 1) = 2, \alpha = 0.1 \rightarrow \text{kritischer Wert} \approx 4.605$$

Da $40.97 \gg 4.605 \rightarrow$ **Carsize und NrOffFines sind abhängig!**

Exam Trap: Häufige Fehler beim Chi-Quadrat-Test:

- DOF vergessen: $(r - 1)(c - 1)$, nicht $r \times c$
- Erwartete Werte falsch berechnet: immer $\frac{\text{Zeilensumme} \times \text{Spaltensumme}}{\text{Gesamtsumme } N}$
- Test sagt “abhängig” $\rightarrow$ bedeutet **nicht** kausal abhängig, nur statistisch assoziiert

Wann welcher Korrelationstest?

- Numerische Daten, linearer Zusammenhang $\rightarrow$ **Pearson r**
- Nominale/kategoriale Daten $\rightarrow$ **Chi-Quadrat χ^2**
- Chi-Quadrat funktioniert auch für nicht-lineare Abhängigkeiten

4. Normalisierung

Kernidee: Attribute mit verschiedenen Einheiten/Skalen werden auf einen gemeinsamen Bereich gebracht, damit kein Attribut durch seinen Wertebereich künstlich übergewichtet wird.

Wann nötig? Bei Algorithmen die auf Distanzen oder Gewichten basieren: k-NN, Clustering, neuronale Netze, lineare Regression.

4.1 Min-Max-Normalisierung

Formel (auf $[0, 1]$):

$$v'_i = \frac{v_i - \min_A}{\max_A - \min_A}$$

Formel (auf beliebiges $[\min_R, \max_R]$):

$$v'_i = \frac{v_i - \min_A}{\max_A - \min_A} \cdot \left(\max_R - \min_R \right) + \min_R$$

Variablen:

- v_i : zu normalisierender Wert
- $\min_A, \max_A$: Minimum und Maximum des Attributs A im Datensatz
- $\min_R, \max_R$: gewünschter Zielbereich

Rezept:

1. $\min_A$ und $\max_A$ im Datensatz bestimmen
2. Formel einsetzen
3. Bei Zielbereich $[\min_R, \max_R]$: erweiterte Formel verwenden

Beispiel (Übungsaufgabe 6a): Alter: $\min = 13$, $\max = 70$, zu normalisieren: $v_i = 35$

$$v'_i = \frac{35 - 13}{70 - 13} = \frac{22}{57} \approx 0.386 \quad [0, 1]$$

Für $[-1, 1]$:

$$v'_i = 0.386 \cdot (1 - (-1)) + (-1) = 0.386 \cdot 2 - 1 \approx -0.228 \quad [-1, 1]$$

Wertebereich: $[\min_R, \max_R]$ — frei wählbar, Standard $[0, 1]$

4.2 Decimal Scaling

Formel:

$$v'_i = \frac{v_i}{10^j}$$

j bestimmen: kleinste ganze Zahl sodass $\max(|v'_i|) < 1$, d.h. $j = \lceil \log_{10}(\max(|v_i|) + 1) \rceil$

Rezept:

1. Betrag des grössten absoluten Werts im Attribut bestimmen

2. j = Anzahl Stellen dieser Zahl (kleinstes j so dass $\max|v'| < 1$)
3. Alle Werte durch 10^j teilen

Beispiel (Übungsaufgabe 6b): Alter: $\max(|A|) = 70 \rightarrow j = 2$ (weil $\frac{70}{100} = 0.70 < 1$, aber $\frac{70}{10} = 7.0 \geq 1$)

$$v'_i = \frac{35}{10^2} = \frac{35}{100} = 0.35$$

Wertebereich: $(-1, 1)$

4.3 Z-Score Normalisierung (Standardisierung)

Formel:

$$v'_i = \frac{v_i - \bar{A}}{\sigma_A}$$

Bei Ausreissern statt σ die mittlere absolute Abweichung (MAD) verwenden:

$$v'_i = \frac{v_i - \bar{A}}{S'_A} \quad \text{mit} \quad S'_A = \frac{1}{N} \sum_{k=1}^N |v_k - \bar{A}|$$

Variablen:

- $\bar{A}$: Mittelwert des Attributs
- σ_A : Standardabweichung des Attributs
- S'_A : mittlere absolute Abweichung

Rezept:

1. Mittelwert $\bar{A}$ berechnen
2. Standardabweichung σ_A (gegeben oder berechnen)
3. Formel anwenden

Beispiel (Übungsaufgabe 6c): $\bar{A} = 29.96$, $\sigma_A = 12.94$, $v_i = 35$

$$v'_i = \frac{35 - 29.96}{12.94} = \frac{5.04}{12.94} \approx 0.389$$

Wertebereich: $(-\infty, +\infty)$ in Einheiten von Standardabweichungen. Typisch ca. $[-3, 3]$.

Welche Normalisierung wann?

- **Min-Max:** Wenn Min/Max bekannt und sinnvoll (z.B. Alter: 0–100). Sensibel gegenüber Ausreissern.
- **Decimal Scaling:** Schnell und einfach. Wenn nur eine Grössenordnung korrigiert werden soll.
- **Z-Score:** Wenn Min/Max unbekannt oder Ausreisser die Skalierung dominieren würden. Setzt ungefähr Normalverteilung voraus.
- Für Alter mit bekannten Grenzen 0–100: **Min-Max** bevorzugt
- Wenn dominante Ausreisser vorhanden: **Z-Score mit MAD**

Exam Trap: Wertebereich-Verwirrung:

- Min-Max $[0, 1]$: $v'_i = \frac{v_i - \min_A}{\max_A - \min_A}$ — Formel ohne R -Term
- Min-Max $[-1, 1]$: $v'_i = \frac{v_i - \min_A}{\max_A - \min_A} \cdot 2 - 1$ — Nicht $\frac{v_i - \min_A}{\max_A - \min_A} \cdot (-1)$ addieren!
- Decimal Scaling: Bereich ist $(-1, 1)$, nicht $[0, 1]$!
- Z-Score: **kein** fixer Bereich — kann theoretisch jeden Wert annehmen

5. Diskretisierung (Discretization)

Kernidee: Kontinuierliche (numerische) Werte werden in diskrete Kategorien/Intervalle umgewandelt. Wichtig: Die Bins müssen so definiert sein, dass **jeder unbekannte Wert** eindeutig einem Bin zugewiesen werden kann.

5.1 Equidepth Binning (gleich viele Elemente pro Bin)

Rezept:

1. Daten sortieren (aufsteigend)
2. Anzahl Bins k festlegen
3. Elemente pro Bin: $m = \frac{N}{k}$ (aufrunden wenn nötig)
4. Grenzen nach jedem m -ten sortierten Wert setzen
5. **Grenzen adjazent und disjunkt** machen: letzte Grenze des Bins i = erste Grenze des Bins $i + 1$
6. Sinnvolle Labels vergeben
7. Grenzen über Trainingsdata hinaus erweitern (z.B. 0 und 100 für Alter)

Beispiel (Übungsaufgabe 8a, $N = 12$, $k = 3$):

Sortiert: 7, 7, 8, 9, 11, 12, 13, 22, 22, 36, 38, 64

$m = \frac{12}{3} = 4$ Elemente pro Bin

- Bin 1: {7, 7, 8, 9} → Grenze: [0, 11) → Label “Children”
- Bin 2: {11, 12, 13, 22} → Grenze: [11, 22) → Label “Youths”
- Bin 3: {22, 36, 38, 64} → Grenze: [22, 100] → Label “Adults”

Wichtig: Einer der beiden Werte “22” landet in Bin 2, der andere in Bin 3 → daher obere Grenze von Bin 2 = < 22 (exklusiv), untere Grenze von Bin 3 = ≥ 22 (inklusive).

5.2 Equiwidth Binning (gleich breite Intervalle)

Rezept:

1. Wertebereich bestimmen: $\text{Range} = \max_A - \min_A$
2. Breite pro Bin: $w = \frac{\text{Range}}{k}$
3. Bin-Grenzen: $\min_A, \min_A + w, \min_A + 2w, \dots, \max_A$
4. Labels vergeben, Grenzen sinnvoll erweitern

Beispiel (Übungsaufgabe 8b, $k = 3$):

$\text{Range} = 64 - 7 = 57$, $w = \frac{57}{3} = 19$

- Bin 1: [0, 26) → “young” (0 bis 7+19=26)
- Bin 2: [26, 45) → “middle”
- Bin 3: [45, 100] → “old”

5.3 Bin-Grenzen und Zuweisung unbekannter Instanzen

Tip: Die wichtigsten Regeln für Bin-Grenzen:

1. **Disjunkt:** Kein Wert darf in zwei Bins fallen
2. **Adjazent:** Keine Lücken — jeder mögliche Wert muss einem Bin zugewiesen werden können
3. **Ausserhalb des Trainingsbereichs:** Sinnvolle Grenzen wählen (z.B. 0 und 100 für Alter), nicht einfach min/max der Trainingsdaten

Für eine neue unbekannte Instanz: Prüfe in welches Bin der Wert fällt (Grenzen inklusiv/exklusiv beachten).

Anwendungsbeispiel (Übungsaufgabe 8c):

| Alter | Equidepth | Equiwidth | |———|———|———| | 8 | Children [0, 11) | young [0, 26) |
 | 10 | Children | young | | 22 | Adults [22, 100] | young [0, 26) | | 45 | Adults | old [45, 100] | | 6
 | Children (ausserhalb, nächster Bin) | young | | 70 | Adults | old |

Equidepth vs. Equiwidth — Unterschied:

- **Equidepth:** Jeder Bin hat *gleich viele Datenpunkte*. Grenzen basieren auf Datenpunkten.
- **Equiwidth:** Jeder Bin hat *gleiche Breite*. Grenzen basieren auf Wertebereich.
- Bei schiefer Verteilung: Equiwidth kann Bins mit sehr wenigen oder sehr vielen Punkten erzeugen.

5.4 Weitere Diskretisierungsmethoden

- **Histogramm-Analyse:** Rekursive Partitionierung basierend auf Histogramm → mehrstufige Konzepthierarchie
- **Clustering:** Clustering-Algorithmus auf Attributwerte anwenden → Cluster = Bins
- **Konzepthierarchien für Nominaldaten:** Verwandte Werte hierarchisch gruppieren (z.B. “Meadow Gold Milk” → “Milk” → “Dairy Products”)

6. Data Reduction

6.1 Curse of Dimensionality

Kernidee: Mit steigender Anzahl Features wächst der Feature-Raum exponentiell — Daten werden spärlicher, Distanzmetriken verlieren ihre Aussagekraft.

Konsequenzen:

- Alle Distanzen konvergieren gegen den gleichen Wert (keine Unterscheidung mehr)
- k-NN, Clustering und andere distanzbasierte Methoden versagen
- Statistische Signifikanz wird schwerer zu erreichen

Lösungen: Dimensionsreduktion, Feature Selection

6.2 Sampling (Numerosity Reduction)

Drei Sampling-Methoden:

SRSWOR (Simple Random Sample Without Replacement):

- Ziehe s Tupel aus N ohne Zurücklegen
- Jedes Tupel gleich wahrscheinlich, kann nur einmal vorkommen

SRSWR (Simple Random Sample With Replacement):

- Wie SRSWOR, aber jedes gezogene Tupel wird zurückgelegt
- Gleiche Tupel können mehrfach in der Stichprobe auftauchen

Stratified Sampling:

1. Datensatz in disjunkte Schichten (Strata) aufteilen
2. Aus jeder Schicht proportional eine SRS ziehen
3. Anteil aus Schicht i : $s_i = \frac{N_i}{N} \cdot s$

Beispiel (Übungsaufgabe 9, $s = 5$, 27 Werte):

- youth (5 Werte): $\frac{5}{27} \cdot 5 \approx 0.93 \rightarrow 1$ Wert
- middle-aged (20 Werte): $\frac{20}{27} \cdot 5 \approx 3.7 \rightarrow 4$ Werte
- senior (2 Werte): $\frac{2}{27} \cdot 5 \approx 0.37 \rightarrow 0$ Werte

Resultat: 1 aus youth + 4 aus middle-aged = 5 Samples

Exam Trap: Bei Stratified Sampling: Anteile müssen auf ganze Zahlen gerundet werden. Gesamtgrösse s muss stimmen. Wenn durch Rundung die Summe nicht s ergibt, grösste Nachkommastelle aufrunden.

6.3 Feature Selection

Zwei Hauptansätze:

Filter-Ansatz (unsupervised):

- Features werden *vor* dem ML-Algorithmus ausgewählt

- Kriterien: statistische Masse (z.B. Varianz, Korrelation, Information Gain, Chi-Quadrat)
- Schnell, algorithmunabhängig

Wrapper-Ansatz (supervised):

- ML-Algorithmus als “Black Box” → suche bestes

Week 11: [Topic Title]

Chapter 1: [Chapter Title]

Core Idea

Content to be filled in during the tutoring session.

Intuition

Why does this concept exist?

When is it used?

Practical Example

Key Rules, Formulas & Definitions

Formula placeholder

$$f(x) = \dots$$

- Definition 1: ...
- Definition 2: ...

Exam Traps

Exam Trap: Placeholder — typical exam mistakes will be listed here.

3-Minute Summary

Tip: Quick recap for last-minute review.

Cheat Sheet

[CHAPTER TITLE] — CHEAT SHEET

Key formulas:

- ...

Key definitions:

- ...

Don't forget:

- ...

Week 12: [Topic Title]

Chapter 1: [Chapter Title]

Core Idea

Content to be filled in during the tutoring session.

Intuition

Why does this concept exist?

When is it used?

Practical Example

Key Rules, Formulas & Definitions

Formula placeholder

$$f(x) = \dots$$

- Definition 1: ...
- Definition 2: ...

Exam Traps

Exam Trap: Placeholder — typical exam mistakes will be listed here.

3-Minute Summary

Tip: Quick recap for last-minute review.

Cheat Sheet

[CHAPTER TITLE] — CHEAT SHEET

Key formulas:

- ...

Key definitions:

- ...

Don't forget:

- ...

Week 13: [Topic Title]

Chapter 1: [Chapter Title]

Core Idea

Content to be filled in during the tutoring session.

Intuition

Why does this concept exist?

When is it used?

Practical Example

Key Rules, Formulas & Definitions

Formula placeholder

$$f(x) = \dots$$

- Definition 1: ...
- Definition 2: ...

Exam Traps

Exam Trap: Placeholder — typical exam mistakes will be listed here.

3-Minute Summary

Tip: Quick recap for last-minute review.

Cheat Sheet

[CHAPTER TITLE] — CHEAT SHEET

Key formulas:

- ...

Key definitions:

- ...

Don't forget:

- ...

Week 14: [Topic Title]

Chapter 1: [Chapter Title]

Core Idea

Content to be filled in during the tutoring session.

Intuition

Why does this concept exist?

When is it used?

Practical Example

Key Rules, Formulas & Definitions

Formula placeholder

$$f(x) = \dots$$

- Definition 1: ...
- Definition 2: ...

Exam Traps

Exam Trap: Placeholder — typical exam mistakes will be listed here.

3-Minute Summary

Tip: Quick recap for last-minute review.

Cheat Sheet

[CHAPTER TITLE] — CHEAT SHEET

Key formulas:

- ...

Key definitions:

- ...

Don't forget:

- ...